Chapter 5 - Network Layer: Control Plane

The slides are derived, and adapted by the teachers, from slides made available to the authors of the following texts

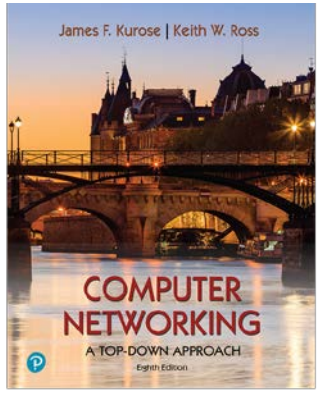
A note on the use of these PowerPoint slides:
We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

For a revision history, see the slide note for this page.

Thanks and enjoy! JFK/KWR

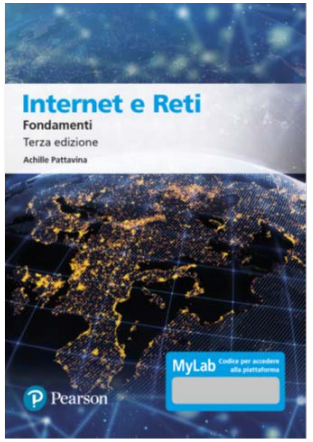
All material copyright 1996-2020
J.F Kurose and K.W. Ross, All Rights Reserved



Computer Networking: A Top-Down Approach
8th edition
Jim Kurose, Keith Ross
Pearson, 2020

- These slides are freely made available to all colleagues willing to deliver a course based on the book Internet and Computer Networks (Internet e Reti, Pearson, 2022).
- Feel free to add/remove/modify any slide.
  It is just asked to keep the copyright indication in the first slide of the slide deck you will use

© 2018-2022 Achille Pattavina



Achille Pattavina
Internet e Reti
Fondamenti - Terza edizione, Pearson, 2022

Network layer control plane: our goals

- understand principles behind network control plane:
  - traditional routing algorithms
  - SDN controllers
  - network management, configuration
- instantiation, implementation in the Internet:
  - OSPF, BGP
  - OpenFlow, ODL and ONOS controllers
  - SNMP, YANG/NETCONF

Network layer: “control plane” roadmap

- introduction
- routing protocols
  - link state
  - distance vector
- intra-ISP routing:
  - RIP
  - OSPF
- routing among ISPs: BGP
- SDN control plane

- network management, configuration
  - SNMP
  - NETCONF/YANG

Network-layer functions

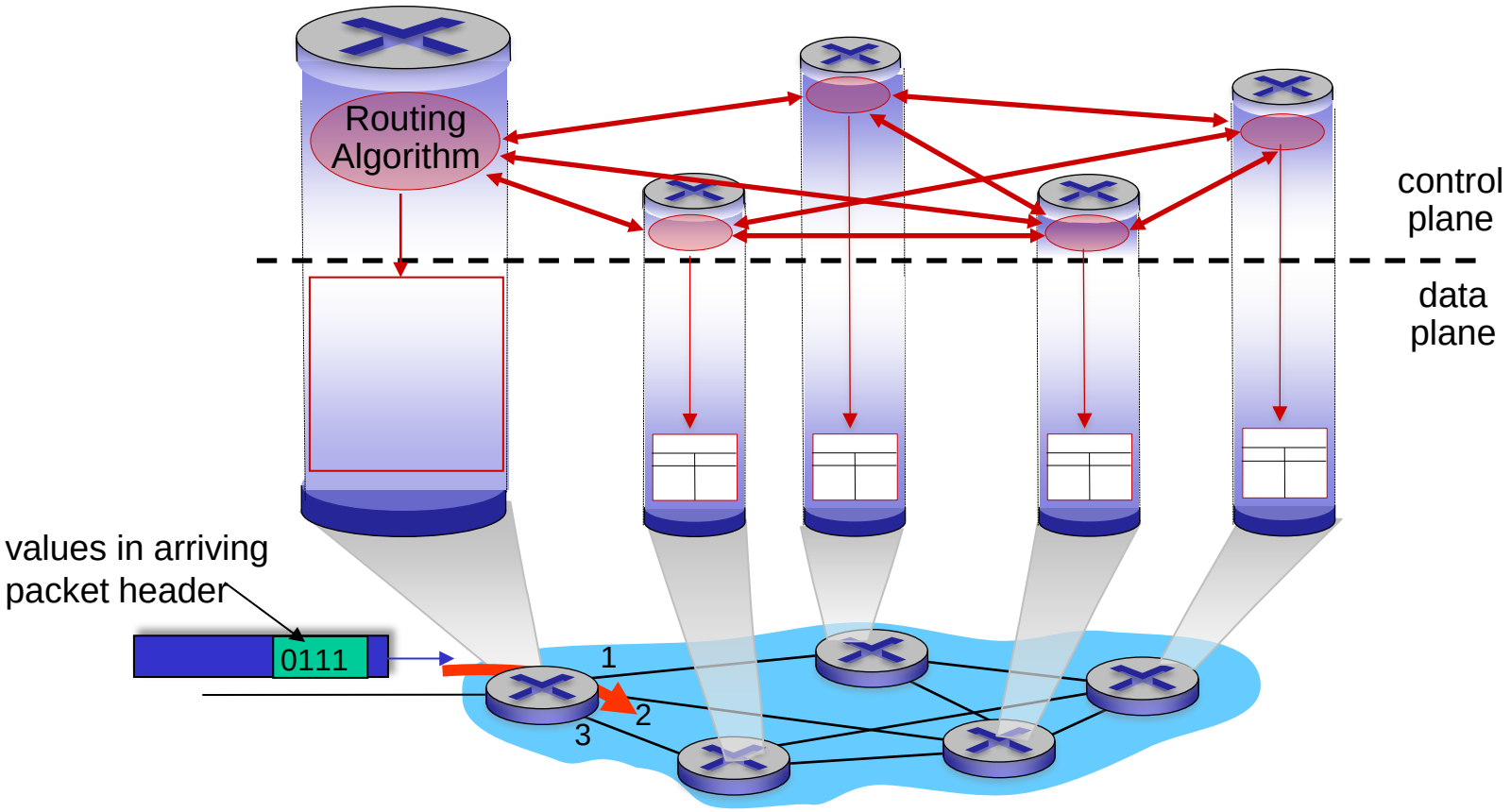
- forwarding: move packets from router's input to appropriate router output — *data plane*
- routing: determine route taken by packets from source to destination — *control plane*

Two approaches to structuring network control plane:

- per-router control (traditional)
- logically centralized control (software defined networking)

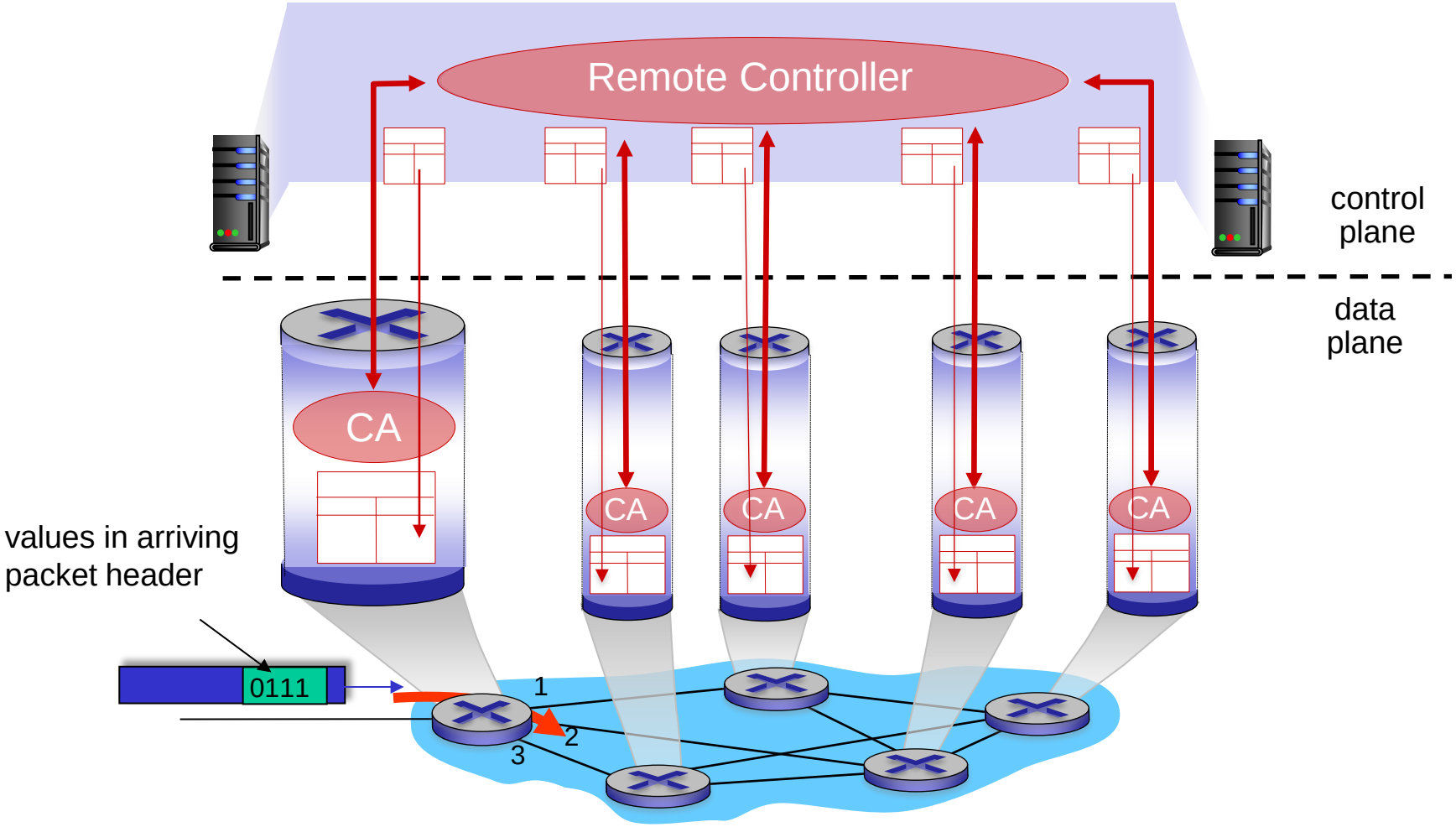
Per-router control plane

Individual routing algorithm components *in each and every router* interact in the control plane



Software-Defined Networking (SDN) control plane

Remote controller computes, installs forwarding tables in routers

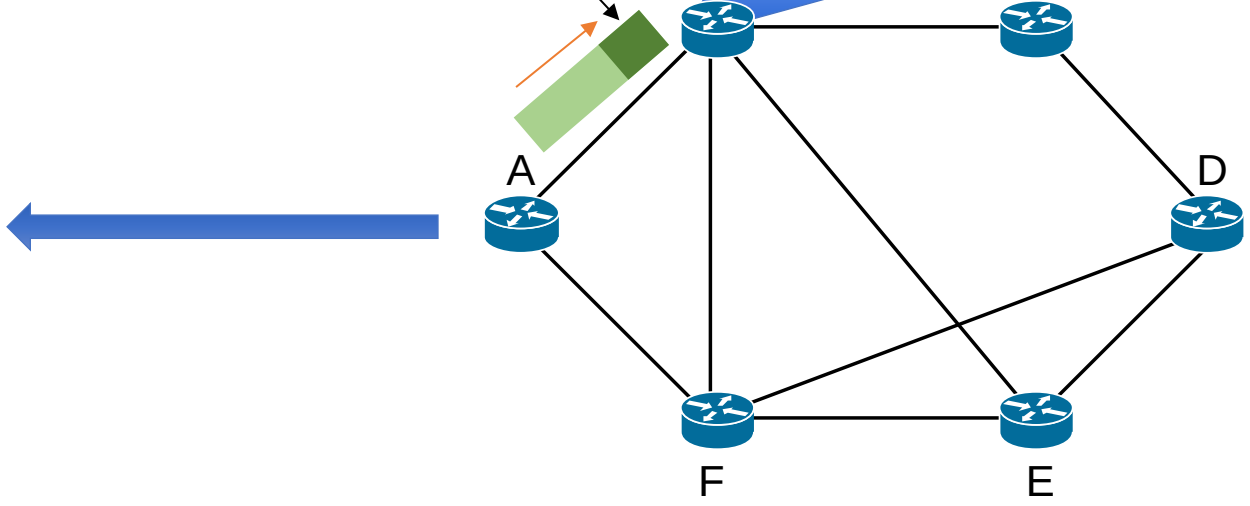


Routing & forwarding

- The *forwarding table* determines the local forwarding of IUs to adjacent routers
- The *routing algorithm* determines the end-to-end path through the network
  - It fills up the node forwarding table

Forwarding table

Destination address	Next hop
Network address 1	C
Network address 2	E
Network address 3	C
Network address 4	F



Internet protocol: *Routing*

A routing protocol defines a method to select the path based on

- A metric and a metric weighting to label the inter-router path
- The transfer mode (type, content, rate) of the update information

Possible routing metrics

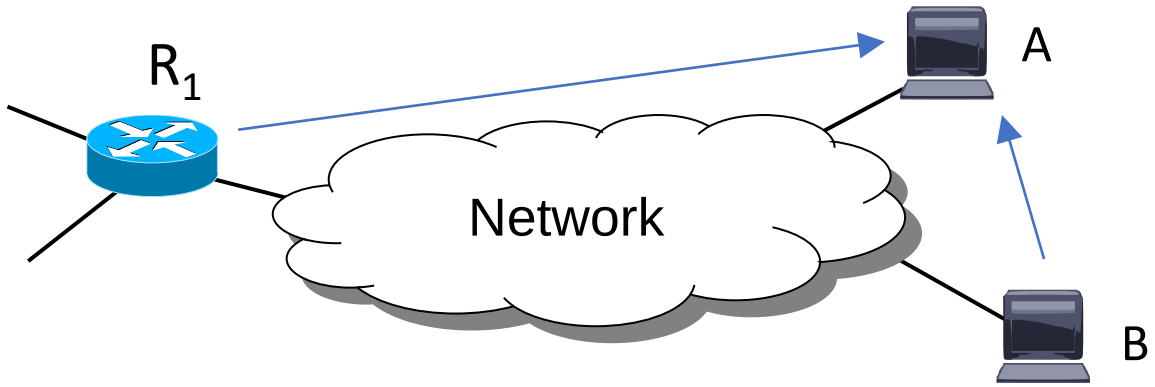
- Delay
- Hop count
- Bandwidth/Carried load
- Reliability

Routing types

- Direct
  - Destination net-id and sending host/router net-id are the same
  - Subnetwork mask used
- Indirect
  - Destination net-id and sending host/router net-id are different
    - Host/router determines which interfaced subnetwork to use and which next hop router to reach

Internet protocol: *Routing*

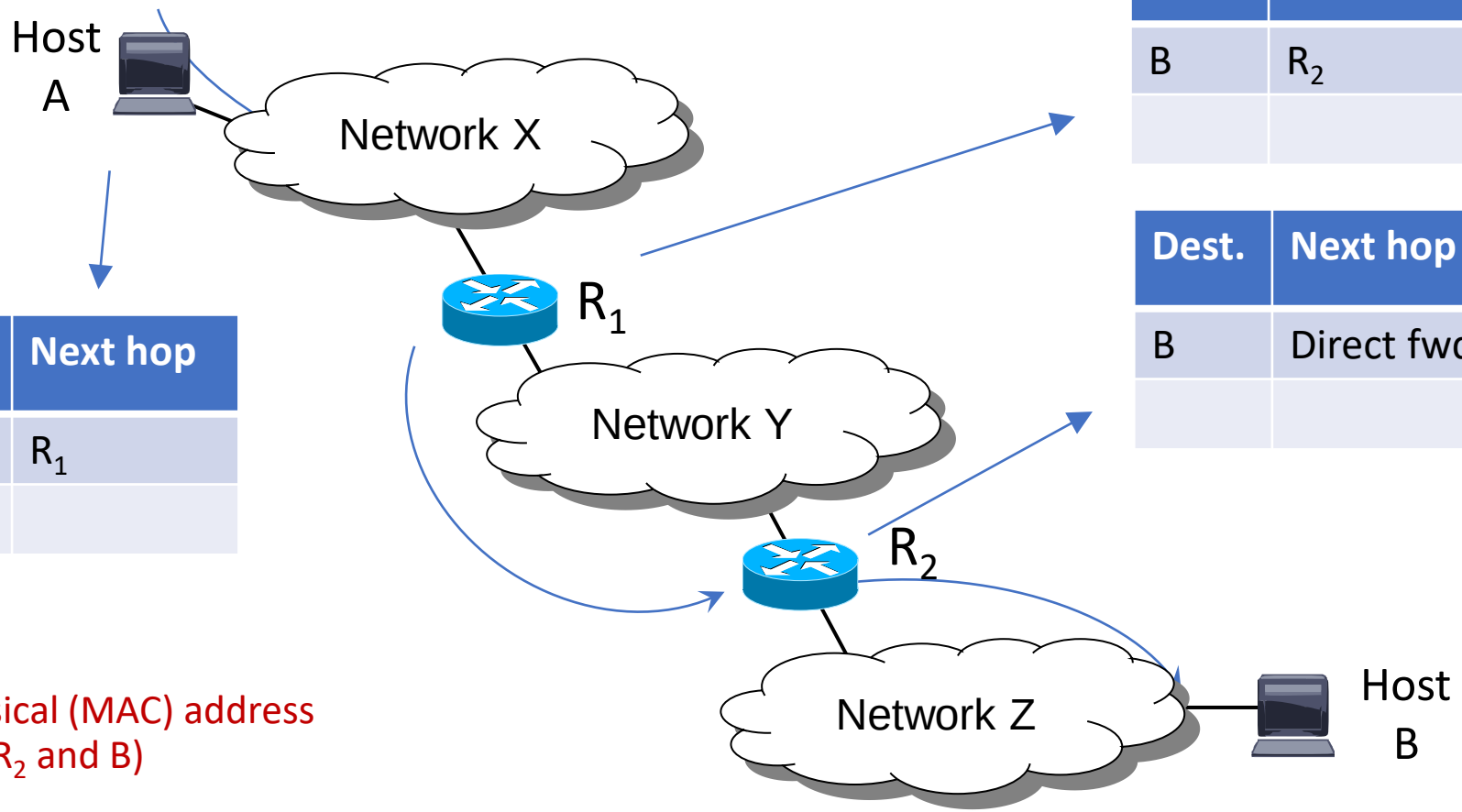
Direct forwarding



Note: R_1 and B MUST know A's physical (MAC) address

Internet protocol: *Routing*

Indirect forwarding (before LAST direct routing)



Dest.	Next hop
B	R_1

Dest.	Next hop
B	R_2

Dest.	Next hop
B	Direct fwd

Note: A, R_1 and R_2 MUST know physical (MAC) address of LOCAL intended destination (R_1, R_2 and B)

Direct/indirect forwarding: *Using netmask*

Host netmask is used to determine if either direct or indirect forwarding is needed

- *Destination network*: AND between *destination* IP address and netmask
- *Source network*: AND between host *source* IP address and netmask
  - POSITIVE match: direct forwarding
  - NEGATIVE match: indirect forwarding using a router

Destination network:
131.175.55.68
255.255.255.192

→ 131.175.55.64

AND

Source network:
131.175.55.111
255.255.255.192

→ 131.175.55.64

↕ Positive → Direct forwarding

Note: a physical (MAC) address of destination *host* is needed

Direct/indirect forwarding: *Using netmask*

Host netmask is used to determine if either direct or indirect forwarding is needed

- *Destination network*: AND between *destination* IP address and netmask
- *Source network*: AND between host *source* IP address and netmask
  - POSITIVE match: direct forwarding
  - NEGATIVE match: indirect forwarding using a router

Destination network:
131.175.99.160
255.255.255.192

→ 131.175.99.128

AND

Source network:
131.175.55.111
255.255.255.192

→ 131.175.55.64

↕ Negative → Indirect forwarding

Note: a physical (MAC) address of *next hop router* is needed

Router: *Packet forwarding*

Router: internetworking device with multiple output interfaces

Forwarding type

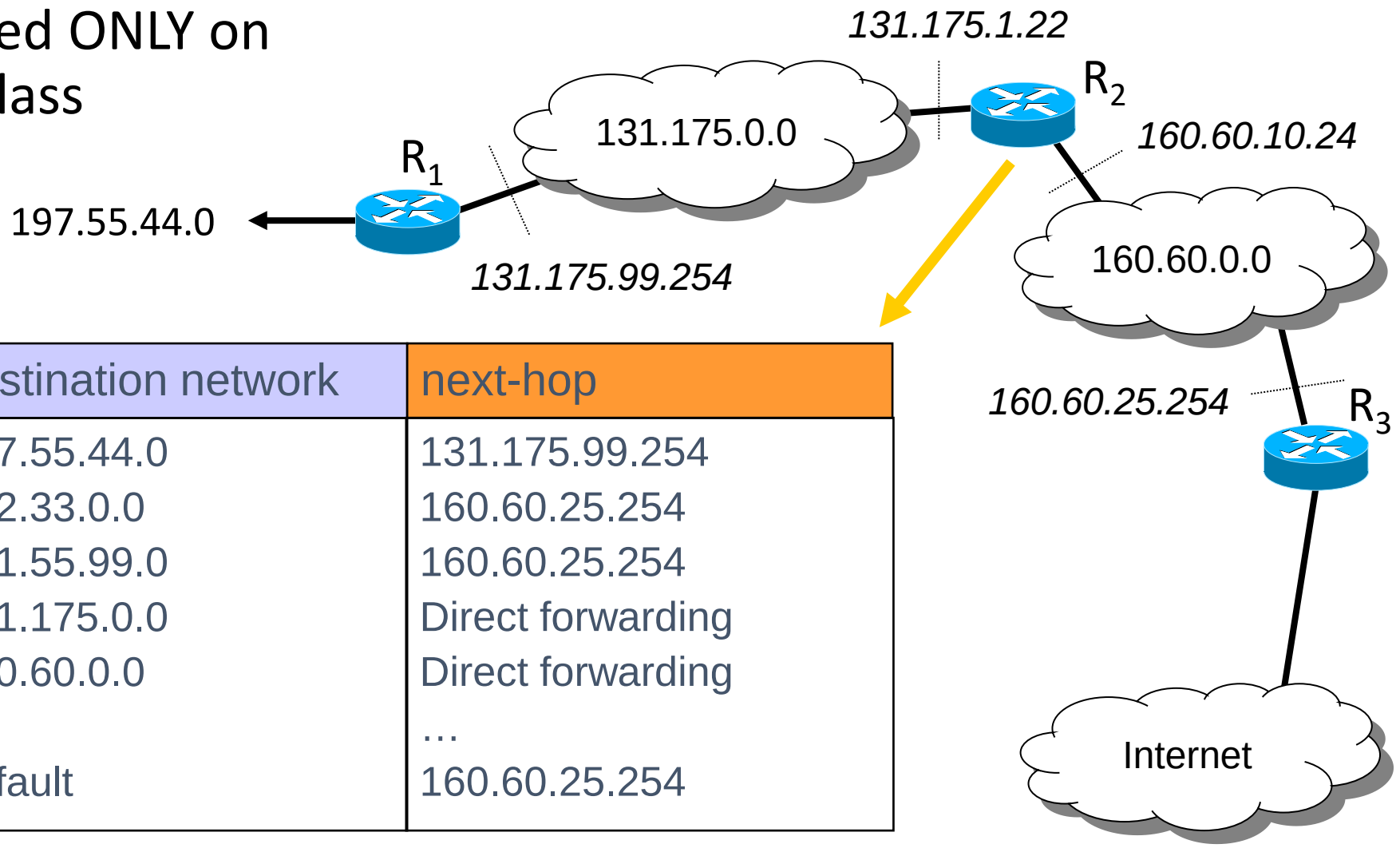
- *Destination based*: *direct* routing as for hosts
  - Only destination interface is specified
- *Next-hop: indirect* routing to another intermediate router
  - Based on routing tables indicating ONLY *next hop router* and interface to be used

Routing table

- A routing protocol determines all the *minimum-cost* paths from source to destination
  - Objective: maximum utilization of overall network capacity
- The routing table is dynamically filled using the results of a specific routing protocol

Packet forwarding: *Classful addressing*

Routing based ONLY on IP address class



destination network	next-hop
197.55.44.0	131.175.99.254
142.33.0.0	160.60.25.254
201.55.99.0	160.60.25.254
131.175.0.0	Direct forwarding
160.60.0.0	Direct forwarding
…	…
default	160.60.25.254

Packet forwarding: *Longest prefix match*

What happens if multiple entries of the routing table match the IP destination address?

- "Longest prefix match" forwarding algorithm is applied

IP packet
Destination: 20.8.16.5

Longest match = smaller network

Routing table		
20.0.0.0	/8	Route 1
20.8.0.0	/16	Route 2
20.8.16.0	/24	Route 3

Three matches

Best (longest) match

Packet forwarding: *Classless addressing*

Classless addressing

- Completely eliminates traditional concepts of Class A, B, C addresses

Network prefix based

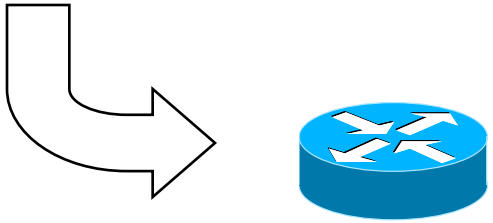
- Routers do not make any assumption on the basis of the three leading bits
- They require an explicit network prefix to determine division point between net-id and host-id
- Capability of advertise prefix must be supported by routing protocol (e.g. BGPv4)

Router requirements: same as with VLSM (but on a worldwide scale)

- Routing protocol (e.g. BGPv4) must carry network prefix information with each route advertising
- All routers must implement a consistent forwarding algorithm based on the "*longest prefix match*"

Packet forwarding: *Longest prefix match*

IP packet
Destination: 150.23.99.15
10010110 . 00010111 . 01100011 . 00001111



Routing table	
150.0.0.0/10	Route 1
150.20.0.0/14	Route 2
150.23.96.0/21	Route 3

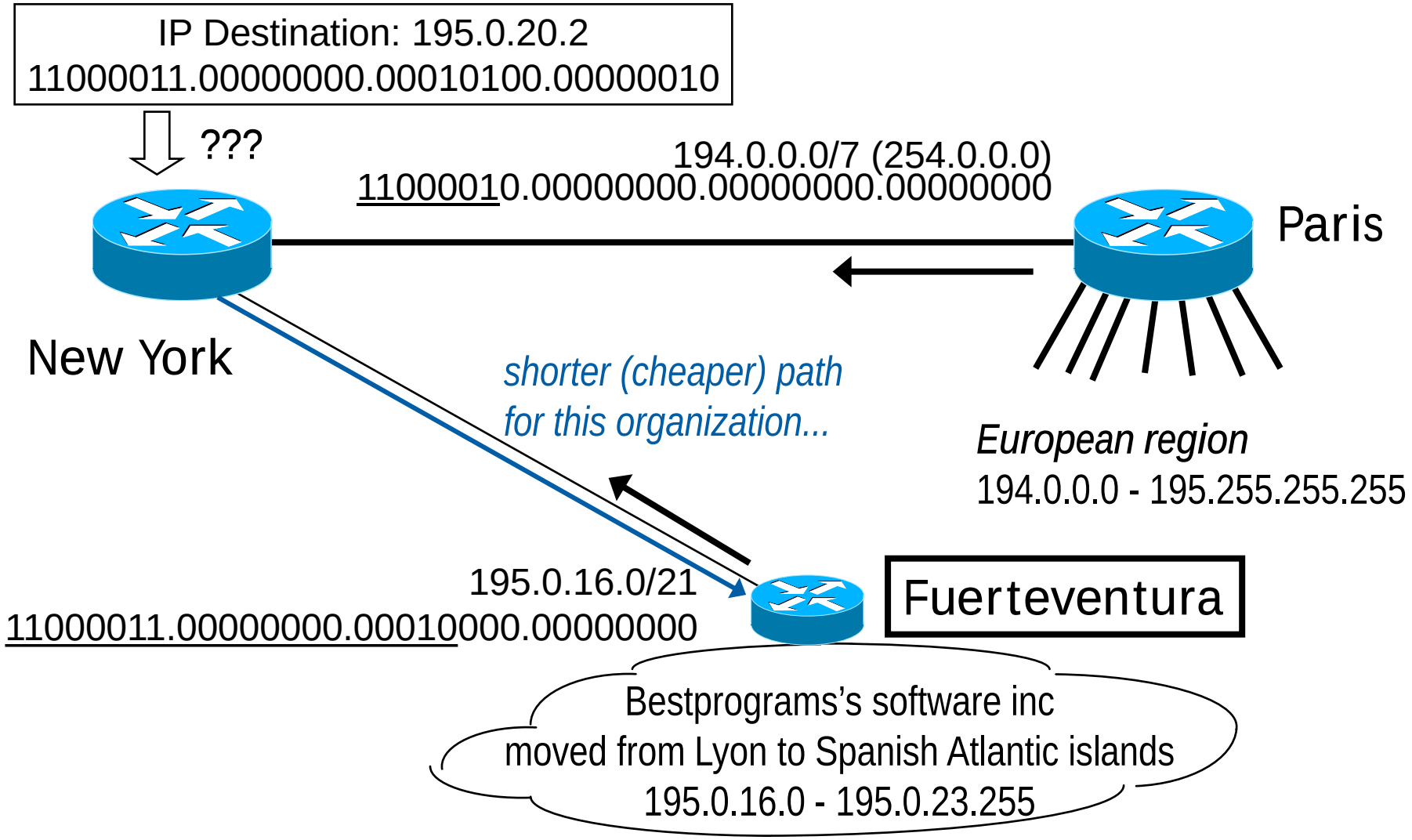
Three matches

Best (longest) match

R1: 10010110 . 00010111 . 01100011 . 00001111
R2: 10010110 . 00010111 . 01100011 . 00001111
R3: 10010110 . 00010111 . 01100011 . 00001111
Longest match(R3) = smaller network

But why longest match is ever needed???

IP packet forwarding: *Exception route*



Packet forwarding: *Using netmask*

Destination address compared bit-by-bit with routing table entries

- Comparison limited to net-id (1-bit of netmask)
- If different entries match, entry with larger number of 1s in netmask is selected (longest prefix match - LPM)

Mandatory use of netmask

- in Internet when CIDR is adopted
- In private networks when VLSM is adopted

Packet forwarding: *Using netmask*

IP destination address

131.175.88.2

Interface	IP address	Netmask	
i_0	131.175.11.1	255.255.255.0	No
i_1	131.175.88.254	255.255.255.0	Yes

131.175.88.2

Destination network	Netmask	Next hop router
131.175.22.0	255.255.255.0	131.175.11.254
131.175.33.0	255.255.255.0	131.175.11.253
131.175.44.0	255.255.255.0	131.175.11.254
131.175.55.0	255.255.255.0	131.175.11.253
131.176.0.0	255.255.0.0	131.175.77.11
131.176.66.0	255.255.255.0	131.175.77.22
131.176.77.0	255.255.255.0	131.175.77.33
0.0.0.0	0.0.0.0	131.175.11.254

Packet forwarding: Using netmask

IP destination address

131.175.44.2

Interface	IP address	Netmask	
i_0	131.175.11.1	255.255.255.0	No
i_1	131.175.88.254	255.255.255.0	No

Destination network	Netmask	Next hop router	
131.175.22.0	255.255.255.0	131.175.11.254	No
131.175.33.0	255.255.255.0	131.175.11.253	No
131.175.44.0	255.255.255.0	131.175.11.254	Yes ← 131.175.44.2
131.175.55.0	255.255.255.0	131.175.11.253	No
131.176.0.0	255.255.0.0	131.175.77.11	No
131.176.66.0	255.255.255.0	131.175.77.22	No
131.176.77.0	255.255.255.0	131.175.77.33	No
0.0.0.0	0.0.0.0	131.175.11.254	Yes

Packet forwarding: *Using netmask*

IP destination address

131.176.66.44

Interface	IP address	Netmask	
i_0	131.175.11.1	255.255.255.0	No
i_1	131.175.88.254	255.255.255.0	No

Destination network	Netmask	Next hop router	
131.175.22.0	255.255.255.0	131.175.11.254	No
131.175.33.0	255.255.255.0	131.175.11.253	No
131.175.44.0	255.255.255.0	131.175.11.254	No
131.175.55.0	255.255.255.0	131.175.11.253	No
131.176.0.0	255.255.0.0	131.175.77.11	Yes
131.176.66.0	255.255.255.0	131.175.77.22	Yes
131.176.77.0	255.255.255.0	131.175.77.33	No
0.0.0.0	0.0.0.0	131.175.11.254	Yes

131.176.66.44

IP addressing: *Router configuration*

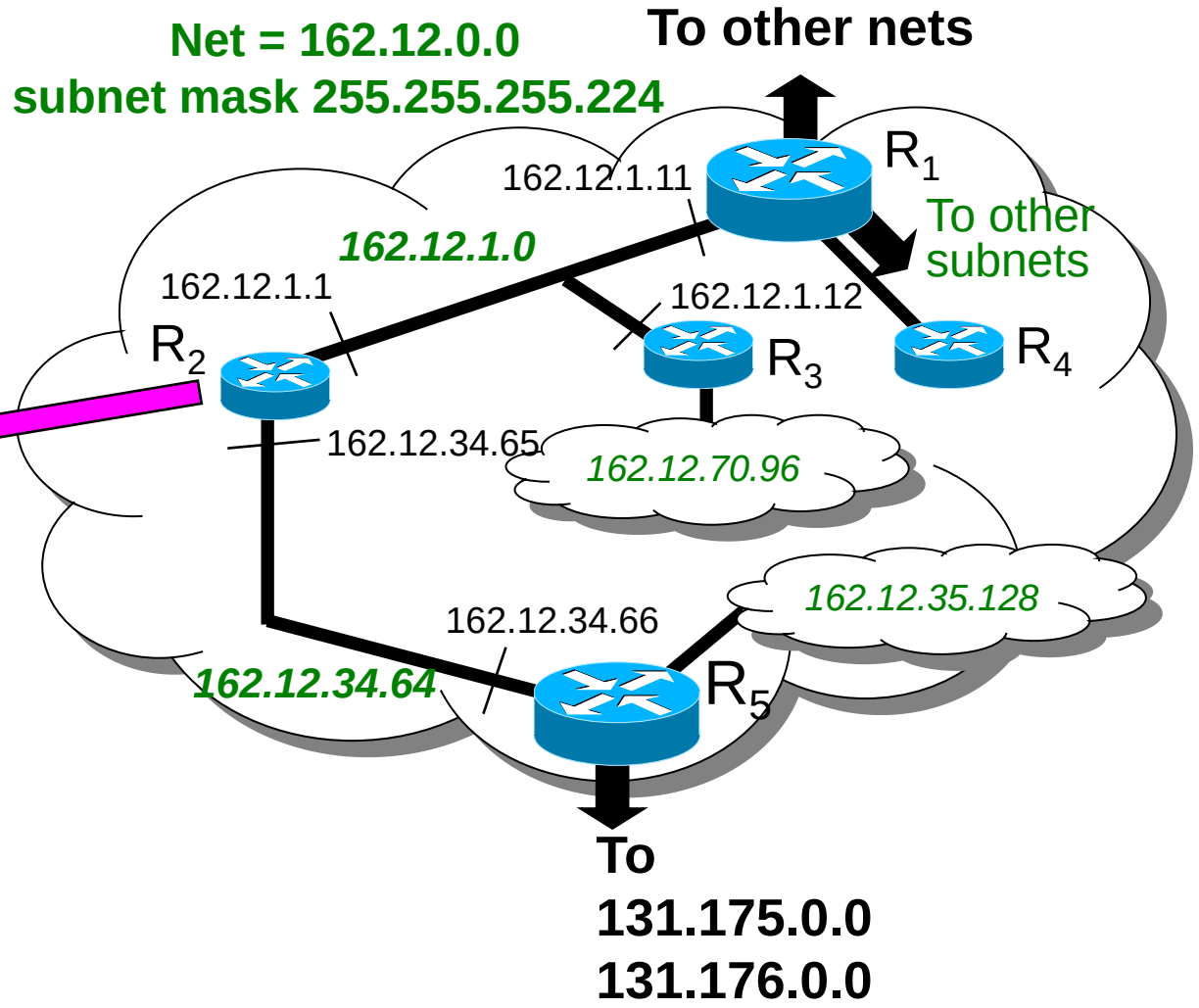
Classful routing

All necessary information included in IP address

Subnet routing

Required specific subnet mask (set by admin)

R2 routing table	
Subnet mask: 255.255.255.224	
dest	Next hop
162.12.1.0	Direct fwd
162.12.34.64	Direct fwd
162.12.35.128	162.12.34.66
162.12.70.96	162.12.1.12
131.175.0.0	162.12.34.66
131.176.0.0	162.12.34.66
default	162.12.1.11



Routing table reduction

Reduction of routing tables obtained through address aggregation

- Aggregation of adjacent network addresses with same NH router into larger blocks with «shorter» net-ID
  - Taking into account of «route exception» in the aggregated block
- Exploit availability of delault route for other addresses with same NH router

Routing table reduction: *Example*

Initial routing table

Entry	Destination network	Netmask	Next hop router
1	0.0.0.0	0.0.0.0	131.175.44.66
2	50.72.0.0	255.255.0.0	131.175.20.39
3	50.73.0.0	255.255.0.0	131.175.20.39
4	50.74.0.0	255.255.0.0	131.175.52.92
5	50.75.0.0	255.255.0.0	131.175.20.39
6	160.192.12.0	255.255.255.0	131.175.44.66
7	200.10.20.64.	255.255.255.224	131.175.32.20
8	200.10.20.96	255.255.255.224	131.175.32.20

00110010.01001000.**0**.**0**
00110010.01001001.**0**.**0**
00110010.01001010.**0**.**0**
00110010.01001011.**0**.**0**

Reduced routing table

Entry	Destination network	Netmask	Next hop router	Aggregated entries
A	0.0.0.0	0.0.0.0	131.175.44.66	1, 6
B	50.72.0.0	255.252.0.0	131.175.20.39	2, 3, 5
C	50.74.0.0	255.255.0.0	131.175.52.92	4
D	200.10.20.64	255.255.255.192	131.175.32.20	7, 8

Network layer: “control plane” roadmap

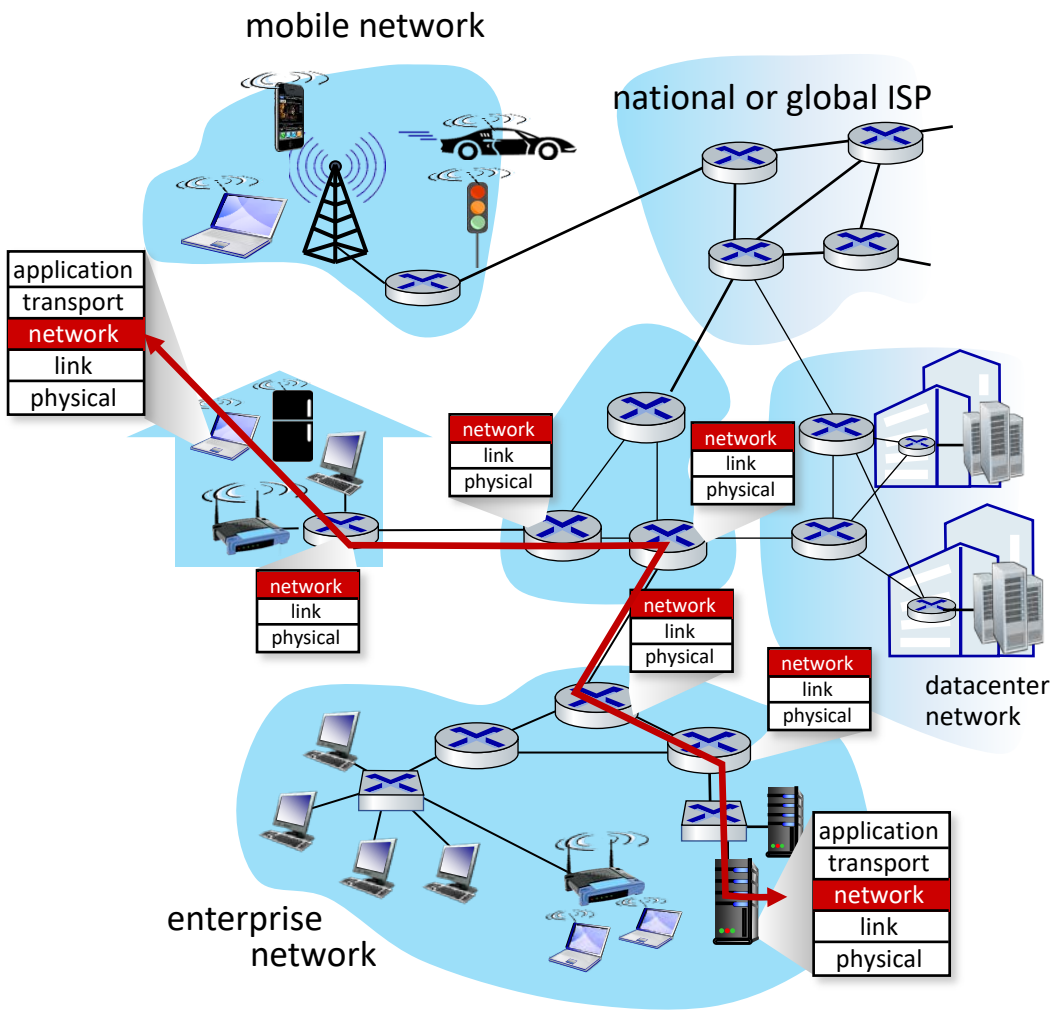
- introduction
- **routing protocols**
  - **link state**
  - **distance vector**
- intra-ISP routing:
  - RIP
  - OSPF
- routing among ISPs: BGP
- SDN control plane

- network management, configuration
  - SNMP
  - NETCONF/YANG

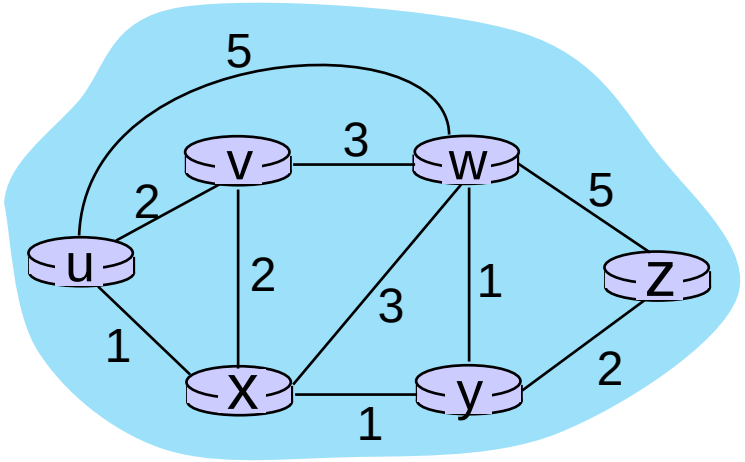
Routing protocols

Routing protocol goal: determine "good" paths (equivalently, routes), from sending hosts to receiving host, through network of routers

- path: sequence of routers packets traverse from given initial source host to final destination host
- "good": least "cost", "fastest", "least congested"
- routing: a "top-10" networking challenge!



Graph abstraction: link costs



$c_{a,b}$: cost of *direct* link connecting *a* and *b*

e.g., $c_{w,z} = 5$, $c_{u,z} = \infty$

cost defined by network operator: could always be 1, or inversely related to bandwidth, or inversely related to congestion

graph: $G = (N,E)$

N: set of routers = { *u, v, w, x, y, z* }

E: set of links ={ *(u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z)* }

Routing algorithm classification

How fast do routes change?

static: routes change slowly over time

dynamic: routes change more quickly

- periodic updates or in response to link cost changes

global: all routers have *complete* topology, link cost info

- "link state" algorithms

decentralized: iterative process of computation, exchange of info with neighbors

- routers initially only know link costs to attached neighbors
- "distance vector" algorithms

global or decentralized information?

Network layer: “control plane” roadmap

- introduction
- routing protocols
  - **link state**
  - distance vector
- intra-ISP routing:
  - RIP
  - OSPF
- routing among ISPs: BGP
- SDN control plane

- network management, configuration
  - SNMP
  - NETCONF/YANG

Dijkstra's link-state routing algorithm

- centralized: network topology, link costs known to *all* nodes
  - accomplished via "link state broadcast"
  - all nodes have same info
- computes least cost paths from one node ("source") to all other nodes
  - gives *forwarding table* for that node
- iterative: after *k* iterations, know least cost path to *k* destinations

notation

- $c_{x,y}$: direct link cost from node *x* to *y*; $= \infty$ if not direct neighbors
- $D(v)$: *current* estimate of cost of least-cost-path from source to destination *v*
- $p(v)$: predecessor node along path from source to *v*
- N': set of nodes whose least-cost-path *definitively* known

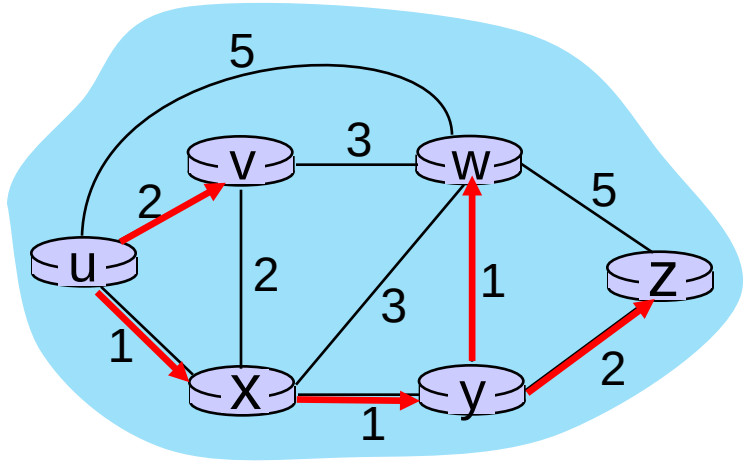
Dijkstra's link-state routing algorithm

```
Initialization:
  N' = {u}                          /* compute least cost path from u to all other nodes */
  for all nodes v
    if v adjacent to u              /* u initially knows direct-path-cost only to direct neighbors */
        then D(v) = c_{u,v}         /* but may not be minimum cost! */
    else D(v) = ∞

Loop
    find w not in N' such that D(w) is a minimum
    add w to N'
    update D(v) for all v adjacent to w and not in N' :
        D(v) = min ( D(v), D(w) + c_{w,v} )
    /* new least-path-cost to v is either old least-cost-path to v or known
    least-cost-path to w plus direct-cost from w to v */
until all nodes in N'
```

Dijkstra's algorithm: an example

Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					

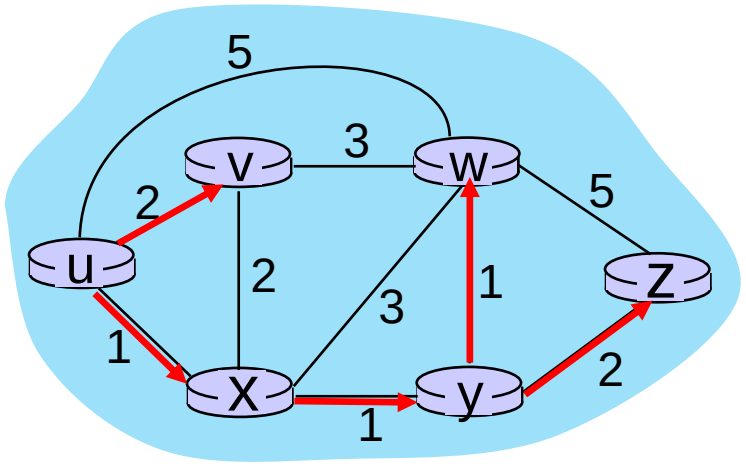


Initialization (step 0): For all a: if *a* adjacent to then $D(a) = c_{u,a}$

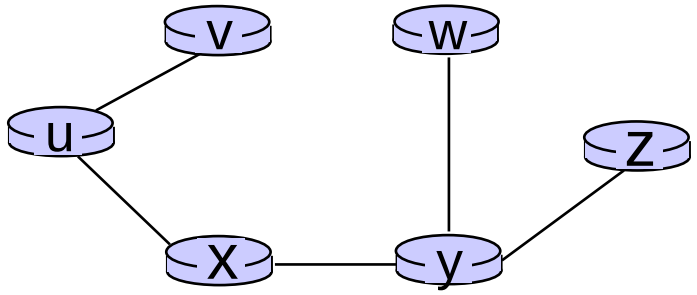
find *a* not in *N'* such that *D(a)* is a minimum
add *a* to *N'*
update *D(b)* for all *b* adjacent to *a* and not in *N'* :

$$D(b) = \min\,(\,D(b),\, D(a) + c_{a,b}\,)$$

Dijkstra's algorithm: an example



resulting least-cost-path tree from u:



resulting forwarding table in u:

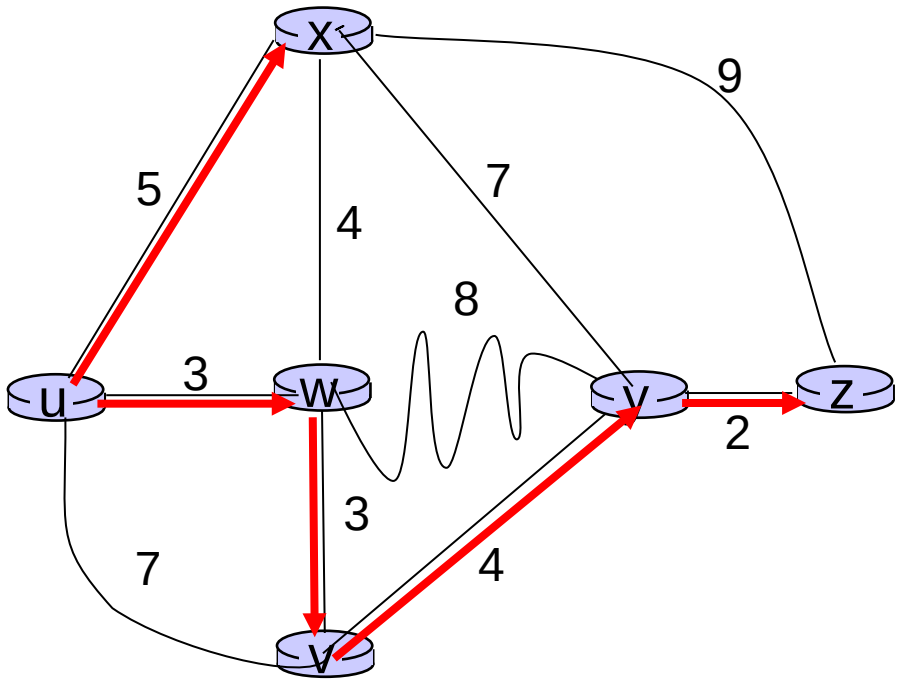
destination	outgoing link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
x	(u,x)

route from *u* to *v* directly

route from u to all other destinations via *x*

Dijkstra's algorithm: another example

Step	*N'*	*v* D(v), p(v)	*w* D(w), p(w)	*x* D(x), p(x)	*y* D(y), p(y)	*z* D(z), p(z)
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwx	6,w			11,w	14,x
3	uwxv				10,v	14,x
4	uwxvy					12,y
5	uwxvyz					



notes:

- construct least-cost-path tree by tracing predecessor nodes
- ties can exist (can be broken arbitrarily)

Dijkstra's algorithm: discussion

algorithm complexity: *n* nodes

- each of *n* iteration: need to check all nodes, *w*, not in *N*
- $n(n+1)/2$ comparisons: $O(n^2)$ complexity
- more efficient implementations possible: $O(n\log n)$

message complexity:

- each router must *broadcast* its link state information to other *n* routers
- efficient (and interesting!) broadcast algorithms: $O(n)$ link crossings to disseminate a broadcast message from one source
- each router's message crosses $O(n)$ links: overall message complexity: $O(n^2)$

Network layer: “control plane” roadmap

- introduction
- routing protocols
  - link state
  - **distance vector**
- intra-ISP routing:
  - RIP
  - OSPF
- routing among ISPs: BGP
- SDN control plane

- network management, configuration
  - SNMP
  - NETCONF/YANG

Distance vector algorithm

Based on *Bellman-Ford* (BF) equation (dynamic programming):

Bellman-Ford equation

Let $D_x(y)$: cost of least-cost path from x to y.
Then:

$$D_x(y) = \min_v \{ c_{x,v} + D_v(y) \}$$

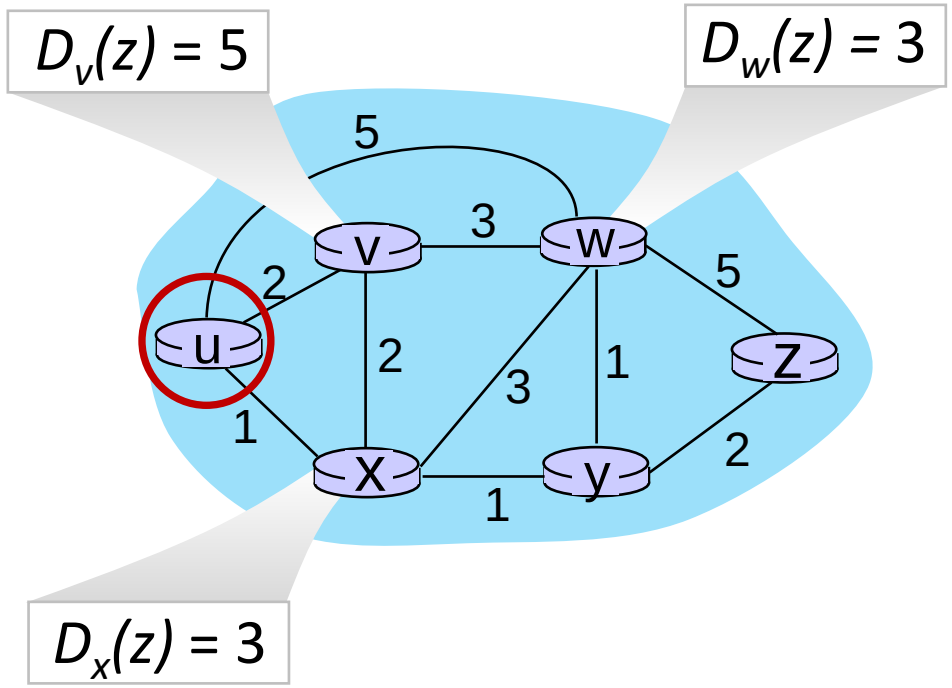
min taken over all neighbors v of x

direct cost of link from x to v

v's estimated least-cost-path cost to y

Bellman-Ford Example

Suppose that *u*'s neighboring nodes, *x,v,w,* know that for destination *z*:



Bellman-Ford equation says:

$$
\begin{aligned}
D_u(z) &= \min \{ c_{u,v} + D_v(z), \\
&\qquad c_{u,x} + D_x(z), \\
&\qquad c_{u,w} + D_w(z) \} \\
&= \min \{2 + 5, \\
&\qquad 1 + 3, \\
&\qquad 5 + 3\} = 4
\end{aligned}
$$

node achieving minimum (x) is next hop on estimated least-cost path to destination (z)

Distance vector algorithm

key idea:

- from time-to-time, each node sends its own distance vector estimate to neighbors
- when *x* receives new DV estimate from any neighbor, it updates its own DV using B-F equation:

$$D_x(y) \leftarrow min_v\{c_{x,v} + D_v(y)\} \text{ for each node } y \in N$$

- under minor, natural conditions, the estimate $D_x(y)$ *converge to the actual least cost* $d_x(y)$

Distance vector algorithm:

each node:

wait for (change in local link cost or msg from neighbor)

recompute DV estimates using DV received from neighbor

if DV to any destination has changed, *notify* neighbors

iterative, asynchronous: each local iteration caused by:

- local link cost change
- DV update message from neighbor

distributed, self-stopping: each node notifies neighbors *only* when its DV changes

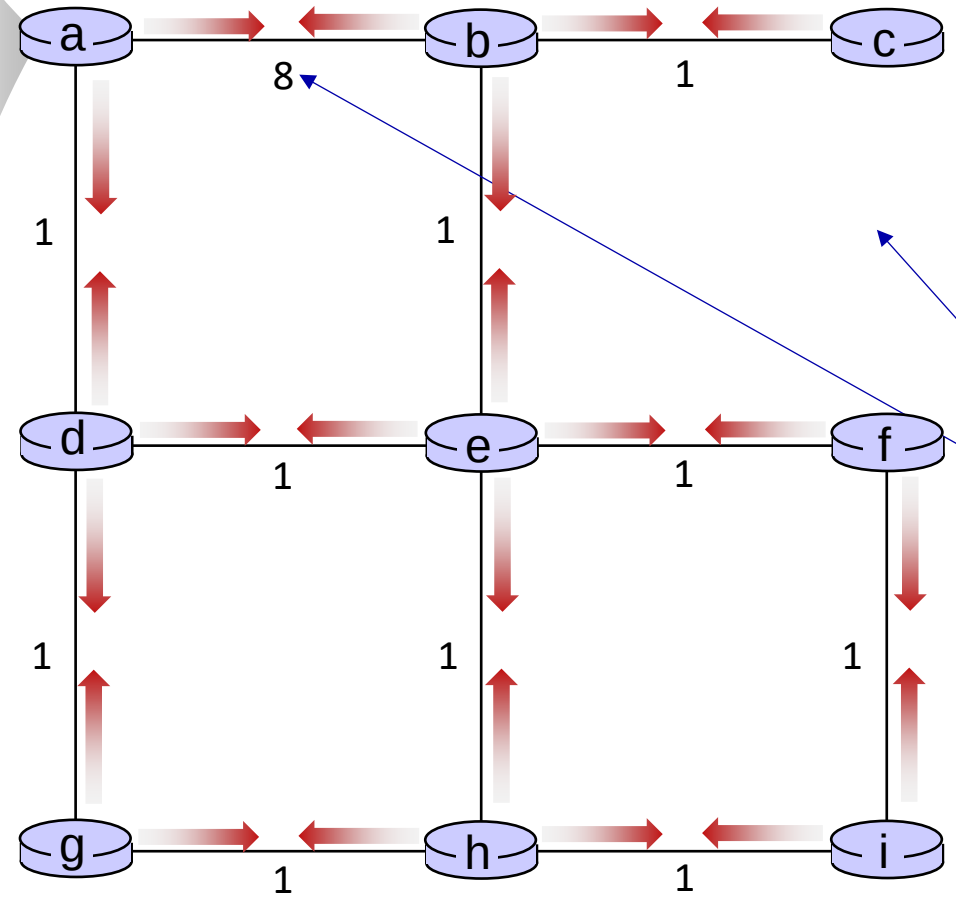
- neighbors then notify their neighbors – *only if necessary*
- no notification received, no actions taken!

Distance vector: example

t=0

- All nodes have distance estimates to nearest neighbors (only)
- All nodes send their local distance vector to their neighbors

DV in a:
$D_a(a)=0$
$D_a(b) = 8$
$D_a(c) = \infty$
$D_a(d) = 1$
$D_a(e) = \infty$
$D_a(f) = \infty$
$D_a(g) = \infty$
$D_a(h) = \infty$
$D_a(i) = \infty$



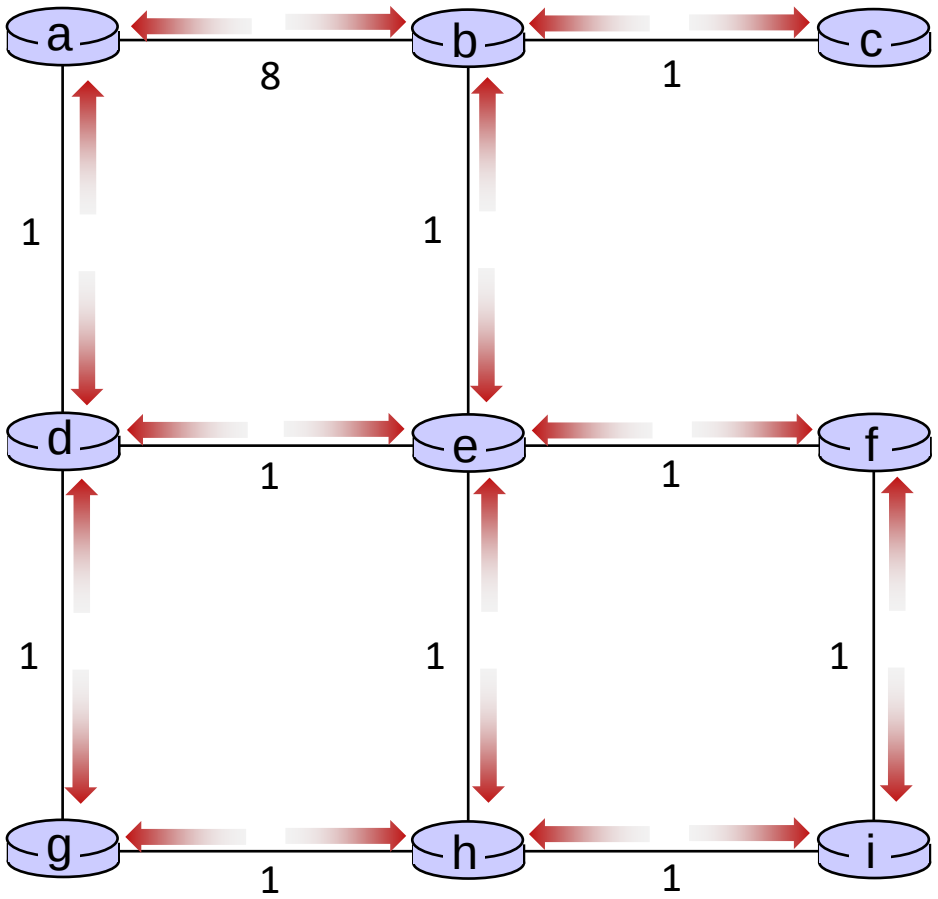
A few asymmetries:
- missing link
- larger cost

Distance vector example: iteration

t=1

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors

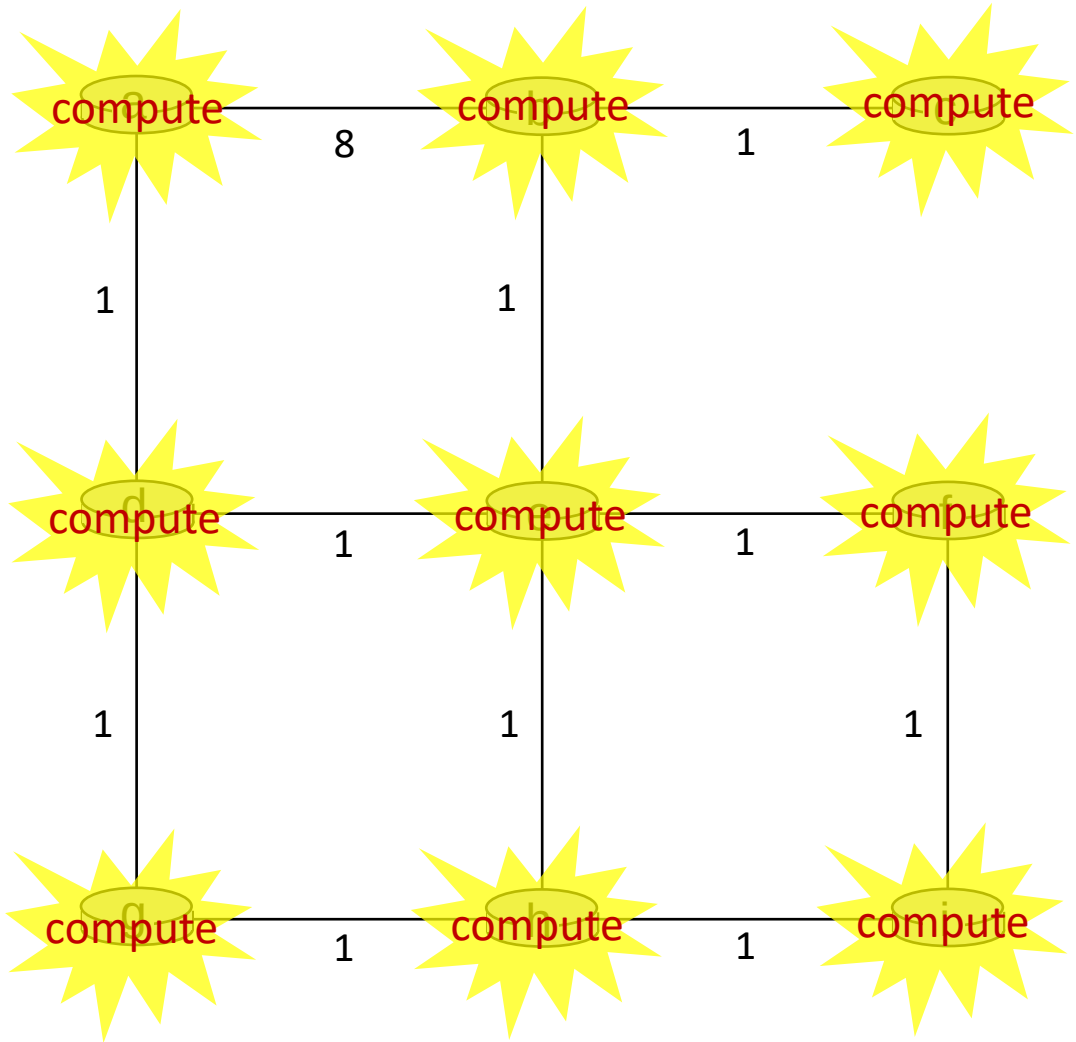


Distance vector example: iteration

t=1

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors

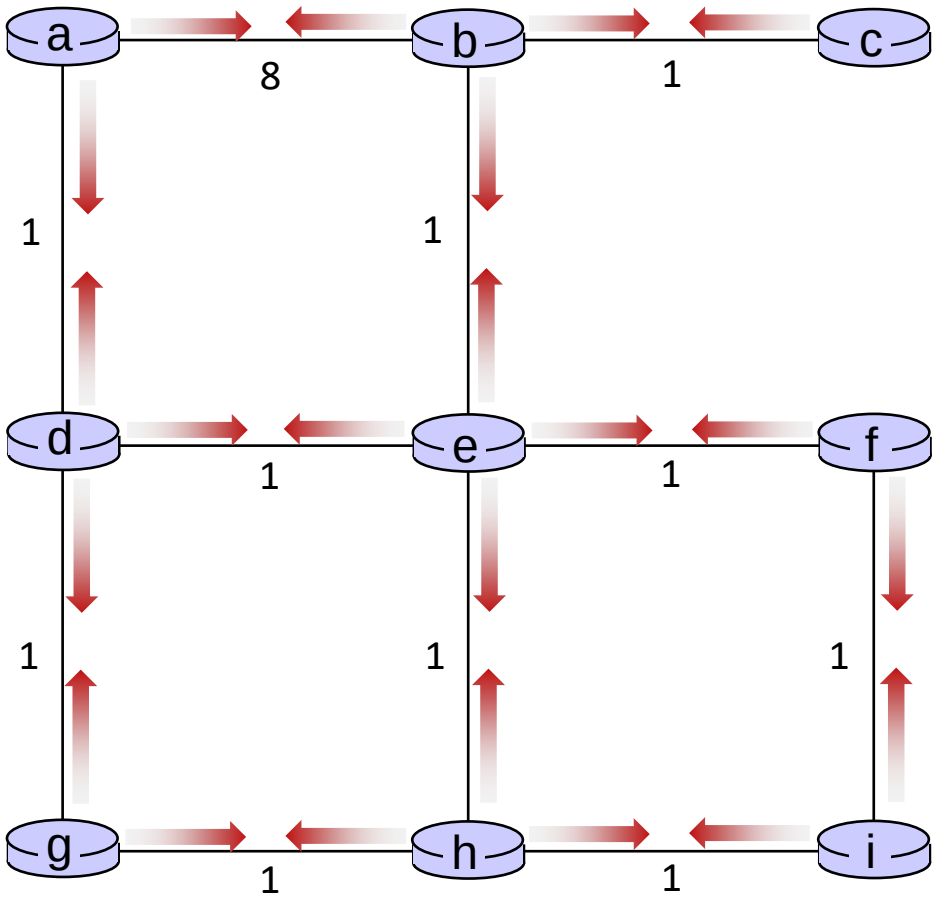


Distance vector example: iteration

t=1

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors

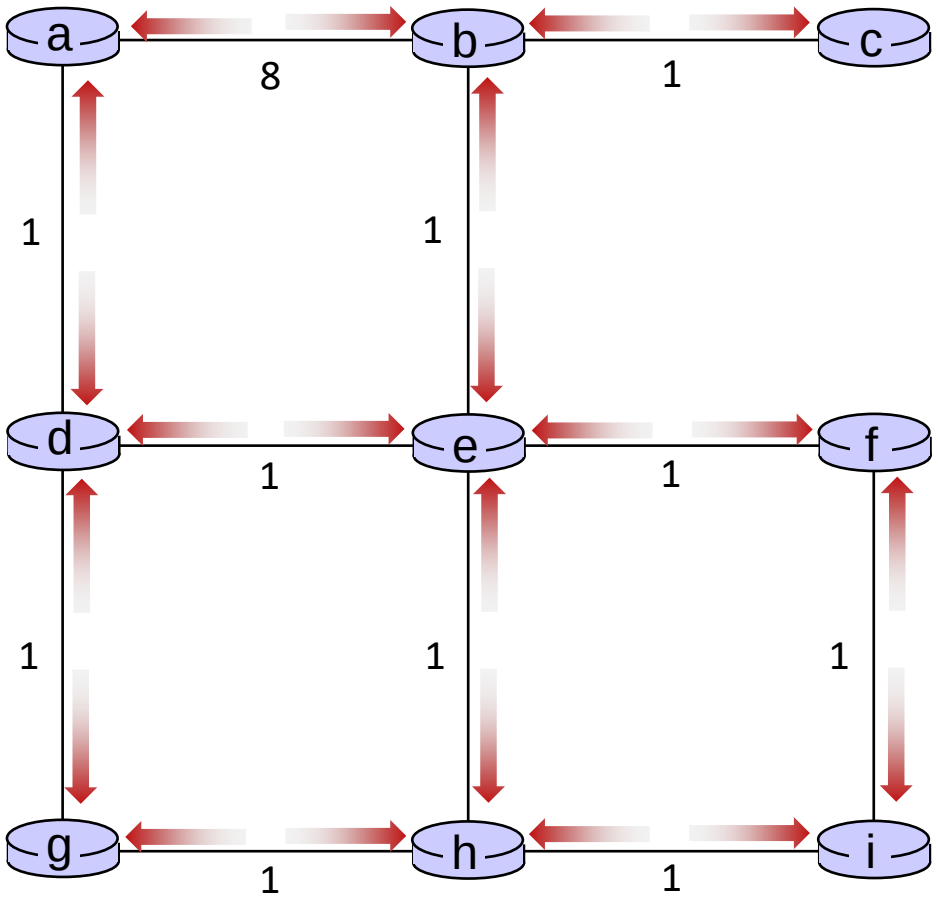


Distance vector example: iteration

t=2

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors

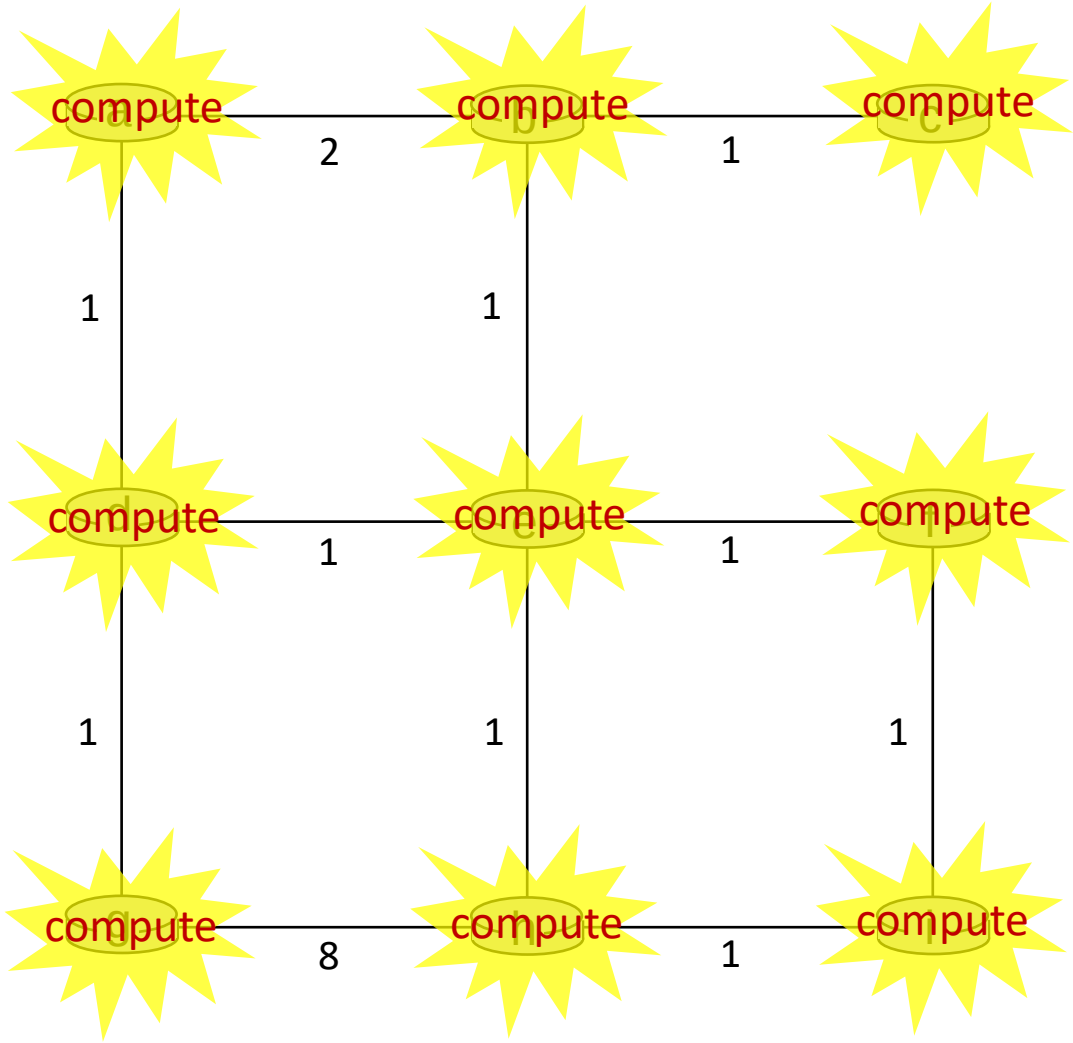


Distance vector example: iteration

t=2

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors

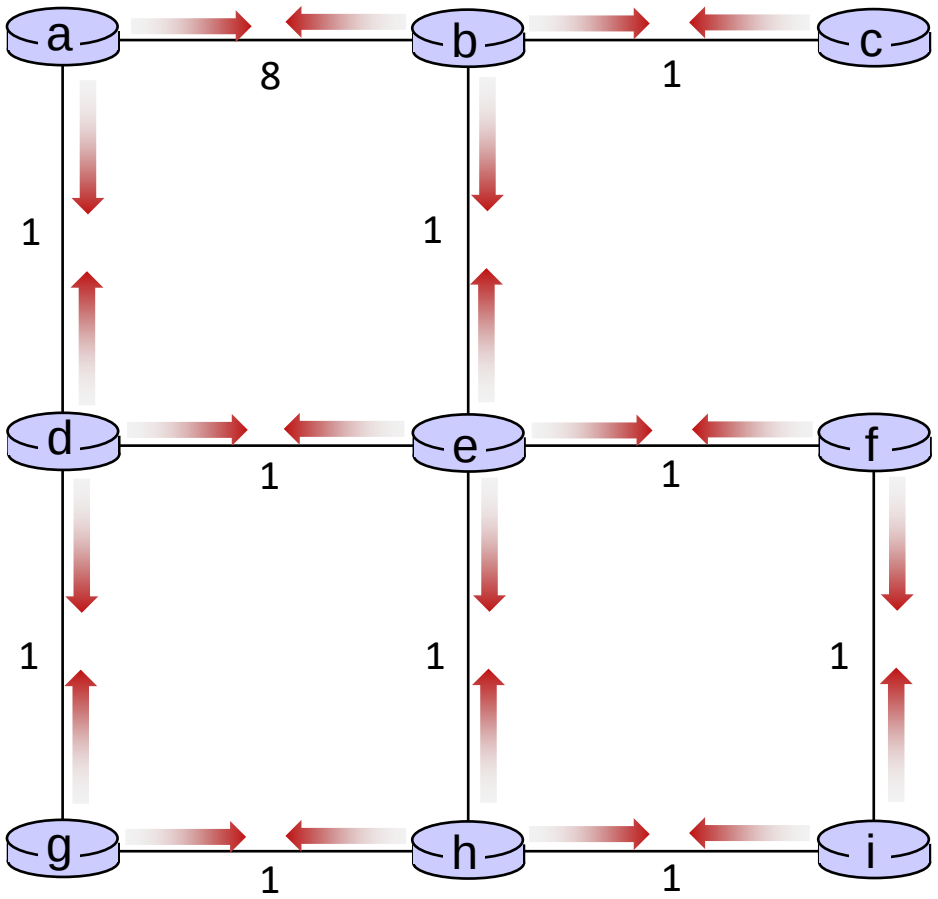


Distance vector example: iteration

t=2

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



Distance vector example: iteration

…. and so on

Let’s next take a look at the iterative *computations* at nodes

Distance vector example:

t=1

- b receives DVs from a, c, e

DV in a:

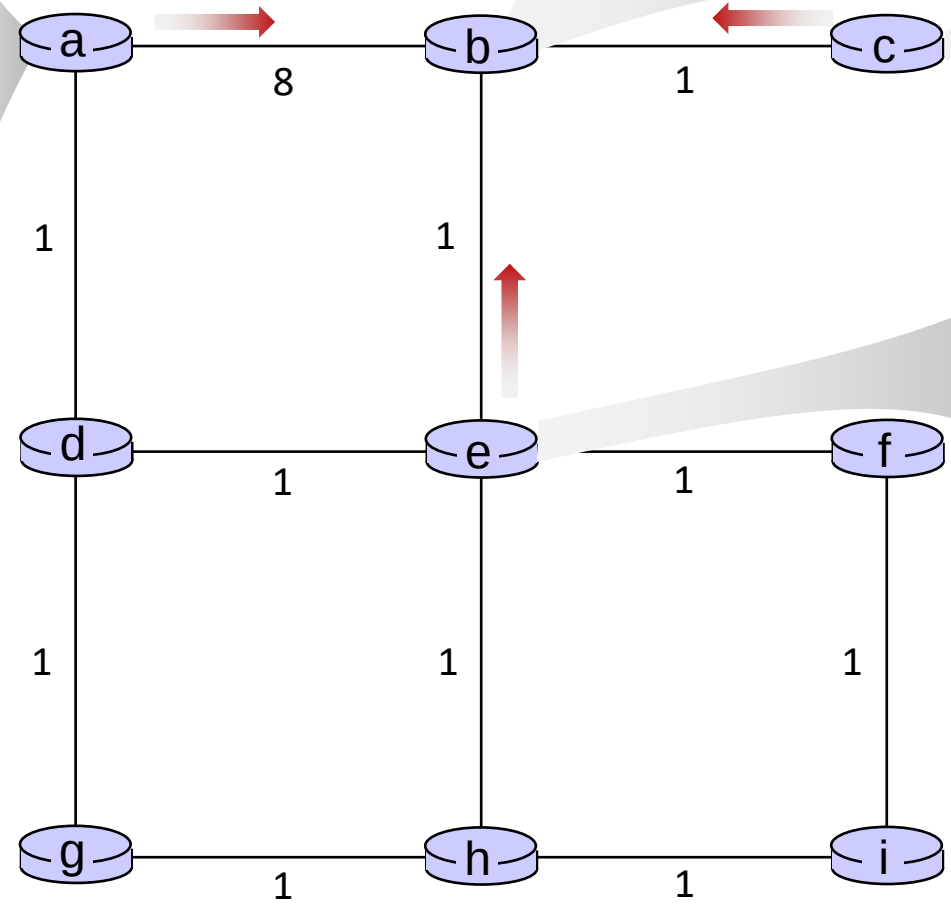
$D_a(a)=0$
$D_a(b) = 8$
$D_a(c) = \infty$
$D_a(d) = 1$
$D_a(e) = \infty$
$D_a(f) = \infty$
$D_a(g) = \infty$
$D_a(h) = \infty$
$D_a(i) = \infty$

DV in b:

$D_b(a) = 8$	$D_b(f) = \infty$
$D_b(c) = 1$	$D_b(g) = \infty$
$D_b(d) = \infty$	$D_b(h) = \infty$
$D_b(e) = 1$	$D_b(i) = \infty$

DV in c:

$D_c(a) = \infty$
$D_c(b) = 1$
$D_c(c) = 0$
$D_c(d) = \infty$
$D_c(e) = \infty$
$D_c(f) = \infty$
$D_c(g) = \infty$
$D_c(h) = \infty$
$D_c(i) = \infty$



DV in e:

$D_e(a) = \infty$
$D_e(b) = 1$
$D_e(c) = \infty$
$D_e(d) = 1$
$D_e(e) = 0$
$D_e(f) = 1$
$D_e(g) = \infty$
$D_e(h) = 1$
$D_e(i) = \infty$

Distance vector example:

DV in a:

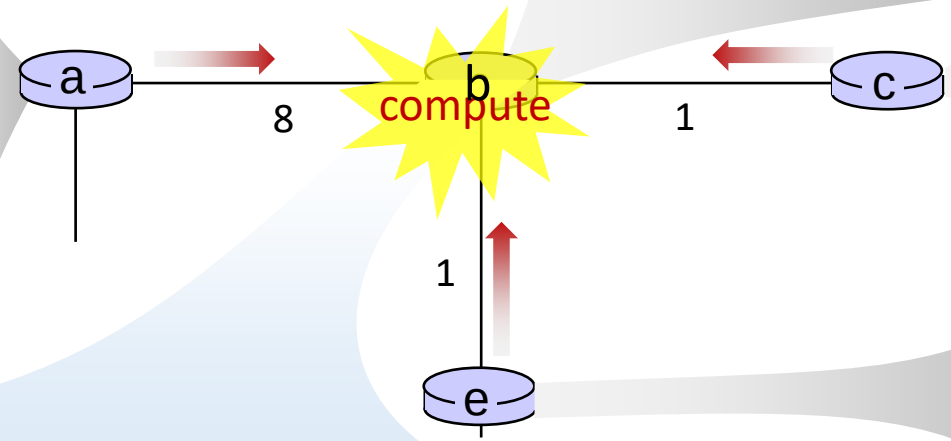
$D_a(a)=0$
$D_a(b) = 8$
$D_a(c) = \infty$
$D_a(d) = 1$
$D_a(e) = \infty$
$D_a(f) = \infty$
$D_a(g) = \infty$
$D_a(h) = \infty$
$D_a(i) = \infty$

DV in b:

$D_b(a) = 8$	$D_b(f) = \infty$
$D_b(c) = 1$	$D_b(g) = \infty$
$D_b(d) = \infty$	$D_b(h) = \infty$
$D_b(e) = 1$	$D_b(i) = \infty$

DV in c:

$D_c(a) = \infty$
$D_c(b) = 1$
$D_c(c) = 0$
$D_c(d) = \infty$
$D_c(e) = \infty$
$D_c(f) = \infty$
$D_c(g) = \infty$
$D_c(h) = \infty$
$D_c(i) = \infty$



DV in e:

$D_e(a) = \infty$
$D_e(b) = 1$
$D_e(c) = \infty$
$D_e(d) = 1$
$D_e(e) = 0$
$D_e(f) = 1$
$D_e(g) = \infty$
$D_e(h) = 1$
$D_e(i) = \infty$

t=1

- b receives DVs from a, c, e, computes:

$D_b(a) = \min\{c_{b,a}+D_a(a),\ c_{b,c}+D_c(a),\ c_{b,e}+D_e(a)\} = \min\{8,\infty,\infty\} = 8$

$D_b(c) = \min\{c_{b,a}+D_a(c),\ c_{b,c}+D_c(c),\ c_{b,e}+D_e(c)\} = \min\{\infty,1,\infty\} = 1$

$D_b(d) = \min\{c_{b,a}+D_a(d),\ c_{b,c}+D_c(d),\ c_{b,e}+D_e(d)\} = \min\{9,2,\infty\} = 2$

$D_b(e) = \min\{c_{b,a}+D_a(e),\ c_{b,c}+D_c(e),\ c_{b,e}+D_e(e)\} = \min\{\infty,\infty,1\} = 1$

$D_b(f) = \min\{c_{b,a}+D_a(f),\ c_{b,c}+D_c(f),\ c_{b,e}+D_e(f)\} = \min\{\infty,\infty,2\} = 2$

$D_b(g) = \min\{c_{b,a}+D_a(g),\ c_{b,c}+D_c(g),\ c_{b,e}+D_e(g)\} = \min\{\infty, \infty, \infty\} = \infty$

$D_b(h) = \min\{c_{b,a}+D_a(h),\ c_{b,c}+D_c(h),\ c_{b,e}+D_e(h)\} = \min\{\infty, \infty, 2\} = 2$

$D_b(i) = \min\{c_{b,a}+D_a(i),\ c_{b,c}+D_c(i),\ c_{b,e}+D_e(i)\} = \min\{\infty, \infty, \infty\} = \infty$

DV in b:

$D_b(a) = 8$	$D_b(f) = 2$
$D_b(c) = 1$	$D_b(g) = \infty$
$D_b(d) = 2$	$D_b(h) = 2$
$D_b(e) = 1$	$D_b(i) = \infty$

Distance vector example:

t=1

- c receives DVs from b

DV in a:

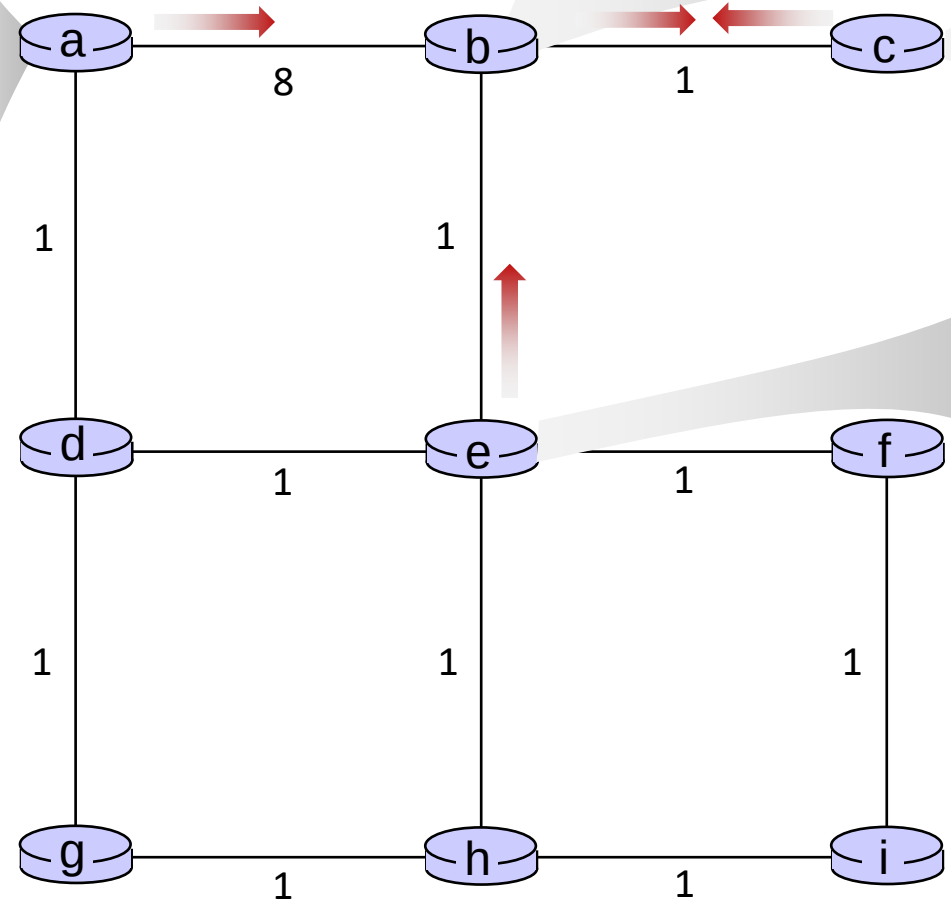
$D_a(a)=0$
$D_a(b) = 8$
$D_a(c) = \infty$
$D_a(d) = 1$
$D_a(e) = \infty$
$D_a(f) = \infty$
$D_a(g) = \infty$
$D_a(h) = \infty$
$D_a(i) = \infty$

DV in b:

$D_b(a) = 8$	$D_b(f) = \infty$
$D_b(c) = 1$	$D_b(g) = \infty$
$D_b(d) = \infty$	$D_b(h) = \infty$
$D_b(e) = 1$	$D_b(i) = \infty$

DV in c:

$D_c(a) = \infty$
$D_c(b) = 1$
$D_c(c) = 0$
$D_c(d) = \infty$
$D_c(e) = \infty$
$D_c(f) = \infty$
$D_c(g) = \infty$
$D_c(h) = \infty$
$D_c(i) = \infty$



DV in e:

$D_e(a) = \infty$
$D_e(b) = 1$
$D_e(c) = \infty$
$D_e(d) = 1$
$D_e(e) = 0$
$D_e(f) = 1$
$D_e(g) = \infty$
$D_e(h) = 1$
$D_e(i) = \infty$

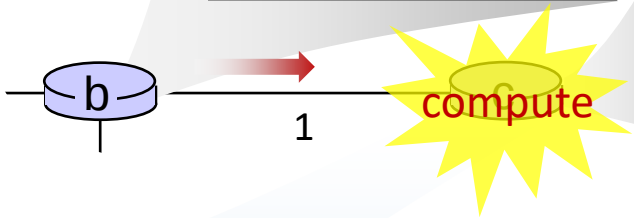
Distance vector example: c

DV in b:

$D_b(a) = 8$	$D_b(f) = \infty$
$D_b(c) = 1$	$D_b(g) = \infty$
$D_b(d) = \infty$	$D_b(h) = \infty$
$D_b(e) = 1$	$D_b(i) = \infty$

DV in c:

$D_c(a) = \infty$
$D_c(b) = 1$
$D_c(c) = 0$
$D_c(d) = \infty$
$D_c(e) = \infty$
$D_c(f) = \infty$
$D_c(g) = \infty$
$D_c(h) = \infty$
$D_c(i) = \infty$



t=1

- c receives DVs from b computes:

$D_c(a) = \min\{c_{c,b}+D_b(a\}\} = 1 + 8 = 9$

$D_c(b) = \min\{c_{c,b}+D_b(b)\} = 1 + 0 = 1$

$D_c(d) = \min\{c_{c,b}+D_b(d)\} = 1+ \infty = \infty$

$D_c(e) = \min\{c_{c,b}+D_b(e)\} = 1 + 1 = 2$

$D_c(f) = \min\{c_{c,b}+D_b(f)\} = 1+ \infty = \infty$

$D_c(g) = \min\{c_{c,b}+D_b(g)\} = 1+ \infty = \infty$

$D_c(h) = \min\{c_{bc,b}+D_b(h)\} = 1+ \infty = \infty$

$D_c(i) = \min\{c_{c,b}+D_b(i)\} = 1+ \infty = \infty$

DV in c:

$D_c(a) = 9$
$D_c(b) = 1$
$D_c(c) = 0$
$D_c(d) = 2$
$D_c(e) = \infty$
$D_c(f) = \infty$
$D_c(g) = \infty$
$D_c(h) = \infty$
$D_c(i) = \infty$

Distance vector example:

t=1

- e receives DVs from b, d, f, h

DV in b:

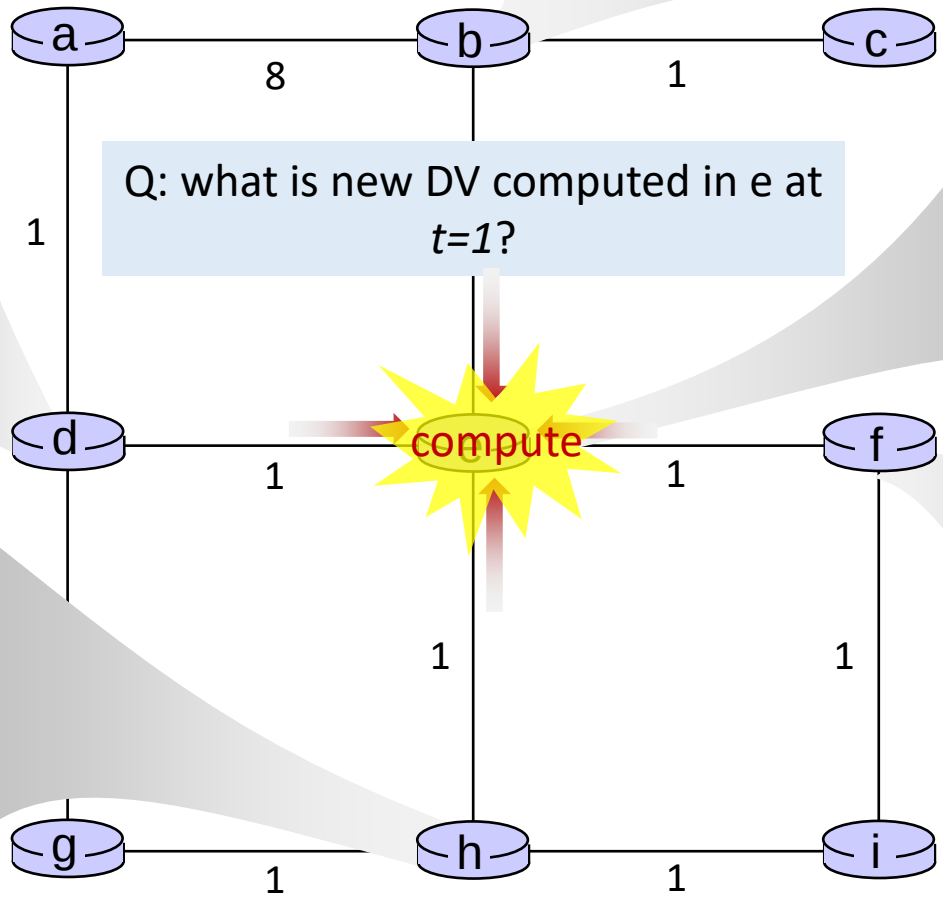
$D_b(a) = 8$ $D_b(f) = \infty$
$D_b(c) = 1$ $D_b(g) = \infty$
$D_b(d) = \infty$ $D_b(h) = \infty$
$D_b(e) = 1$ $D_b(i) = \infty$

DV in d:

$D_c(a) = 1$
$D_c(b) = \infty$
$D_c(c) = \infty$
$D_c(d) = 0$
$D_c(e) = 1$
$D_c(f) = \infty$
$D_c(g) = 1$
$D_c(h) = \infty$
$D_c(i) = \infty$

DV in e:

$D_e(a) = \infty$
$D_e(b) = 1$
$D_e(c) = \infty$
$D_e(d) = 1$
$D_e(e) = 0$
$D_e(f) = 1$
$D_e(g) = \infty$
$D_e(h) = 1$
$D_e(i) = \infty$



Q: what is new DV computed in e at *t=1*?

DV in h:

$D_c(a) = \infty$
$D_c(b) = \infty$
$D_c(c) = \infty$
$D_c(d) = \infty$
$D_c(e) = 1$
$D_c(f) = \infty$
$D_c(g) = 1$
$D_c(h) = 0$
$D_c(i) = 1$

DV in f:

$D_c(a) = \infty$
$D_c(b) = \infty$
$D_c(c) = \infty$
$D_c(d) = \infty$
$D_c(e) = 1$
$D_c(f) = 0$
$D_c(g) = \infty$
$D_c(h) = \infty$
$D_c(i) = 1$

Distance vector: state information diffusion

Iterative communication, computation steps diffuses information through network:

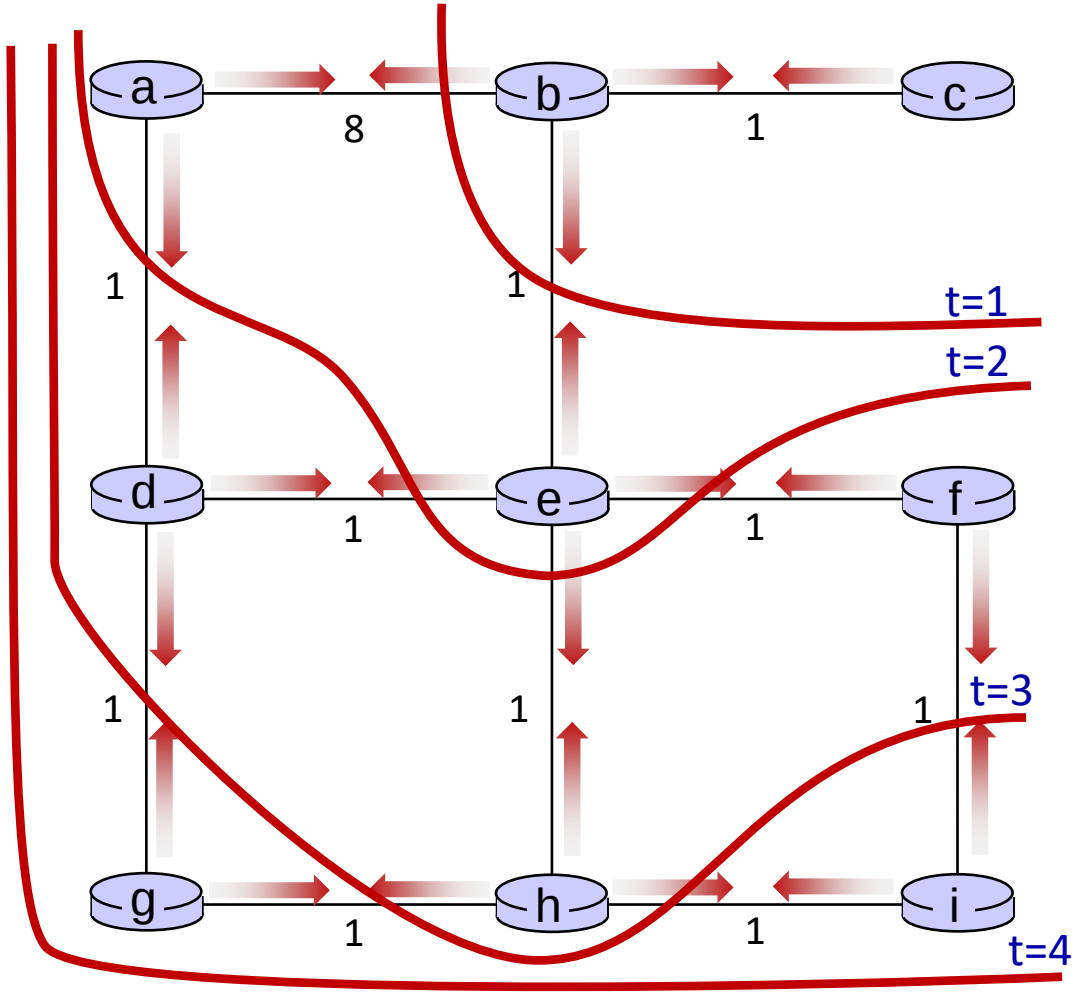
t=0 c's state at t=0 is at c only

t=1 c's state at t=0 has propagated to b, and may influence distance vector computations up to **1** hop away, i.e., at b

t=2 c's state at t=0 may now influence distance vector computations up to **2** hops away, i.e., at b and now at a, e as well

t=3 c's state at t=0 may influence distance vector computations up to **3** hops away, i.e., at b,a,e and now at c,f,h as well

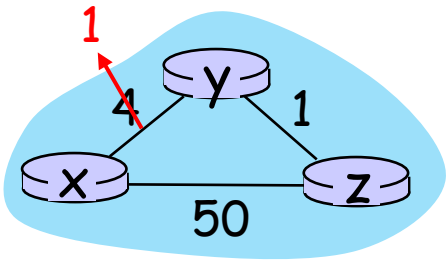
t=4 c's state at t=0 may influence distance vector computations up to **4** hops away, i.e., at b,a,e, c, f, h and now at g,i as well



Distance vector: link cost changes

link cost changes:

- node detects local link cost change
- updates routing info, recalculates local DV
- if DV changes, notify neighbors



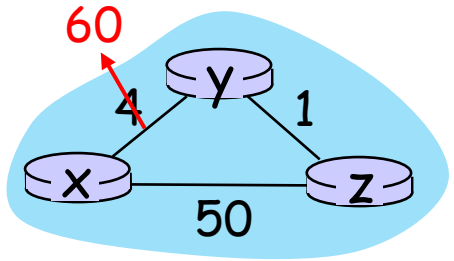
"good news travels fast"

t_0 : *y* detects link-cost change, updates its DV, informs its neighbors.

t_1 : *z* receives update from *y*, updates its table, computes new least cost to *x* , sends its neighbors its DV.

t_2 : *y* receives *z*'s update, updates its distance table. *y*'s least costs do *not* change, so *y* does *not* send a message to *z*.

Distance vector: link cost changes



link cost changes:

- node detects local link cost change
- "bad news travels slow" – count-to-infinity problem:
  - *y* sees direct link to *x* has new cost 60, but z has said it has a path at cost of 5. So *y* computes "my new cost to x will be 6, via z); notifies *z* of new cost of 6 to *x.*
  - *z* learns that path to *x* via *y* has new cost 6, so *z* computes "my new cost to *x* will be 7 via y), notifies *y* of new cost of 7 to *x.*
  - *y* learns that path to *x* via *z* has new cost 7, so *y* computes "my new cost to *x* will be 8 via y), notifies *z* of new cost of 8 to *x.*
  - *z* learns that path to *x* via *y* has new cost 8, so *z* computes "my new cost to *x* will be 9 via y), notifies *y* of new cost of 9 to *x.*

    ...

Distributed algorithms are tricky!

Minimum distance routing: *Algorithms*

Distance vector algorithm

- Used in ARPANET until 1979 with 125 ms update period
- Based on *queue length*, not on *link speed*
- Too slow to converge in a dynamic context

Subsequent algorithm for ARPANET (1979): link state

- Each node estimates the distance to all its neighbors using the *delay* metric
- Such distance is communicated to ALL nodes by flooding
- Each node can determine the minimum cost path to ALL nodes
- Update period in the order of 10 s

Comparison of LS and DV algorithms

Distance vector and link state algorithms converge to the same solution in static conditions

speed of convergence

LS: O(n^2) algorithm, O(n^2) messages
- may have oscillations
- O(n^3) in highly meshed network since E ≅ n^2
- Dijkstra: O(n^2) - n nodes processed and n – i + 1 nodes to examine at step i

DV: convergence time varies
- Bellman-Ford: O(n E) - up to n steps each processing all edges
- may have routing loops
- count-to-infinity problem

message complexity

LS: *n* routers, O(n^2) messages sent

DV: exchange between neighbors; convergence time varies

robustness: what happens if router malfunctions, or is compromised?

LS:
- router can advertise incorrect *link* cost
- each router computes only its *own* table

DV:
- DV router can advertise incorrect *path* cost ("I have a *really* low cost path to everywhere"): black-holing
- each router's table used by others: error propagate thru network

Distance-vector algorithm
- Implicitly distributed (each router computes its routing tables)
- Each node knows only what its neighbors know

Link state algorithm
- Its implementation can be both centralized and distributed
- In the distributed approach each node must "see" the whole network

Network layer: “control plane” roadmap

- introduction
- routing protocols
  - link state
  - distance vector
- **intra-ISP routing:**
  - RIP
  - OSPF
- routing among ISPs: BGP
- SDN control plane

- network management, configuration
  - SNMP
  - NETCONF/YANG

Making routing scalable

our routing study thus far - idealized

- all routers identical
- network “flat”

… not true in practice

scale: billions of destinations:

- can’t store all destinations in routing tables!
- routing table exchange would swamp links!

administrative autonomy:

- Internet: a network of networks
- each network admin may want to control routing in its own network

Internet approach to scalable routing

aggregate routers into regions known as "autonomous systems" (AS) (a.k.a. "domains")

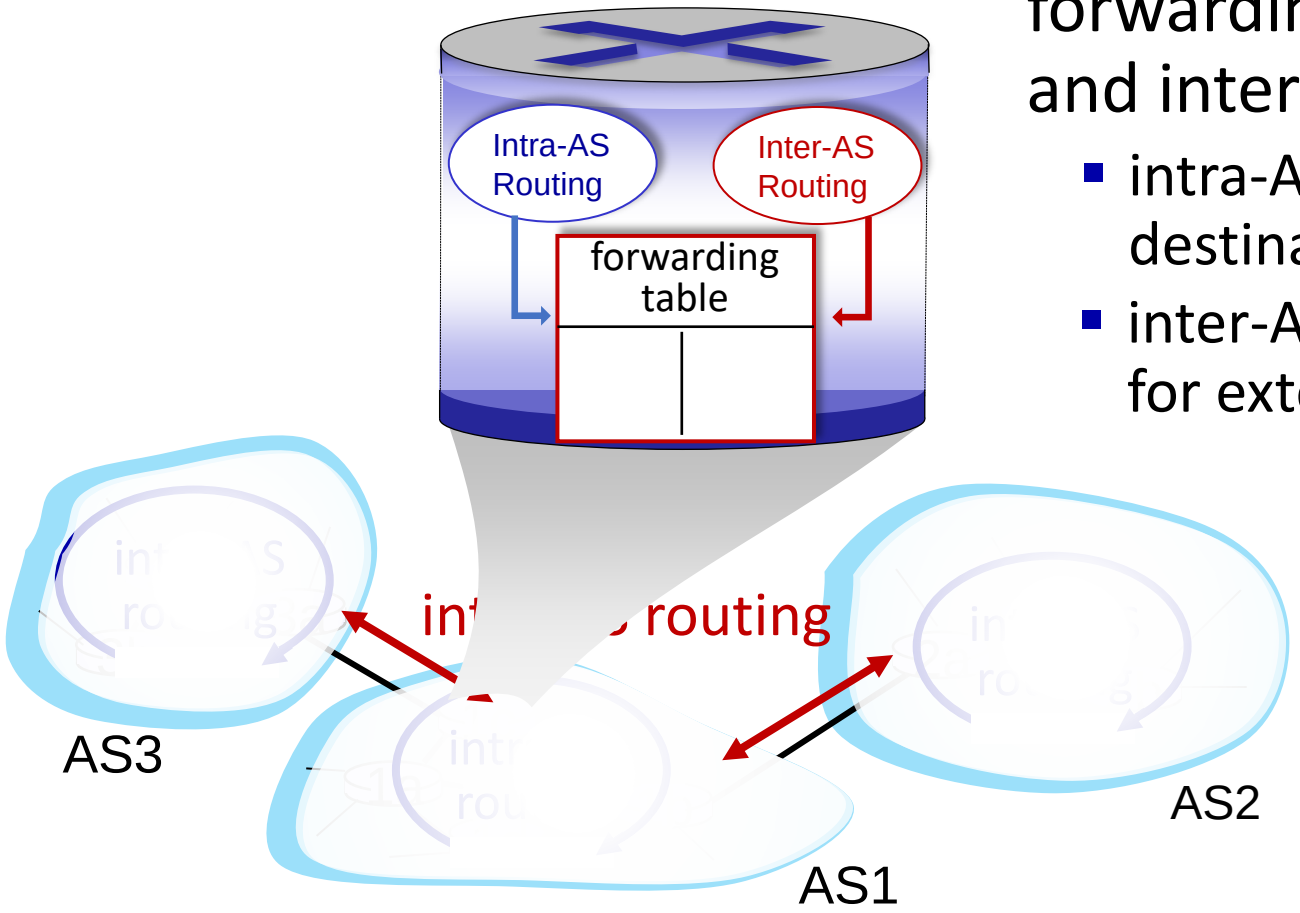
intra-AS (aka "intra-domain"): routing among *within same AS ("network")*

- all routers in AS must run same intra-domain protocol
- routers in different AS can run different intra-domain routing protocols
- gateway router: at "edge" of its own AS, has link(s) to router(s) in other AS'es

inter-AS (aka "inter-domain"): routing *among* AS'es

- gateways perform inter-domain routing (as well as intra-domain routing)

Interconnected ASes



forwarding table configured by intra- and inter-AS routing algorithms

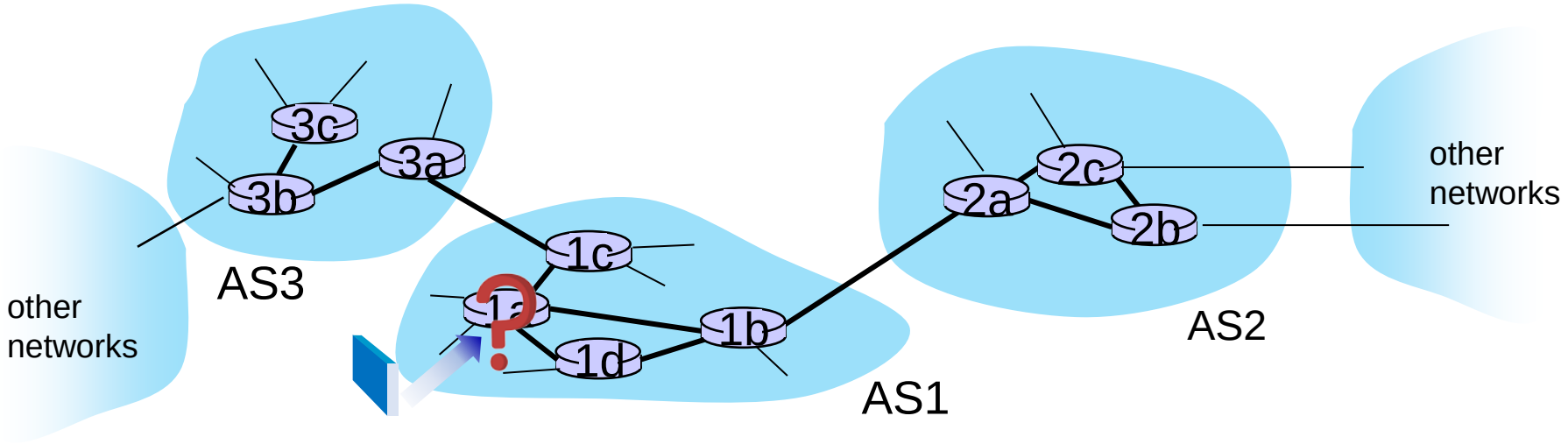
- intra-AS routing determine entries for destinations within AS
- inter-AS & intra-AS determine entries for external destinations

Inter-AS routing: a role in intradomain forwarding

- suppose router in AS1 receives datagram destined outside of AS1:
  - router should forward packet to gateway router in AS1, but which one?

AS1 inter-domain routing must:

1. learn which destinations reachable through AS2, which through AS3
2. propagate this reachability info to all routers in AS1



Internet: *Routing protocols*

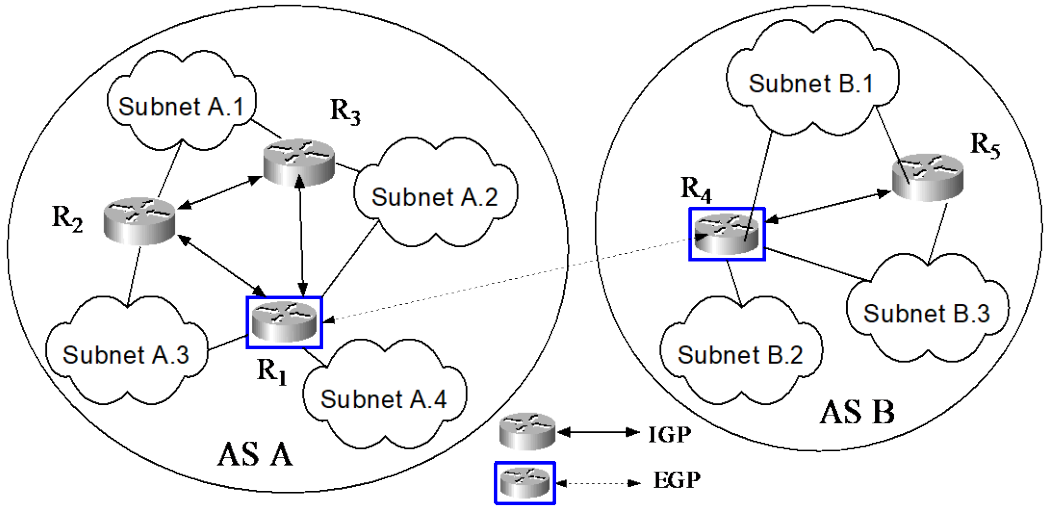
Internet organized as a collection of autonomous systems (ASs)

Autonomous system

- Set of subnetworks connected by homogeneous routers with possibly a unique administrative entity

Routing protocols

- Interior gateway protocols (IGP)
  - intra-domain (inside an AS)
- Exterior gateway protocols (EGP)
  - inter-domain (among ASs)

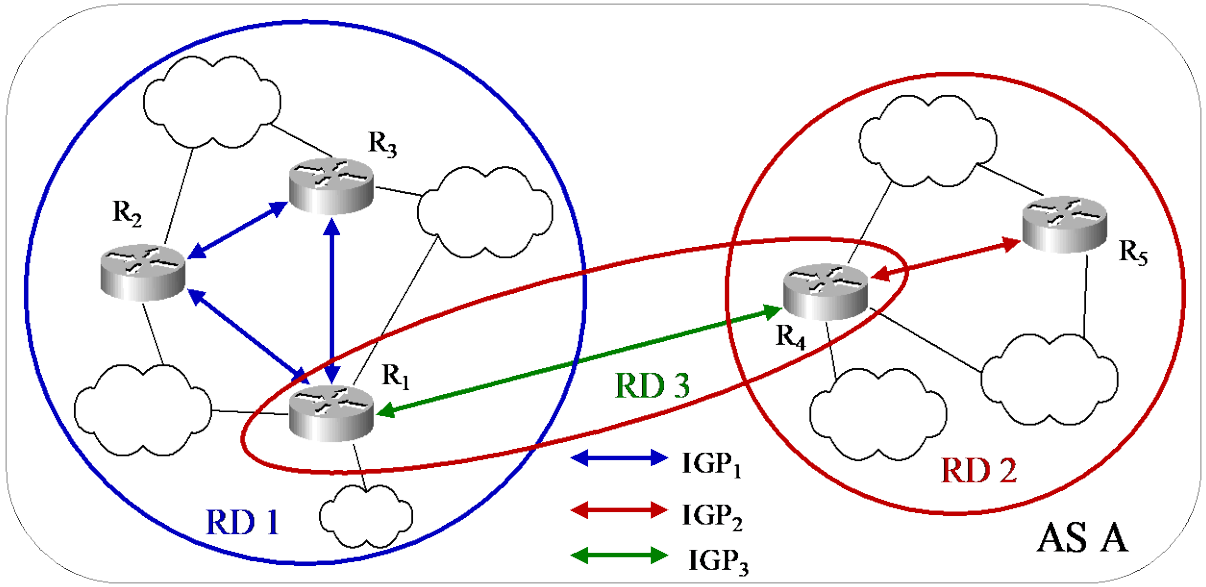


Routing protocols: *Routing domains*

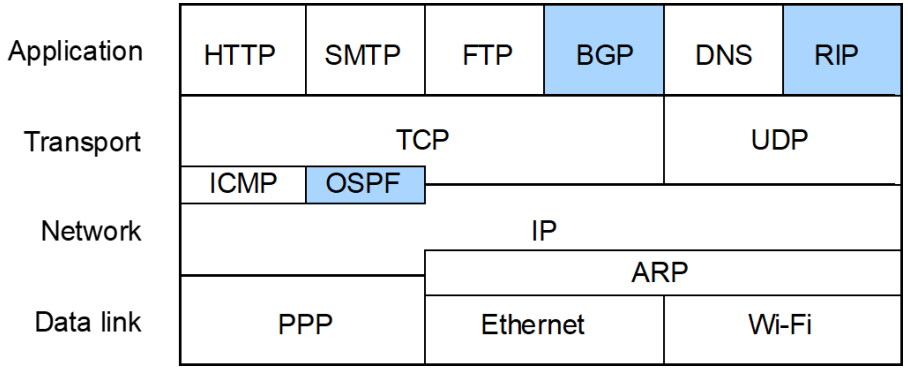
Routing domain (RD)

- **AS portion where a unique routing protocol is used**

Routers belonging to different RDs can use different routing protocols



Inter-AS routing: routing within an AS



most common intra-AS routing protocols:

- RIP: Routing Information Protocol [RFC 1723]
  - classic DV: DVs exchanged every 30 secs
  - no longer widely used
- EIGRP: Enhanced Interior Gateway Routing Protocol
  - DV based
  - formerly Cisco-proprietary for decades (became open in 2013 [RFC 7868])
- OSPF: Open Shortest Path First [RFC 2328]
  - link-state routing
  - IS-IS protocol (ISO standard, not RFC standard) essentially same as OSPF

Network layer: “control plane” roadmap

- introduction
- routing protocols
  - link state
  - distance vector
- intra-ISP routing:
  - **RIP**
  - OSPF
- routing among ISPs: BGP
- SDN control plane

- network management, configuration
  - SNMP
  - NETCONF/YANG

Routing protocols

IGP – Intra-domain

Distance vector

RIP: routing information protocol, version 1 and 2

IGRP: interior gateway routing protocol (proprietary CISCO)

IS-IS: intermediate system to intermediate system

OSPF: open shortest path first

Link state

EGP – Inter-domain

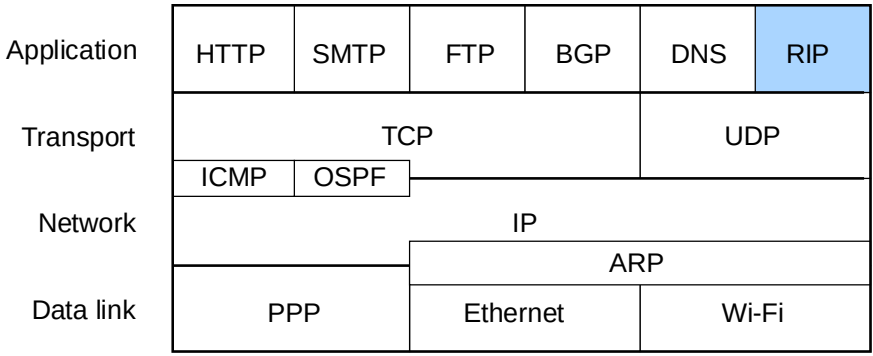
BGP: border gateway protocol

Path vector

EGP (RFC 827 and RFC 924): now obsolete

Same name as whole class!

Interior Gateway Protocols: *RIP*



<u>Routing information protocol</u> (RIP): RFC 1058 (1988)

- Relies on UDP transport protocol (well known port 520)
- Fully distributed protocol
  - Distance vector (table) updates via communication with neighbors
- Distance metric: hop count
  - Max distance: 16 hops
- Routing broadcast updates every 30 s

Pros

- Easy implementation

Cons

- High complexity: $O(N^2)$- $O(N^3)$
  - N: network nodes
- Routing loops
- Slow convergence (count-to-infinity)

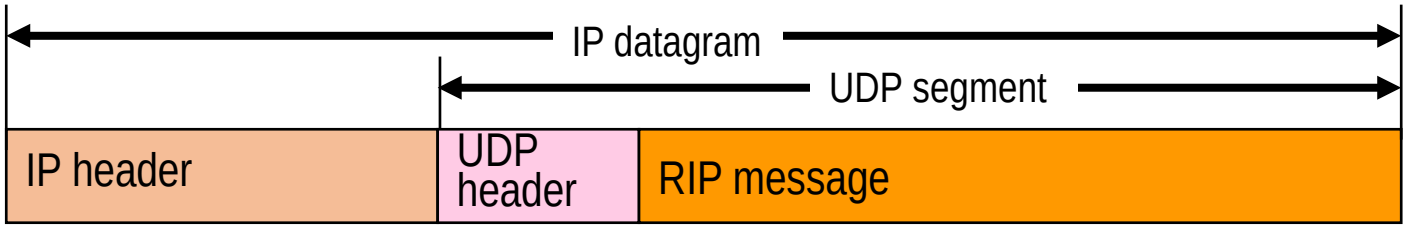
RIP: *Message format*

Each router maintains a table with

- best known distance (in hop count) to each destination
- link to use to reach the destination

RIP message carried by UDP segments: max 512-byte UDP segment

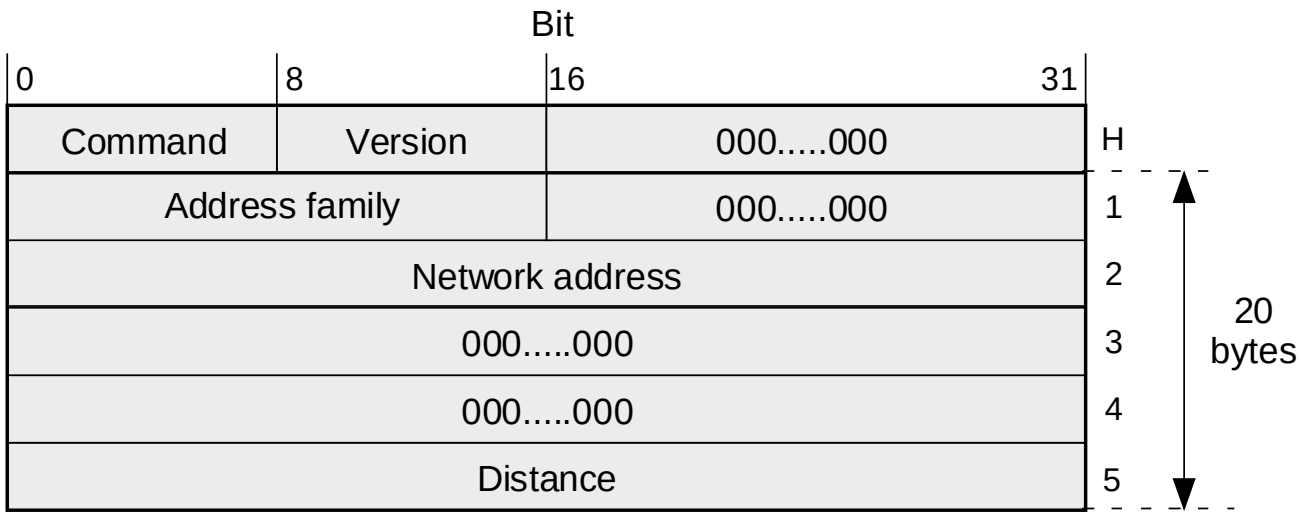
- 8 bytes: UDP header
- 4 bytes: RIP header
- 20 bytes x up to 25 entries: too small to transfer an entire routing table
  → more than 1 UDP datagram generally needed



RIPv1: *Message format*

Message format (RIP version 1)

- *Command* (8 bit): type of message (request (1), response (2))
- *Version* (8 bit): 1
- *Address family* (16 bit): family of addressing used (1 for IPv4, 2 for IPv6))
- *Network address* (14 byte): address of destination network (4 byte for IPv4)
- *Distance* (32 bit): hop count from advertising router to destination network
  - (valid from 1 to 15; 16 = infinite)



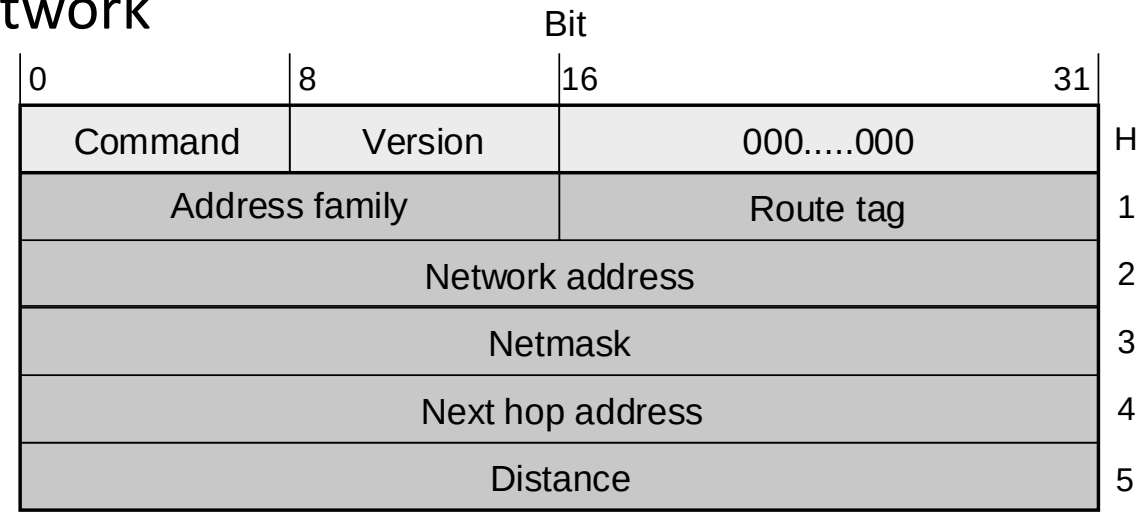
RIPv2: *Message format*

Message format (RIP version 2) (RFC 2453)

- Version (8 bit): 2
- Simply adds information in the all-0 fields of the RIPv1 message
- It is designed to maintain full compatibility with RIPv1 routers

New fields

- Route tag (16 bit): Autonomous system number originating the message
- Netmask (4 byte): subnet mask of the intended network (Network address)
- Next hop address (4 byte): useful if two ASs share a network (e.g. a backbone network



RIP: *Initialization*

When routing daemon started, it sends special RIP request on every interface

- command = 1 (request)
- address family = 0 (instead of 2)
- metric set to 16 (infinite)

This asks for complete routing table from all connected routers

- allows to discover adjacent routers!

RIP: *Operation after initialization*

Request

- asks for response relative to specific IP addresses listed in the request message

Response

- returns list of IP addresses with associated metric
- if router does not have a route to specified destination, it returns 16

Regular update

- routers send part (or all) of their table every 30 s to adjacent routers
- a router deletes (sets metric to 16) an entry from its routing table if not refreshed within 6 cycles (180 s)
  - deletion after additional 60 s to ensure propagation of entry invalidation

Triggered update

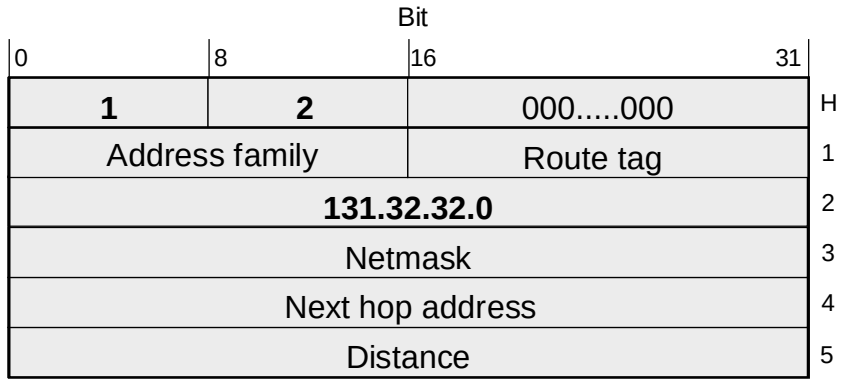
- upon change of metric for a route (transmits only entries changed)

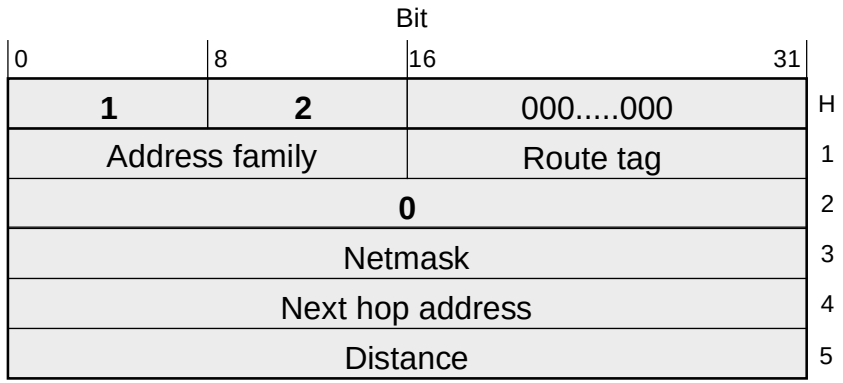
RIPv2: *Request messages*

- Request sent by a router that
  - has just come up or
  - has some timed-out entries
- Request can ask for
  - specific entries

or

  - all entries





RIPv2: *Response message*

- Solicited, as answer to a request, or
- Unsolicited, sent periodically (30 s)

2	**2**	000.....000	H
Address family		Route tag	1
151.32.32.0			2
255.255.224.0			3
Next hop address			4
6			5

2	**2**	000.....000	H
Address family		Route tag	1
204.110.111.112			2
255.255.255.240			3
Next hop address			4
2			5

2	**2**	000.....000	H
Address family		Route tag	1
84.0.0.0			2
255.0.0.0			3
Next hop address			4
11			5

Routing table
(with all link cost = 1)

Destination network	*Netmask*	*Distance*
151.32.32.0	255.255.224.0	7
204.110.111.112	255.255.255.240	3
84.0.0.0	255.0.0.0	12

RIP: *Limits*

Hop count metric is too simple!

- Can be trivially extended to different distance metric: the core of the algorithm does not change!
  - queue length on considered hop
  - time delay in ms
  - packet loss ratio measured
  - etc…

Slow convergence

- Routers distant *N* hops need *N* steps to update their tables

Limited to small network sizes

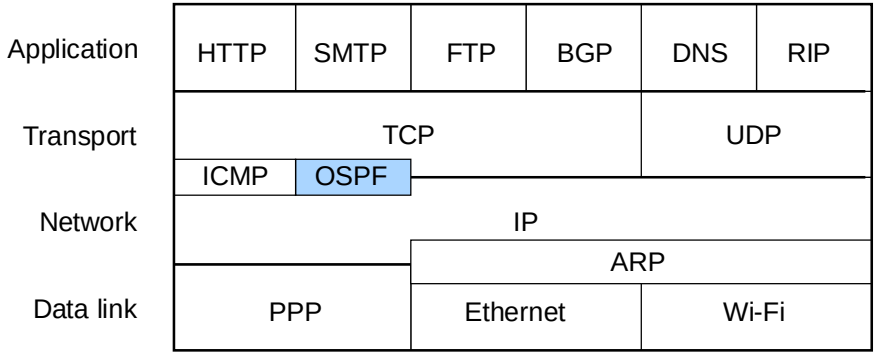
- As infinite = 16, nodes cannot be more than 15 hops far away
  - but just raise infinite to 32...

Network layer: “control plane” roadmap

- introduction
- routing protocols
  - link state
  - distance vector
- intra-ISP routing:
  - RIP
  - **OSPF**
- routing among ISPs: BGP
- SDN control plane

- network management, configuration
  - SNMP
  - NETCONF/YANG

OSPF (Open Shortest Path First)



Open shortest path first version 2:
RFC 1583, RFC 2178, RFC 2328

Classic link-state

- each router floods OSPF link-state advertisements (directly over IP rather than using TCP/UDP) to all other routers in entire AS
- multiple link costs metrics possible: bandwidth, delay. configurable by system administrator
- each router has full topology, uses Dijkstra's algorithm to compute forwarding table

OSPF routing is hierarchical

- High scalability is ensured
- Arbitrarily big networks can be managed

security: all OSPF messages authenticated (to prevent malicious intrusion)

Interior gateway protocols: *OSPF*

Each router must

- discover its neighbors
- measure a “cost” of the line connecting to the neighbor
- construct a message containing the information about all the connected links
- send the message to all other routers

Link state vs distance vector protocol

- Much faster convergence
- Does not need periodical update
  - Rather it needs event update: packet transmission is triggered by link state changes
- Does not need to transmit large routing tables
  - only state of local links

Interior gateway protocols: *OSPF*

OSPF messages carried as payload of IP packets using multicasting network capability

Connection types and networks supported

- Point-to-point lines
- Multiaccess networks with broadcasting (most LANs) or without broadcasting (most WANs)

Routing table updates

- Periodical: link state packets are broadcasted by a router to signal up/down condition of outgoing links and their current cost
- Non-periodical: due to link state variation

Other features

- Type of service routing
- Dynamic load balancing by using multiple routes if routes have same length (given metric)
- Authentication

Message types: *Message types*

Message header

Bit 0	8	16 – 31	
Version	Type	Message length	1
Source router IP address			2
Area ID			3
Checksum		Authentication type	4
Authentication (octets 0-3)			5
Authentication (octets 4-7)			6

Message types

Hello	Tests neighbor reachability
Database description (LSDB)	Announces which updates the sender has
Link state request (LSR)	Requests information for a specific link
Link state update (LSU)	Provides the sender's cost to neighbors
Link state ack (LSA)	Acknowledges link state update

OSPF: *Flooding approach*

All routers inform neighboring routers about state (cost) of outgoing links

Each router, upon receiving a LSU packet, performs selective flooding

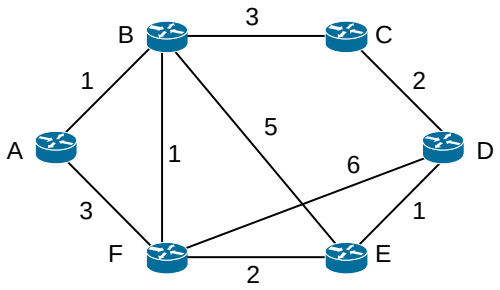
- A copy of all the received LSU is stored in memory and compared to the received LSUs
- The router checks if the LSU is a copy or new
  - If a copy - discard
  - If new - selectively forward on all links except the one it came from

Through all received messages, routers reconstruct dynamically the network topology

- Shortest path trivially computed by Dijkstra algorithm
- and routing tables built accordingly

In a multiaccess LAN, only one router (designated router) broadcasts routing information after agreement with other routers

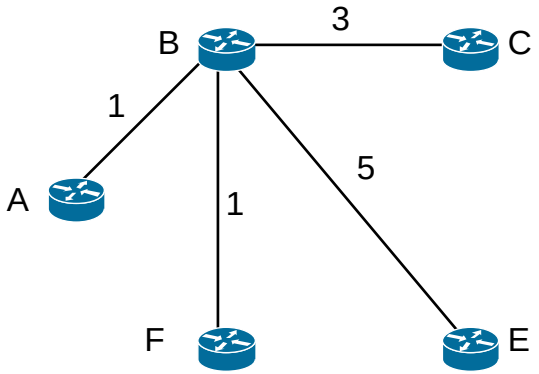
OSPF: *Network topology*



Node A receives in sequence LSUs from nodes B, C, F, E, D

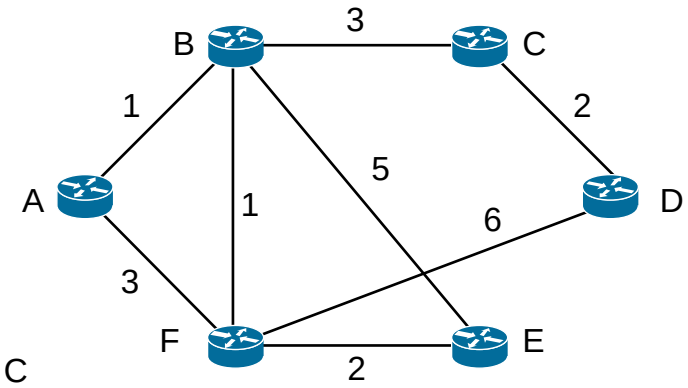
Node B

A	1
C	3
E	5
F	1



Node F

A	3
B	1
D	6
E	2



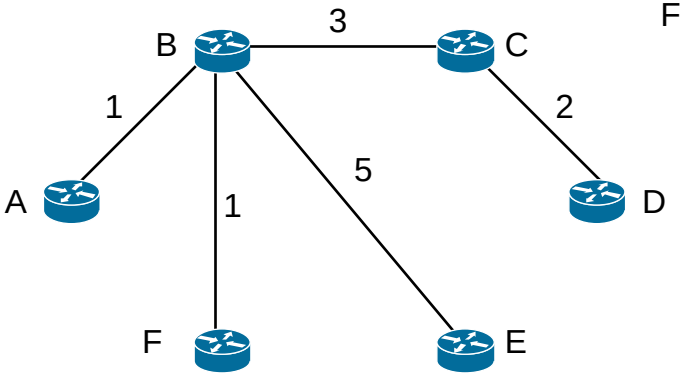
Node D

C	2
E	1
F	6

No additional information

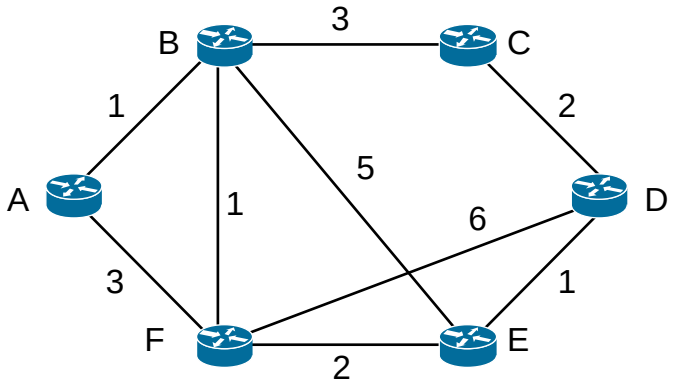
Node C

B	3
D	2



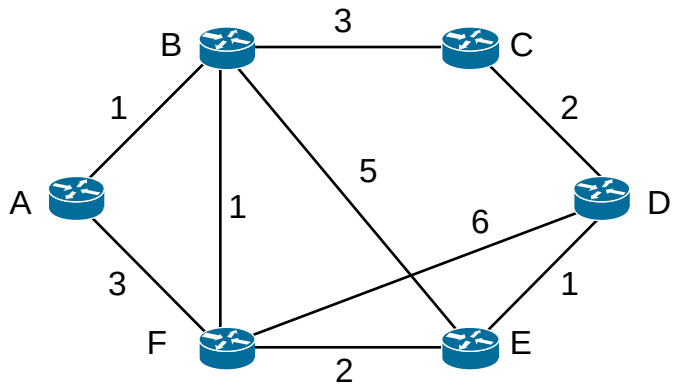
Node E

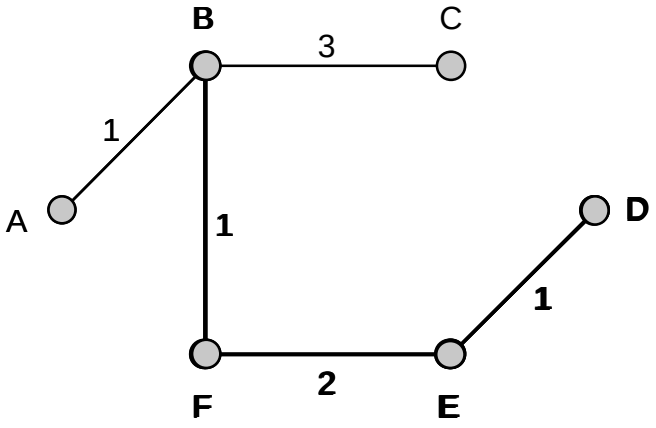
B	5
D	1
F	2



SPT: *Dijkstra algorithm*

Find shortest path tree (SPT) with A as root node





		B		C		D		E		F	
Passo	M	D_B	P_B	D_C	P_C	D_D	P_D	D_E	P_E	D_F	P_F
0	A	1	A							3	A
1	A,B			4	B			6	B	2	B
2	A,B,F			4	B	8	F	4	F		
3	A,B,C,F					6	C	4	F		
4	A,B,C,E,F					5	E				
5	A,B,C,D,E,F										

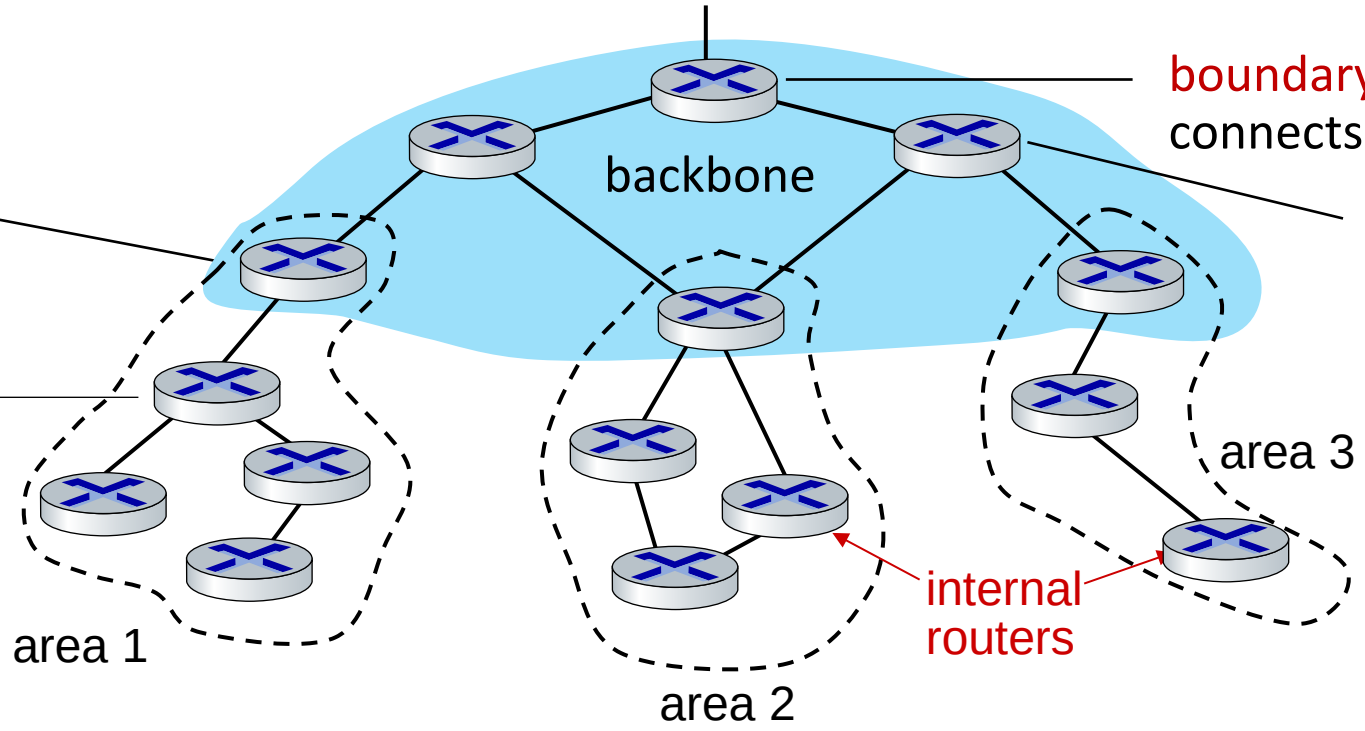
Note: now node C joins the SPT before node E (both distant 4 from A)

Hierarchical OSPF

- two-level hierarchy: local area, backbone.
  - link-state advertisements flooded only in area, or backbone
  - each node has detailed area topology; only knows direction to reach other destinations

area border routers:
"summarize" distances to destinations in own area, advertise in backbone

local routers:
- flood LS in area only
- compute routing within area
- forward packets to outside via area border router



boundary router:
connects to other ASes

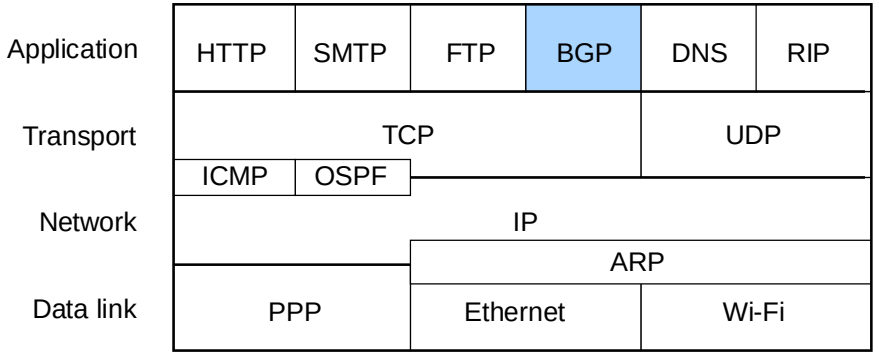
backbone router:
runs OSPF limited to backbone

Network layer: “control plane” roadmap

- introduction
- routing protocols
  - link state
  - distance vector
- intra-ISP routing:
  - RIP
  - OSPF
- **routing among ISPs: BGP**
- SDN control plane

- network management, configuration
  - SNMP
  - NETCONF/YANG

Exterior Gateway Protocols: *BGP*



Border gateway protocol version 4 (BGP-4): RFC 4271

- Means to allow router in different ASs to exchange information
- Protocol based on messages transported by TCP messages, using “well known” port 179
- Two BGP protocols
  - eBGP (external BGP): AS border routers exchange routing information
  - iBGP (internal BGP): AS border router propagates routing information inside the AS
- Uses a distance-vector like algorithm (path vector)
- Each routing table entry contains for each destination network announced
  - Origin of the information
  - Next hop IP router
  - Path to reach the destination, i.e. the ID sequence of ASs to cross
- It is totally independent from the IGP adopted in each AS

BGP: *Operations*

- Routing table content
  - BGP maintains a routing table with all feasible paths, but advertises only primary path
  - BGP metric is an arbitrary unit, assigned by network administrator
- Routing table construction/update
  - The two routers recognize each other and become peer
  - Initially two routers exchange the entire BGP routing table
  - Incremental updates sent out as the routing tables change
- Message types
  - Open: to activate neighborhood
  - Update: to update routing information on reachable networks
  - Notification: to notify procedure errors
  - Keepalive: to signal router activity in absence of updates

BGP: *Functional procedures*

Neighbor acquisition

- Used by routers to agree on becoming neighbors (or peer)
- One router sends Open to another, specifying AS ID (ASN), update periodicity (hold time), authentication
- The other router, accepting, responds with Keepalive

Neighbor reachability

- Used by routers to maintain neighboring
- The two routers periodically issue Keepalive before expiration of Hold time

Network reachability

- Each router maintains a DB of reachable subnets and the preferred route to each of them
- Upon changing in the DB, a router broadcasts Update to all the neighbor BGP routers specifying
  - Withdrawn networks
  - New reachable networks ({destination AS/AS path} pairs)

  (AS path: string of ASs through which the specified AS can be reached)

BGP: *Functional procedures*

Count-to-infinity and routing loops not occurring, since the paths are specified by means of all the routers crossed

Path selection by a router to reach a given destination network given many announcements received

- Best attributes, as selected by the AS administrator
- With same attributes, path with least number of crossed ASs
- With same attributes and crossed ASs, path whose exit router in the AS is the closest one (*hot potato* rouing)

BGP
Example

Final routing tables

	R_1		R_2		R_3		R_4		R_5		R_6		R_7	
Dest.	NH	Cost	NH	Cost	NH	Cost	NH	Cost	NH	Cost	NH	Cost	NH	Cost
N_1	Dir.	-	R_1	1	R_2	1	R_3	2	R_3	2	R_5	3	R_6	1
N_2	Dir	-	Dir	-	R_2	1	R_3	2	R_3	2	R_5	3	R_6	1
N_3	R_2	2	R_3	1	R_4	1	Dir.	-	R_6	1	Dir.	-	R_6	1
N_4	R_2	2	R_3	1	R_5	1	R_6	1	Dir.	-	Dir.	-	R_6	1
N_5	R_2	2	R_3	1	Dir.	-	R_3	1	Dir.	-	R_5	1	R_6	1
N_6	R_2	2	R_3	1	Dir.	-	Dir.	-	R_3	1	R_4	1	R_6	1
N_7	R_2	2	R_3	1	R_5	3	R_6	2	R_6	2	R_7	1	Dir.	-
N_8	R_2	2	R_3	1	R_5	3	R_6	2	R_6	2	R_7	1	Dir.	-

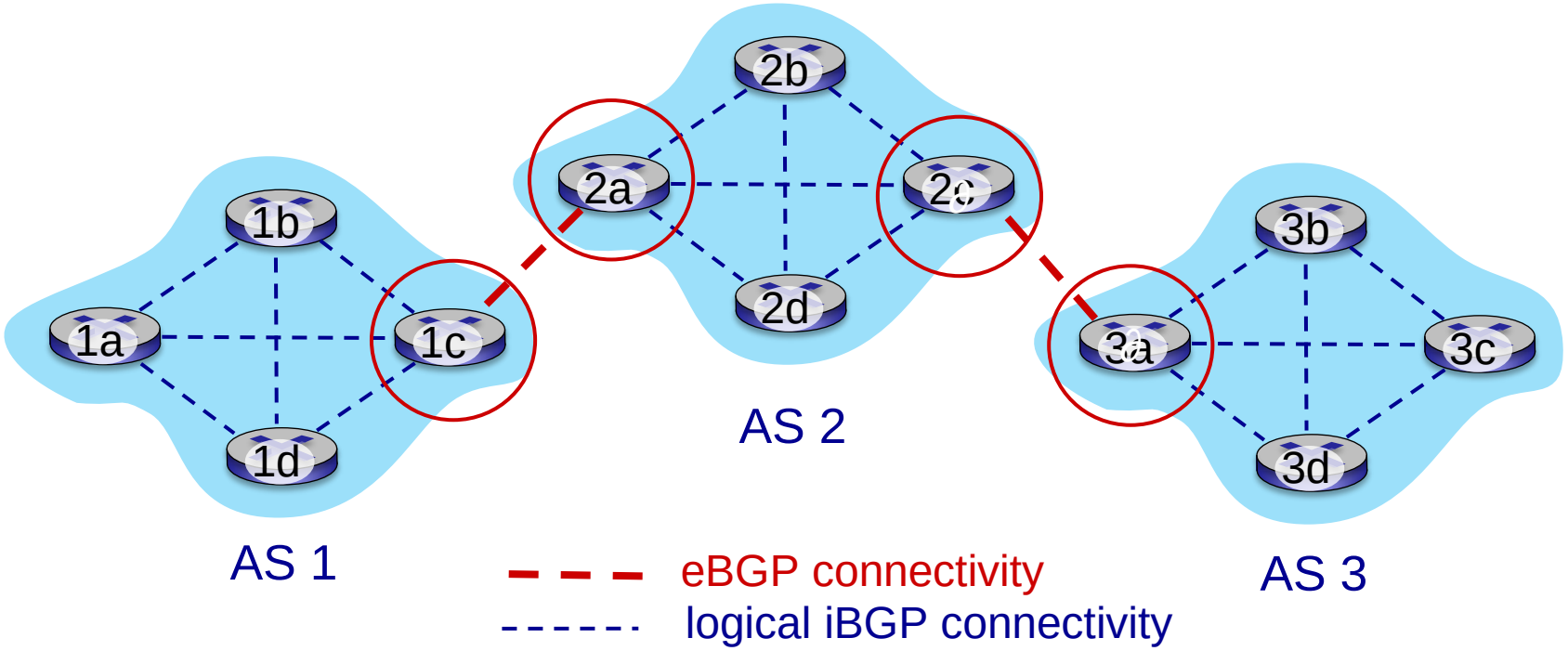
Final interdomain paths

Sorgente	Destinazione	NH router	AS path
R_1	N_3, N_4, N_5, N_6	R_2	AS_1, AS_2
	N_7, N_8	R_2	AS_1, AS_2, AS_3
R_2	N_3, N_4, N_5, N_6	R_3	AS_1, AS_2
	N_7, N_8	R_3	AS_1, AS_2, AS_3
R_3	N_1, N_2	R_2	AS_2, AS_1
	N_7, N_8	R_4	AS_2, AS_3
R_4	N_1, N_2	R_3	AS_2, AS_1
	N_7, N_8	R_6	AS_2, AS_3
R_5	N_1, N_2	R_3	AS_2, AS_1
	N_7, N_8	R_6	AS_2, AS_3
R_6	N_1, N_2	R_5	AS_2, AS_1
	N_7, N_8	R_7	AS_2, AS_3
R_7	N_1, N_2	R_6	AS_3, AS_2, AS_1
	N_3, N_4, N_5, N_6	R_6	AS_3, AS_2

Internet inter-AS routing: BGP

- BGP (Border Gateway Protocol): *the* de facto inter-domain routing protocol
  - "glue that holds the Internet together"
- allows subnet to advertise its existence, and the destinations it can reach, to rest of Internet: *"I am here, here is who I can reach, and how"*
- BGP provides each AS a means to:
  - eBGP: obtain subnet reachability information from neighboring ASes
  - iBGP: propagate reachability information to all AS-internal routers.
  - determine "good" routes to other networks based on reachability information and *policy*

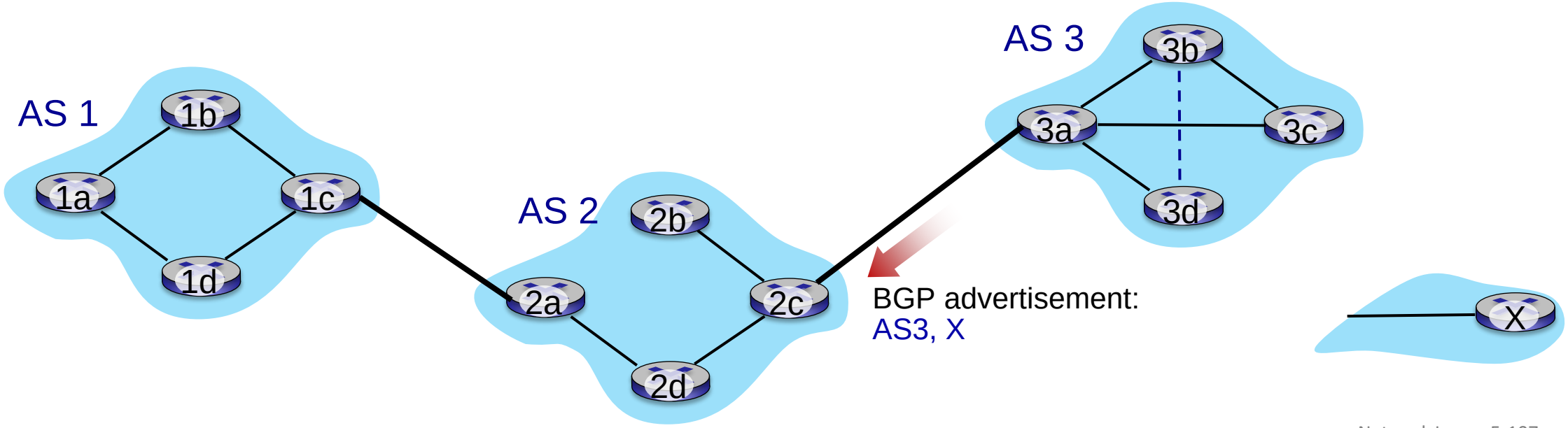
eBGP, iBGP connections



gateway routers run both eBGP and iBGP protocols

BGP basics

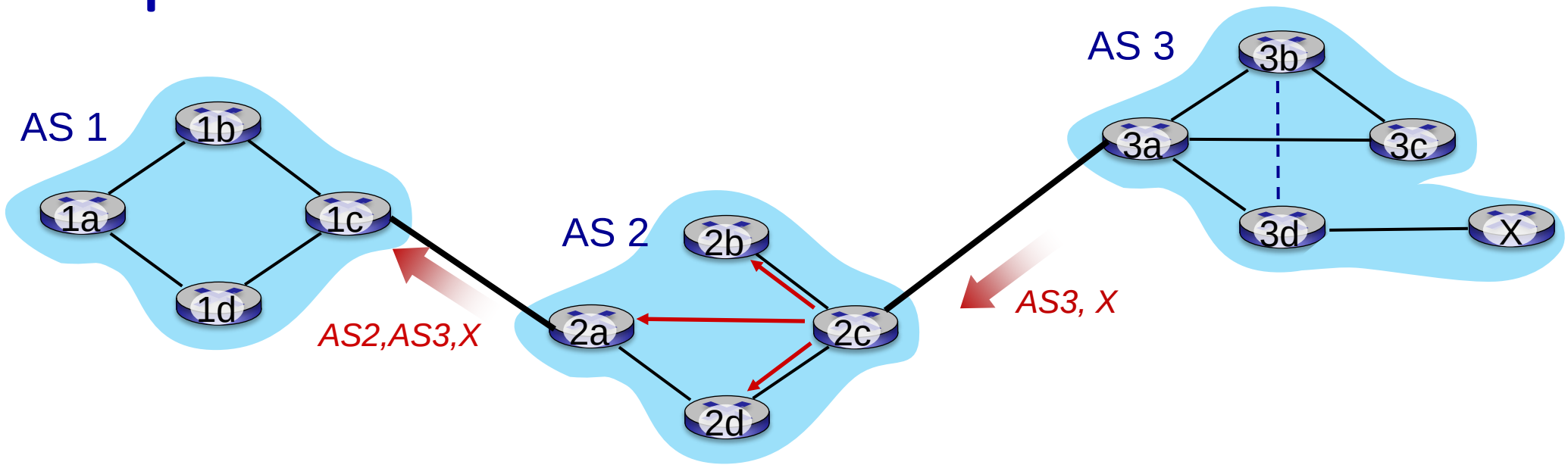
- BGP session: two BGP routers ("peers") exchange BGP messages over semi-permanent TCP connection:
  - advertising *paths* to different destination network prefixes (BGP is a "path vector" protocol)
- when AS3 gateway 3a advertises path AS3,X to AS2 gateway 2c:
  - AS3 *promises* to AS2 it will forward datagrams towards X



Path attributes and BGP routes

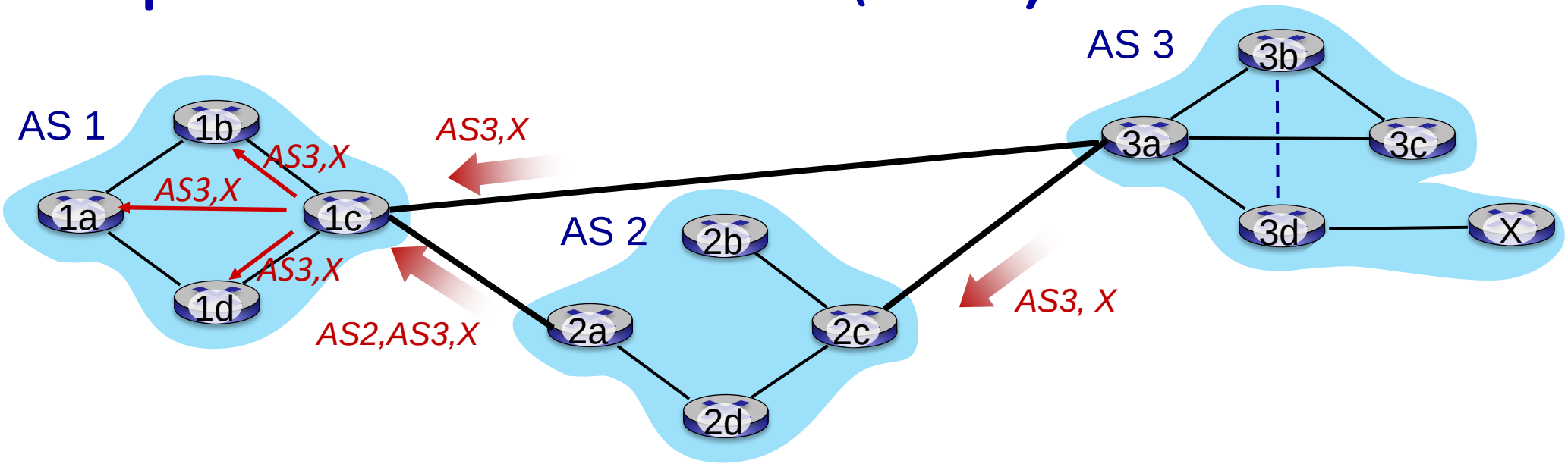
- BGP advertised route: prefix + attributes
  - prefix: destination being advertised
  - two important attributes:
    - AS-PATH: list of ASes through which prefix advertisement has passed
    - NEXT-HOP: indicates specific internal-AS router to next-hop AS
- policy-based routing:
  - gateway receiving route advertisement uses *import policy* to accept/decline path (e.g., never route through AS Y).
  - AS policy also determines whether to *advertise* path to other other neighboring ASes

BGP path advertisement



- AS2 router 2c receives path advertisement AS3,X (via eBGP) from AS3 router 3a
- based on AS2 policy, AS2 router 2c accepts path AS3,X, propagates (via iBGP) to all AS2 routers
- based on AS2 policy, AS2 router 2a advertises (via eBGP) path AS2, AS3, X to AS1 router 1c

BGP path advertisement (more)



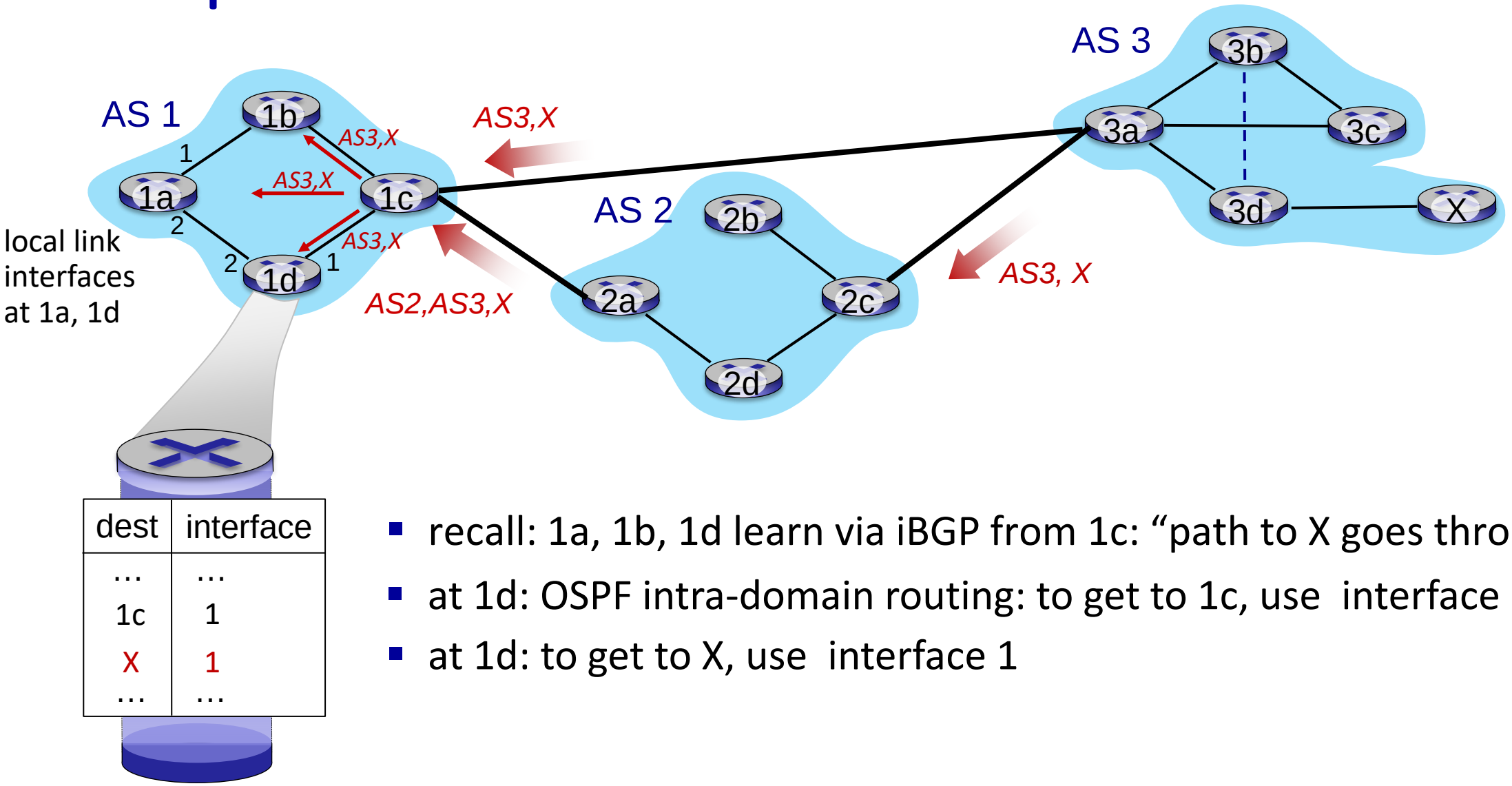
gateway router may learn about multiple paths to destination:

- AS1 gateway router 1c learns path *AS2,AS3,X* from 2a
- AS1 gateway router 1c learns path *AS3,X* from 3a
- based on *policy,* AS1 gateway router 1c chooses path *AS3,X* and advertises path within AS1 via iBGP

BGP messages

- BGP messages exchanged between peers over TCP connection
- BGP messages:
  - OPEN: opens TCP connection to remote BGP peer and authenticates sending BGP peer
  - UPDATE: advertises new path (or withdraws old)
  - KEEPALIVE: keeps connection alive in absence of UPDATES; also ACKs OPEN request
  - NOTIFICATION: reports errors in previous msg; also used to close connection

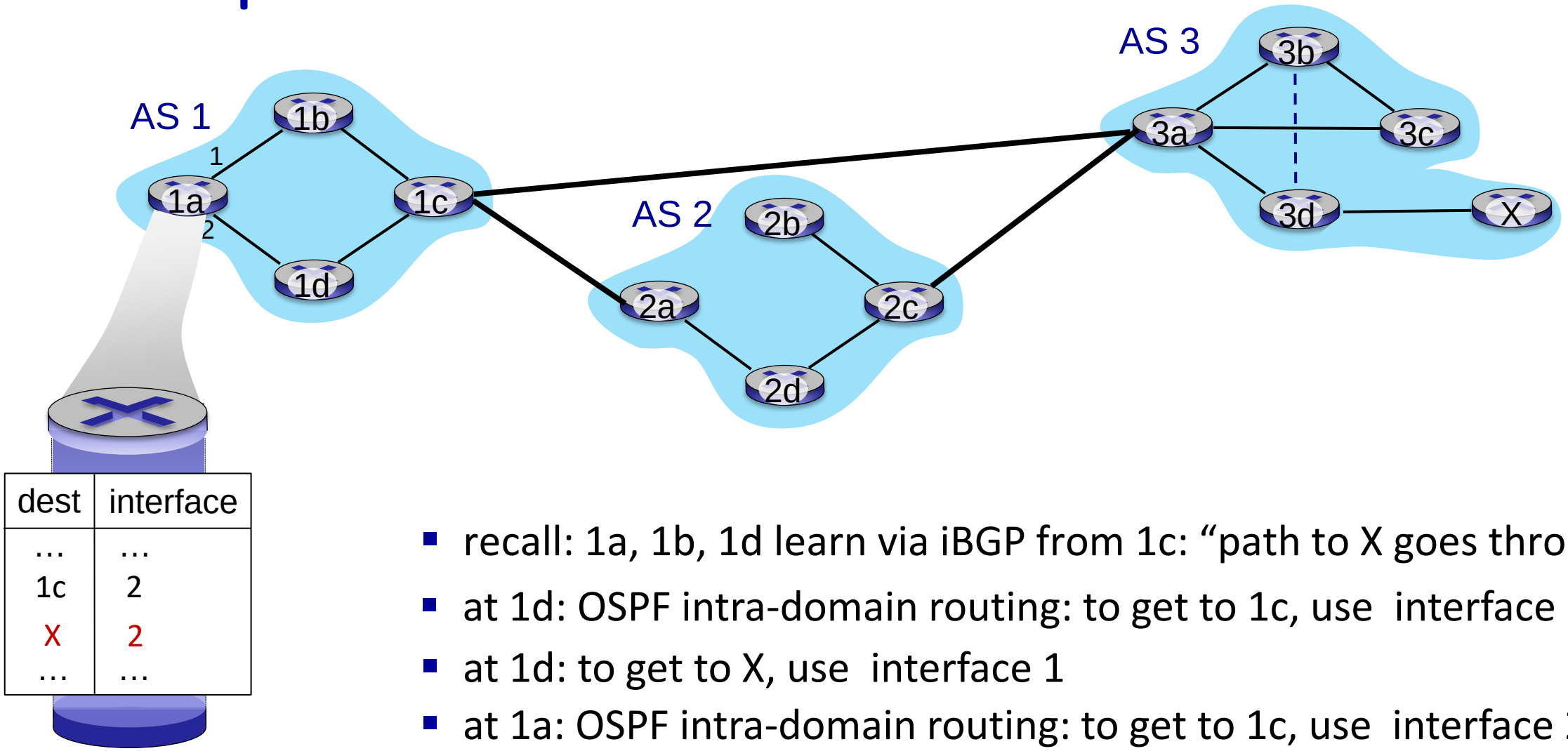
BGP path advertisement



dest	interface
…	…
1c	1
X	1
…	…

- recall: 1a, 1b, 1d learn via iBGP from 1c: "path to X goes through 1c"
- at 1d: OSPF intra-domain routing: to get to 1c, use interface 1
- at 1d: to get to X, use interface 1

BGP path advertisement



- recall: 1a, 1b, 1d learn via iBGP from 1c: “path to X goes through 1c”
- at 1d: OSPF intra-domain routing: to get to 1c, use interface 1
- at 1d: to get to X, use interface 1
- at 1a: OSPF intra-domain routing: to get to 1c, use interface 2
- at 1a: to get to X, use interface 2

Why different Intra-, Inter-AS routing ?

policy:

- inter-AS: admin wants control over how its traffic routed, who routes through its network
- intra-AS: single admin, so policy less of an issue

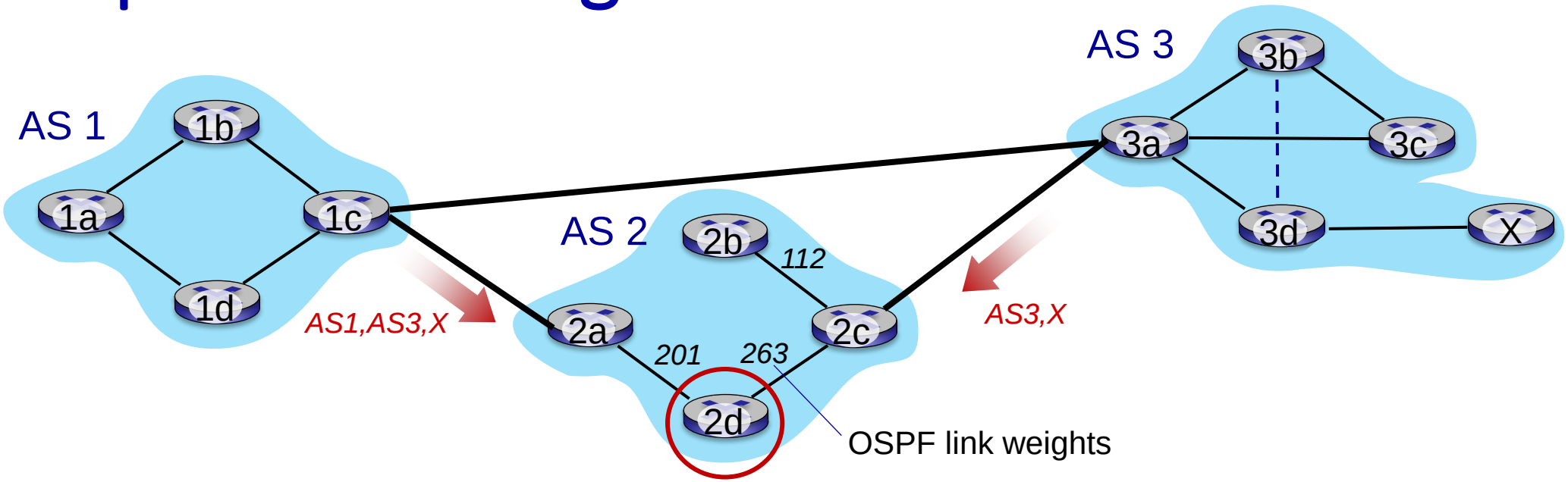
scale:

- hierarchical routing saves table size, reduced update traffic

performance:

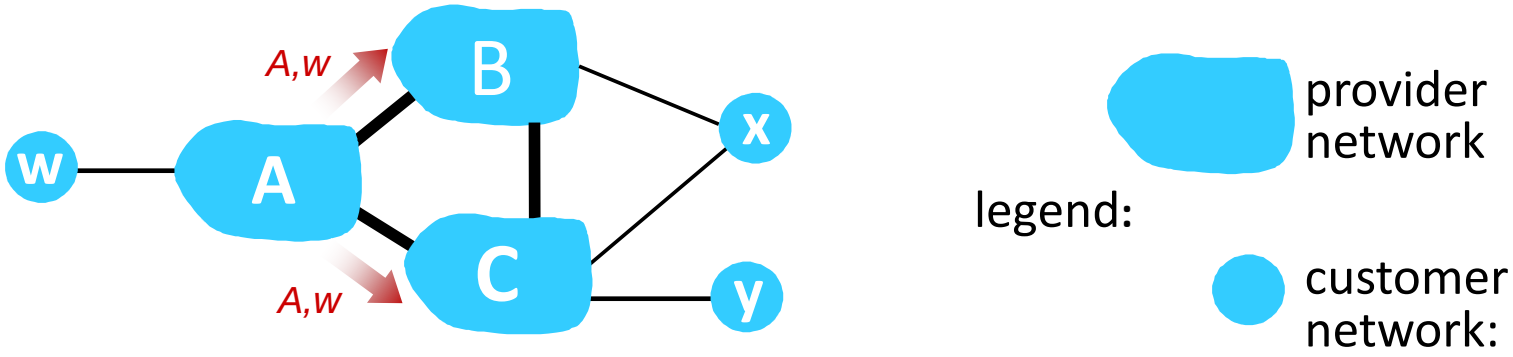
- intra-AS: can focus on performance
- inter-AS: policy dominates over performance

Hot potato routing



- 2d learns (via iBGP) it can route to X via 2a or 2c
- hot potato routing: choose local gateway that has least *intra-domain* cost (e.g., 2d chooses 2a, even though more AS hops to *X*): don't worry about inter-domain cost!

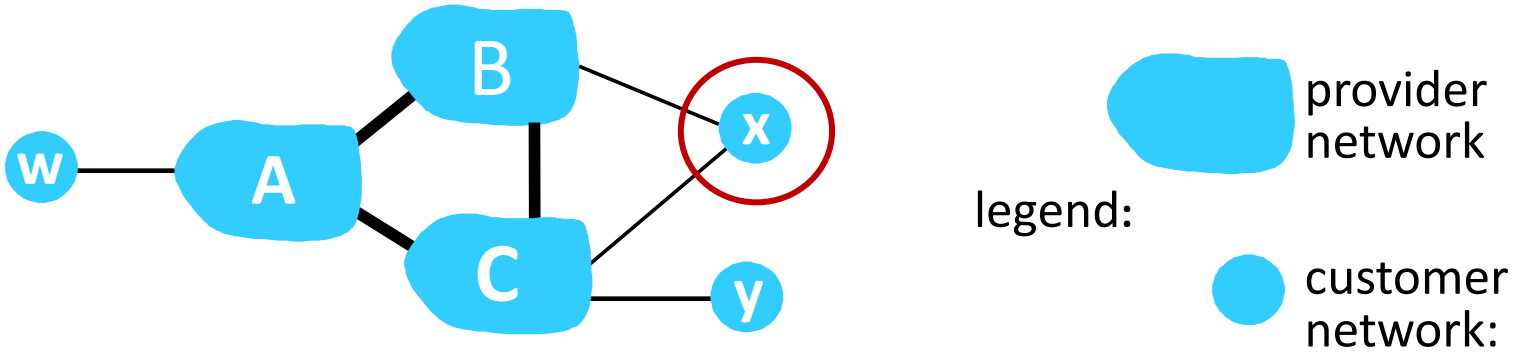
BGP: achieving policy via advertisements



ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs – a typical "real world" policy)

- A advertises path Aw to B and to C
- B *chooses not to advertise* BAw to C!
  - B gets no "revenue" for routing CBAw, since none of C, A, w are B's customers
  - C does *not* learn about CBAw path
- C will route CAw (not using B) to get to w

BGP: achieving policy via advertisements (more)



ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs – a typical "real world" policy)

- A,B,C are provider networks
- x,w,y are customer (of provider networks)
- x is dual-homed: attached to two networks
- *policy to enforce:* x does not want to route from B to C via x
  - .. so x will not advertise to B a route to C

BGP route selection

- router may learn about more than one route to destination AS, selects route based on:
  1. local preference value attribute: policy decision
  2. shortest AS-PATH
  3. closest NEXT-HOP router: hot potato routing
  4. additional criteria

Network layer: “control plane” roadmap

- introduction
- routing protocols
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol

- network management, configuration
  - SNMP
  - NETCONF/YANG

Network layer: “control plane” roadmap

- introduction
- routing protocols
  - link state
  - distance vector
- intra-ISP routing:
  - RIP
  - OSPF
- routing among ISPs: BGP
- **SDN control plane**

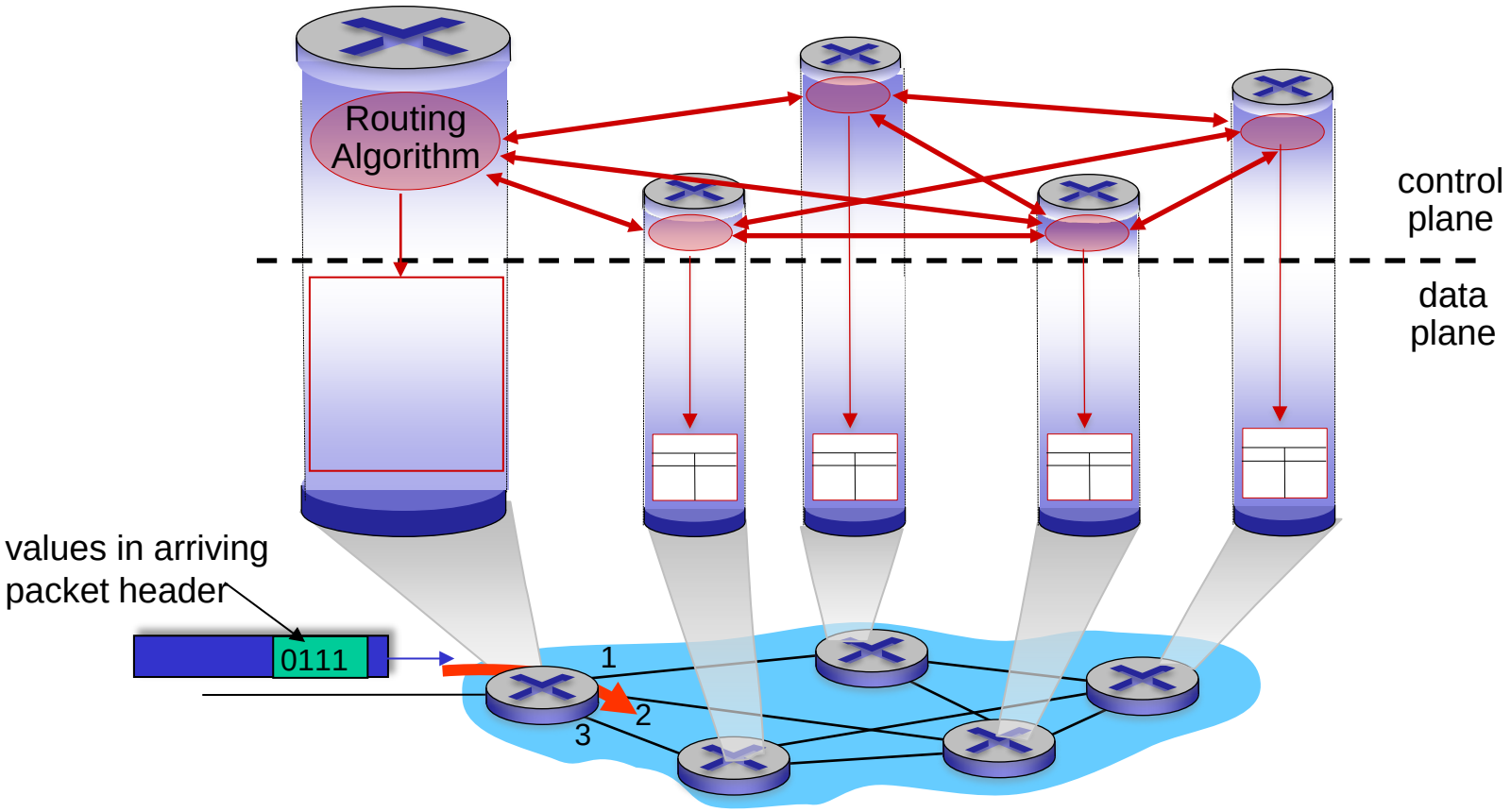
- network management, configuration
  - SNMP
  - NETCONF/YANG

Software defined networking (SDN)

- Internet network layer: historically implemented via distributed, per-router control approach:
  - *monolithic* router contains switching hardware, runs proprietary implementation of Internet standard protocols (IP, RIP, IS-IS, OSPF, BGP) in proprietary router OS (e.g., Cisco IOS)
  - different "middleboxes" for different network layer functions: firewalls, load balancers, NAT boxes, ..
- ~2005: renewed interest in rethinking network control plane

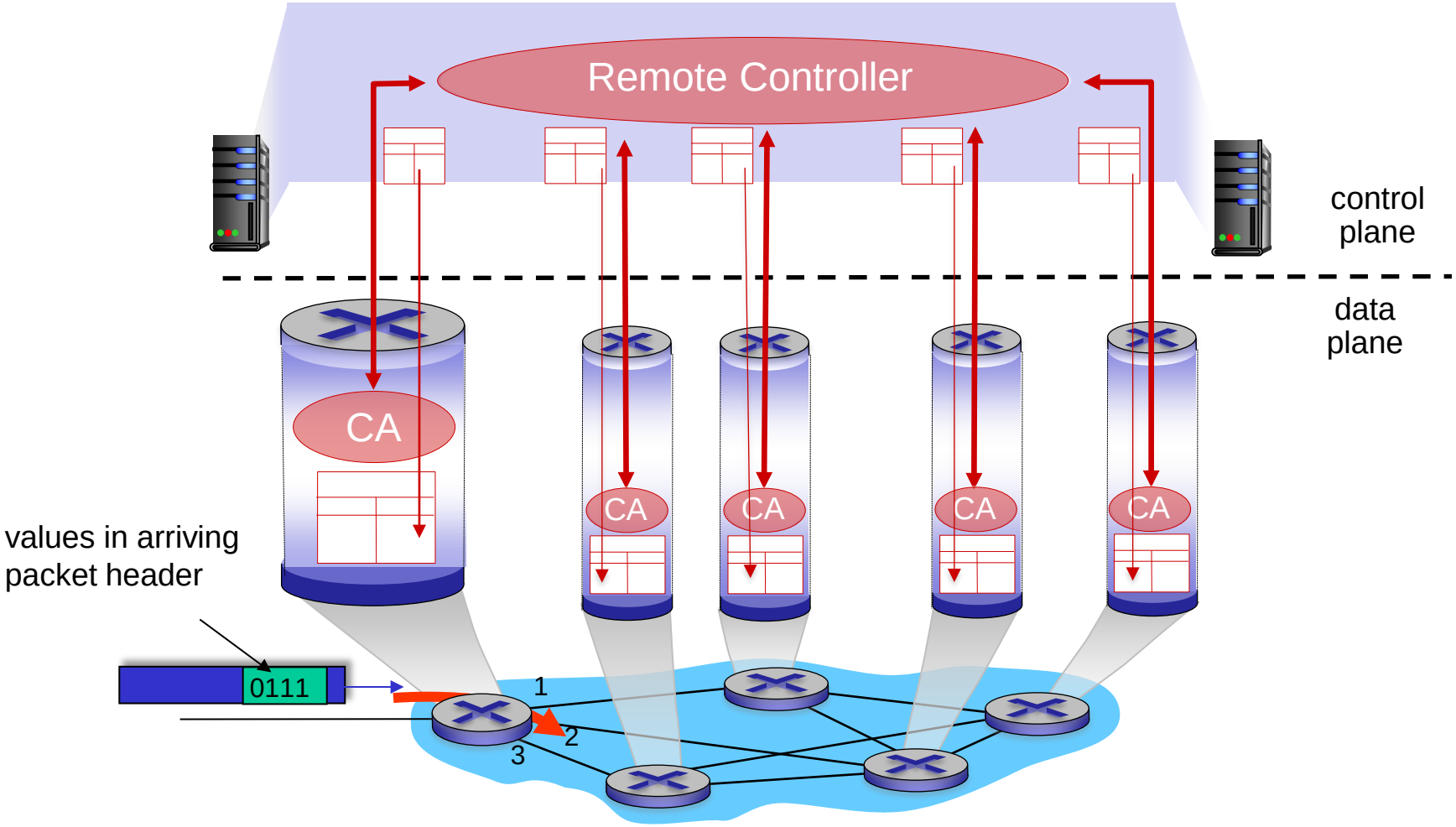
Per-router control plane

Individual routing algorithm components *in each and every router* interact in the control plane to computer forwarding tables



Software-Defined Networking (SDN) control plane

Remote controller computes, installs forwarding tables in routers

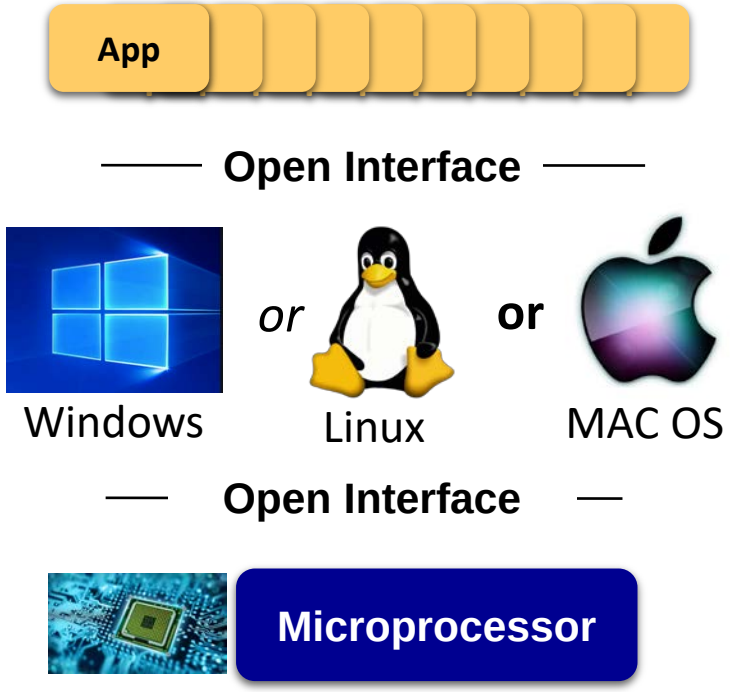


Software defined networking (SDN)

Why a *logically centralized* control plane?

- easier network management: avoid router misconfigurations, greater flexibility of traffic flows
- table-based forwarding (recall OpenFlow API) allows "programming" routers
  - centralized "programming" easier: compute tables centrally and distribute
  - distributed "programming" more difficult: compute tables as result of distributed algorithm (protocol) implemented in each-and-every router
- open (non-proprietary) implementation of control plane
  - foster innovation: let 1000 flowers bloom

SDN analogy: mainframe to PC revolution

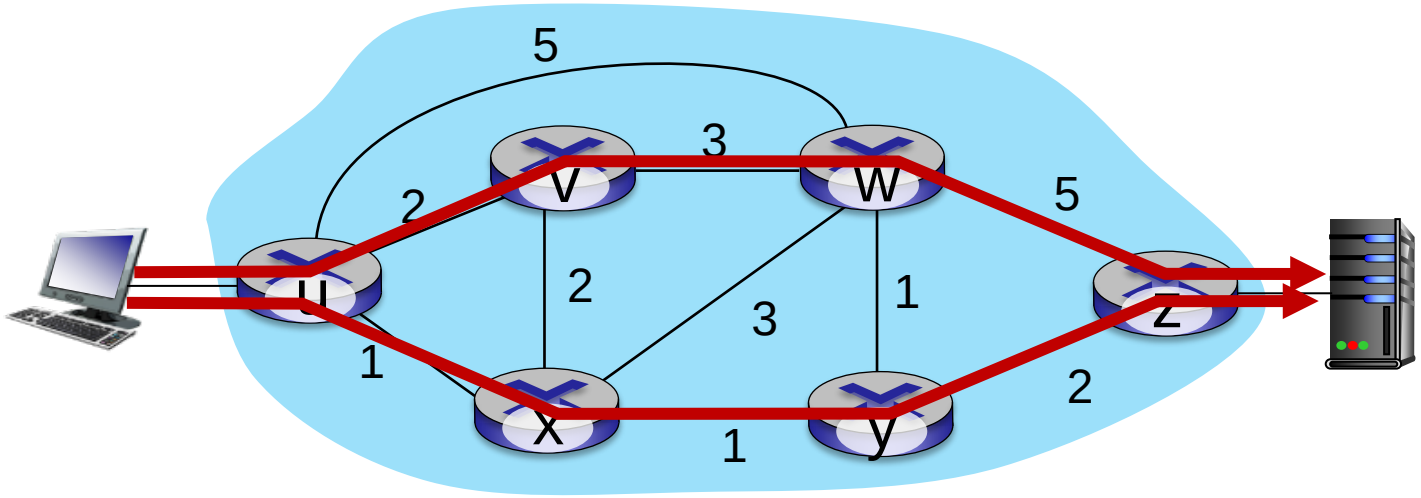


Vertically integrated
Closed, proprietary
Slow innovation
Small industry

Horizontal
Open interfaces
Rapid innovation
Huge industry

* Slide courtesy: N. McKeown

Traffic engineering: difficult with traditional routing

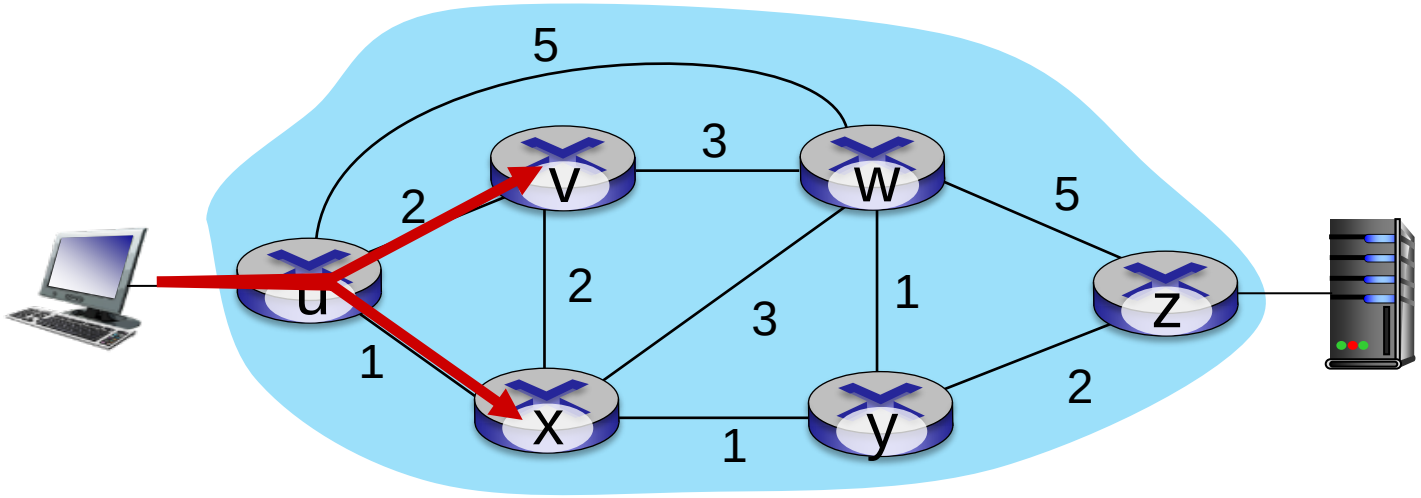


Q: what if network operator wants u-to-z traffic to flow along *uvwz*, rather than *uxyz*?

A: need to re-define link weights so traffic routing algorithm computes routes accordingly (or need a new routing algorithm)!

link weights are only control "knobs": not much control!

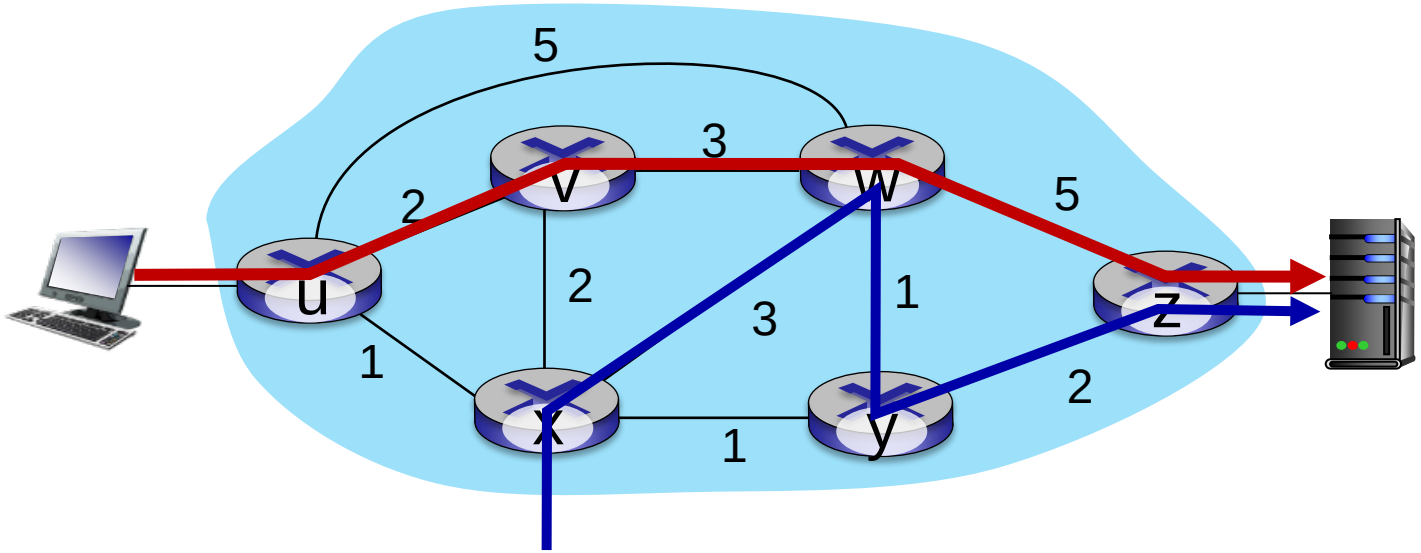
Traffic engineering: difficult with traditional routing



Q: what if network operator wants to split u-to-z traffic along uvwz *and* uxyz (load balancing)?

A: can't do it (or need a new routing algorithm)

Traffic engineering: difficult with traditional routing

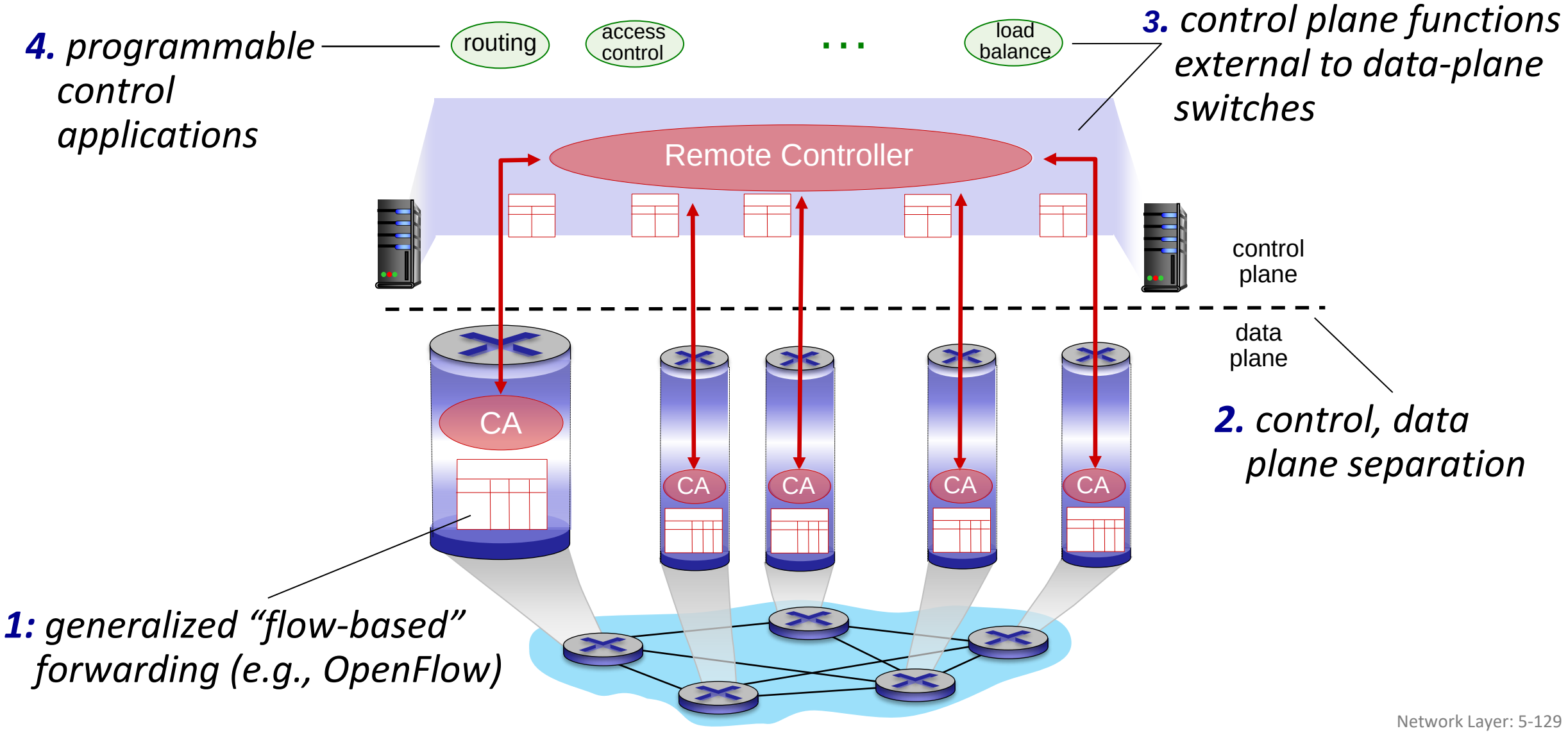


Q: what if w wants to route blue and red traffic differently from w to z?

A: can't do it (with destination-based forwarding, and LS, DV routing)

We learned in Chapter 4 that generalized forwarding and SDN can be used to achieve *any* routing desired

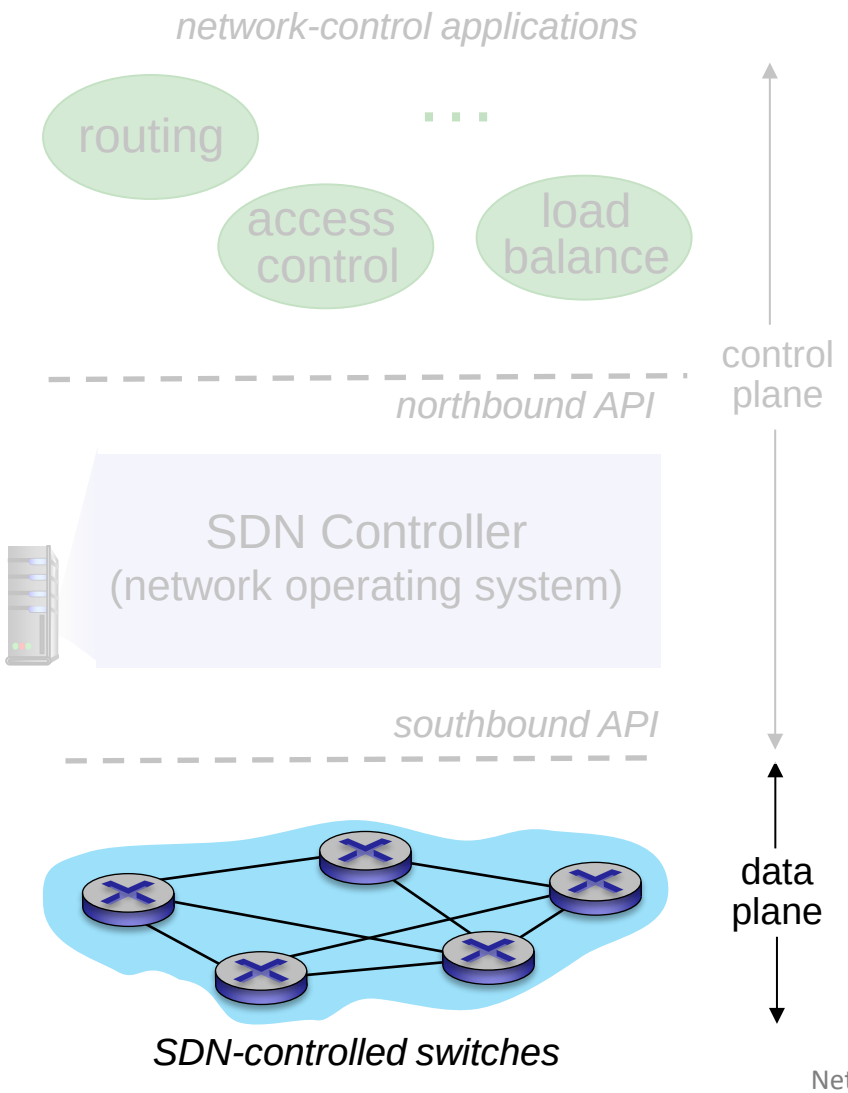
Software defined networking (SDN)



Software defined networking (SDN)

Data-plane switches:

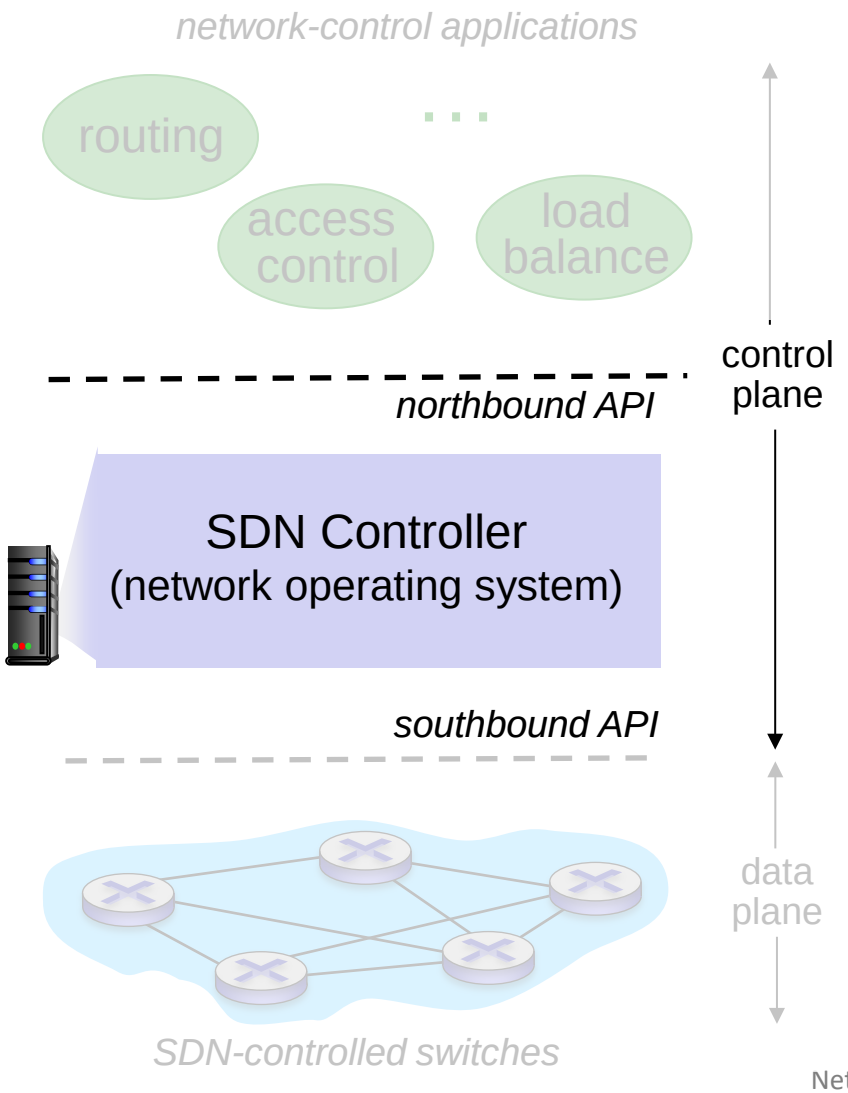
- fast, simple, commodity switches implementing generalized data-plane forwarding (Section 4.4) in hardware
- flow (forwarding) table computed, installed under controller supervision
- API for table-based switch control (e.g., OpenFlow)
  - defines what is controllable, what is not
- protocol for communicating with controller (e.g., OpenFlow)



Software defined networking (SDN)

SDN controller (network OS):

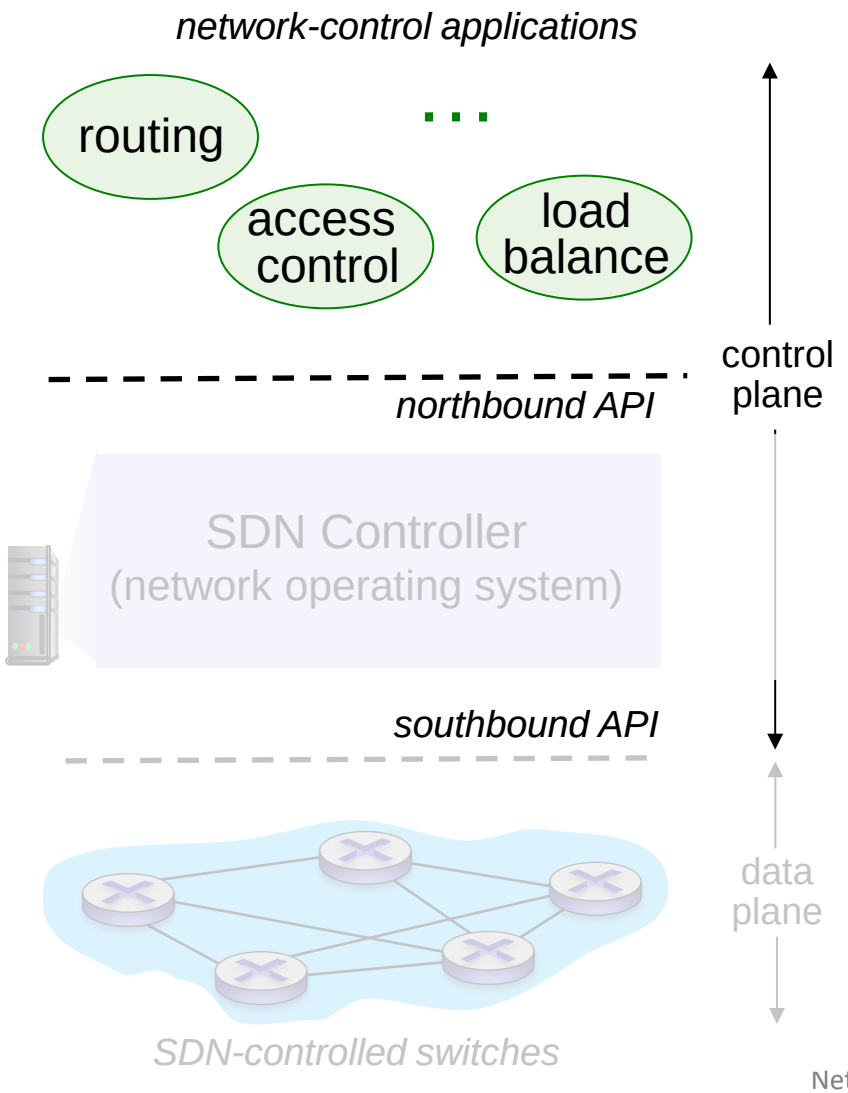
- maintain network state information
- interacts with network control applications "above" via northbound API
- interacts with network switches "below" via southbound API
- implemented as distributed system for performance, scalability, fault-tolerance, robustness



Software defined networking (SDN)

network-control apps:

- "brains" of control: implement control functions using lower-level services, API provided by SDN controller
- *unbundled:* can be provided by 3rd party: distinct from routing vendor, or SDN controller

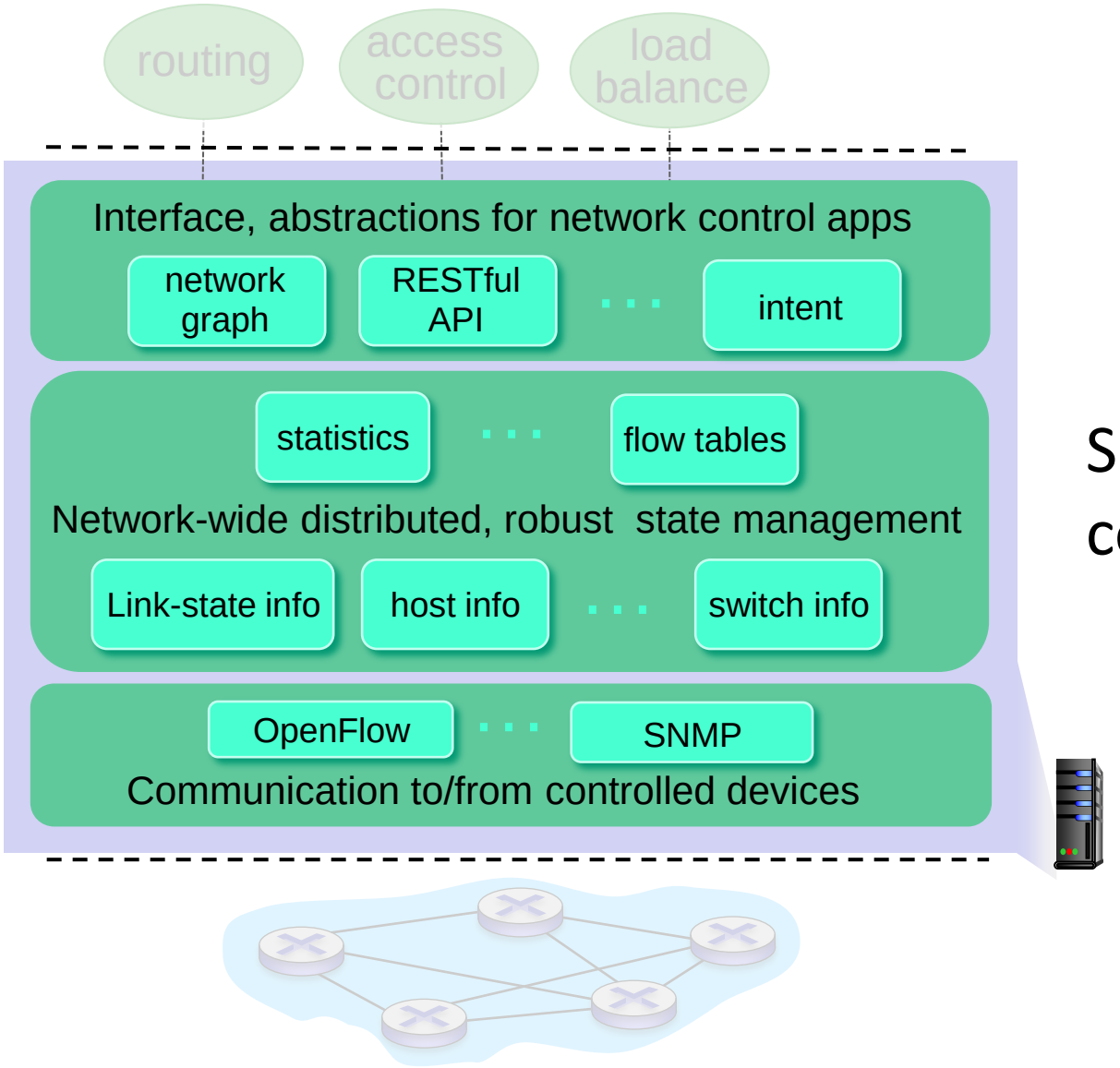


Components of SDN controller

interface layer to network control apps: abstractions API

network-wide state management : state of networks links, switches, services: a *distributed database*

communication: communicate between SDN controller and controlled switches



SDN controller

SDN: selected challenges

- hardening the control plane: dependable, reliable, performance-scalable, secure distributed system
  - robustness to failures: leverage strong theory of reliable distributed system for control plane
  - dependability, security: “baked in” from day one?
- networks, protocols meeting mission-specific requirements
  - e.g., real-time, ultra-reliable, ultra-secure
- Internet-scaling: beyond a single AS
- SDN critical in 5G cellular networks

SDN and the future of traditional network protocols

- SDN-computed versus router-computer forwarding tables:
  - just one example of logically-centralized-computed versus protocol computed
- one could imagine SDN-computed congestion control:
  - controller sets sender rates based on router-reported (to controller) congestion levels

How will implementation of network functionality (SDN versus protocols) evolve?

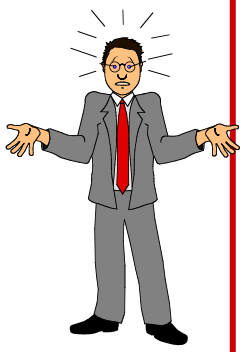
Network layer: “control plane” roadmap

- introduction
- routing protocols
  - link state
  - distance vector
- intra-ISP routing:
  - RIP
  - OSPF
- routing among ISPs: BGP
- SDN control plane

- **network management, configuration**
  - **SNMP**
  - **NETCONF/YANG**

What is network management?

- autonomous systems (aka “network”): 1000s of interacting hardware/software components
- other complex systems requiring monitoring, configuration, control:
  - jet airplane, nuclear power plant, others?

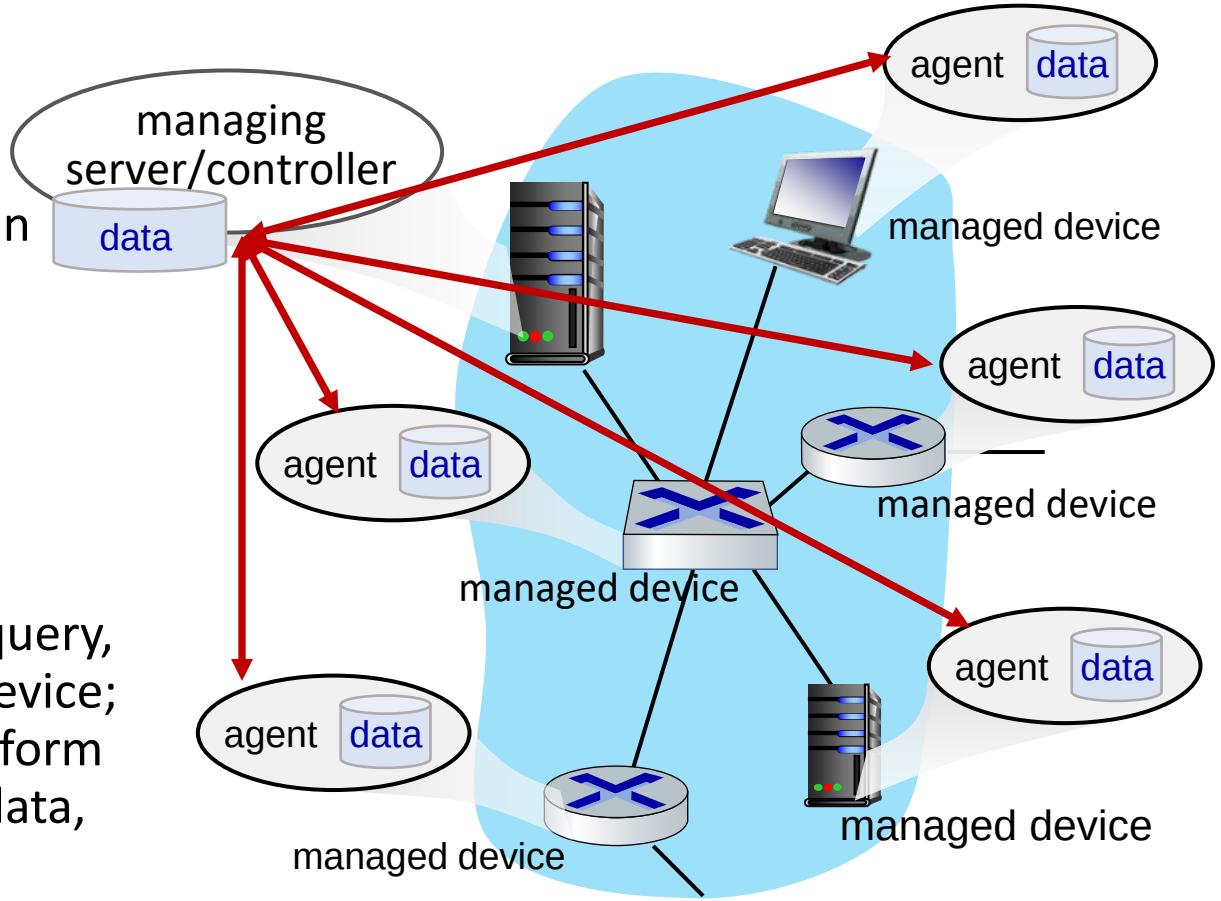


"Network management includes the deployment, integration and coordination of the hardware, software, and human elements to monitor, test, poll, configure, analyze, evaluate, and control the network and element resources to meet the real-time, operational performance, and Quality of Service requirements at a reasonable cost."

Components of network management

Managing server: application, typically with network managers (humans) in the loop

Network management protocol: used by managing server to query, configure, manage device; used by devices to inform managing server of data, events.



Managed device: equipment with manageable, configurable hardware, software components

Data: device "state" configuration data, operational data, device statistics

Network operator approaches to management

CLI (Command Line Interface)

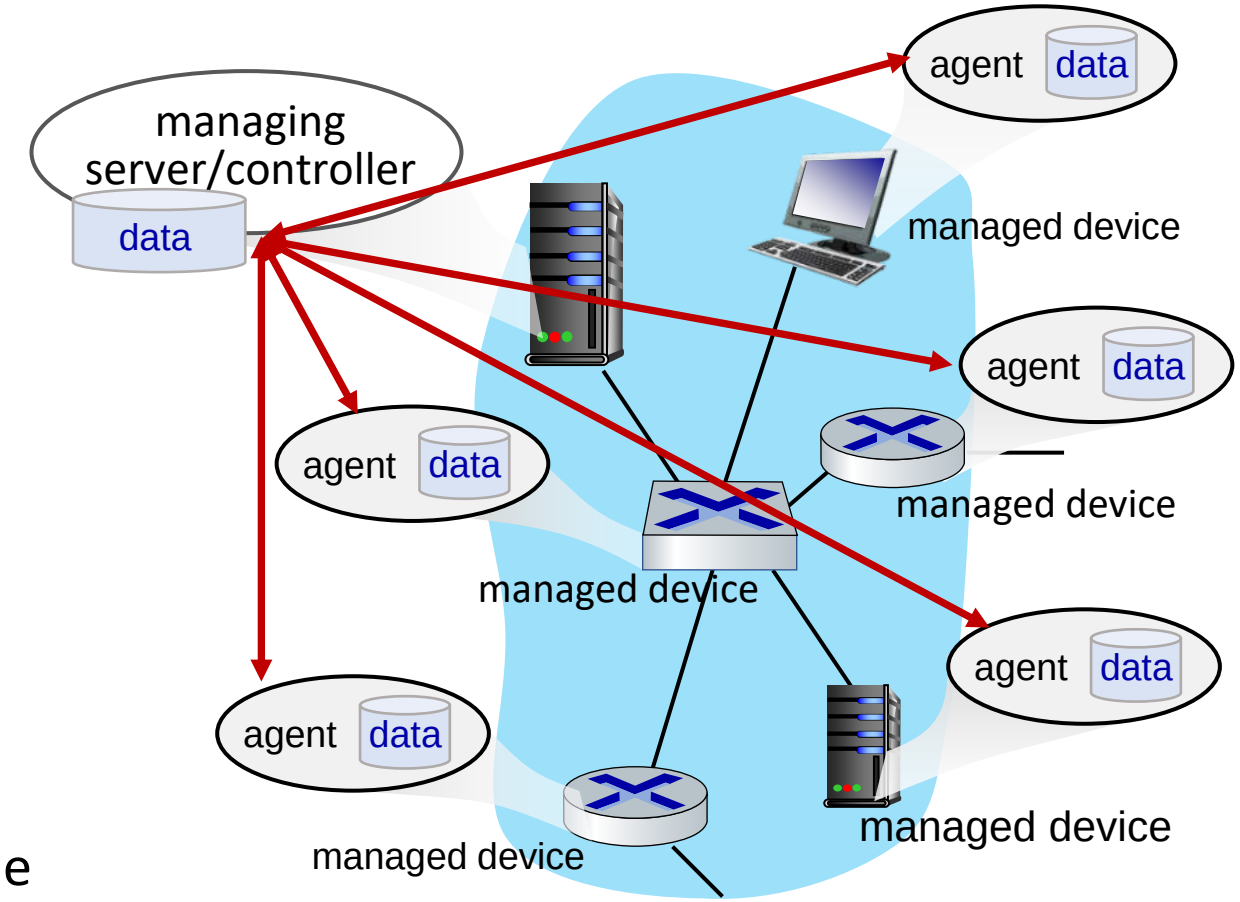
- operator issues (types, scripts) direct to individual devices (e.g., vis ssh)

SNMP/MIB

- operator queries/sets devices data (MIB) using Simple Network Management Protocol (SNMP)

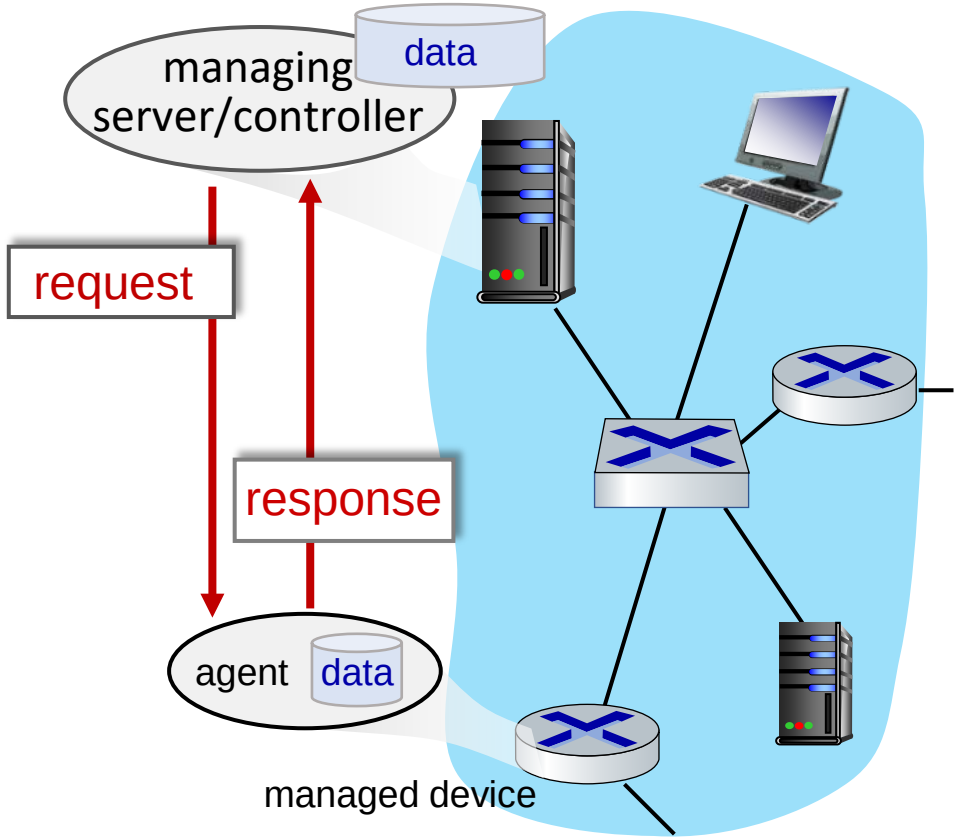
NETCONF/YANG

- more abstract, network-wide, holistic
- emphasis on multi-device configuration management.
- YANG: data modeling language
- NETCONF: communicate YANG-compatible actions/data to/from/among remote devices

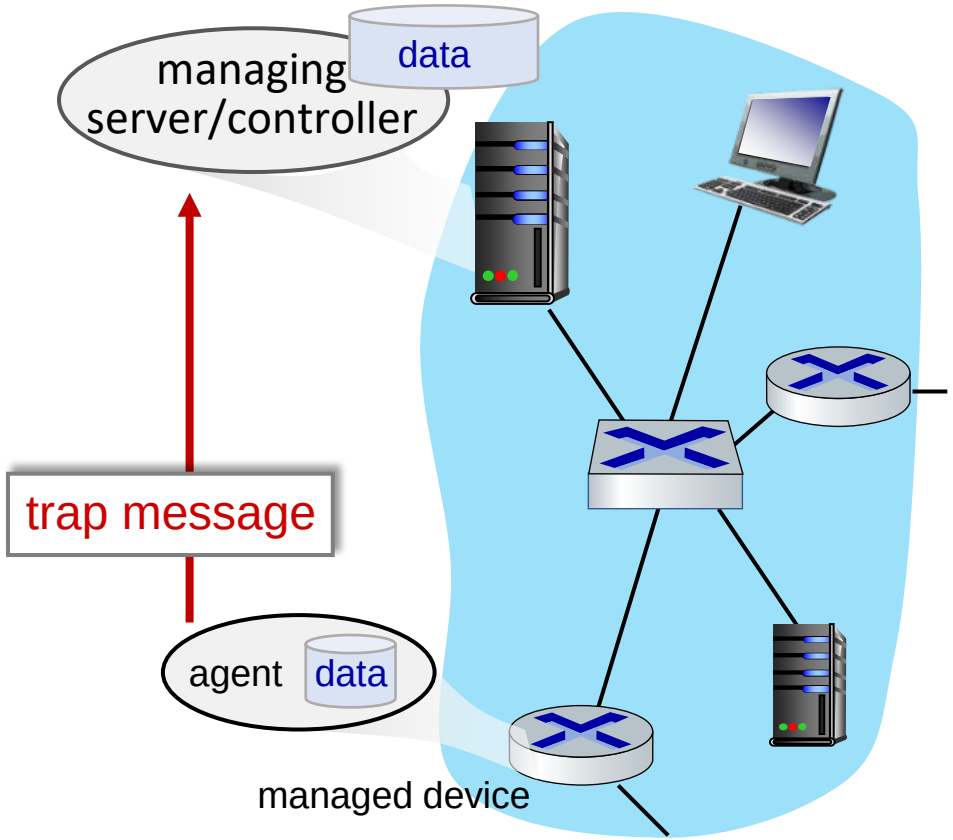


SNMP protocol

Two ways to convey MIB info, commands:



request/response mode

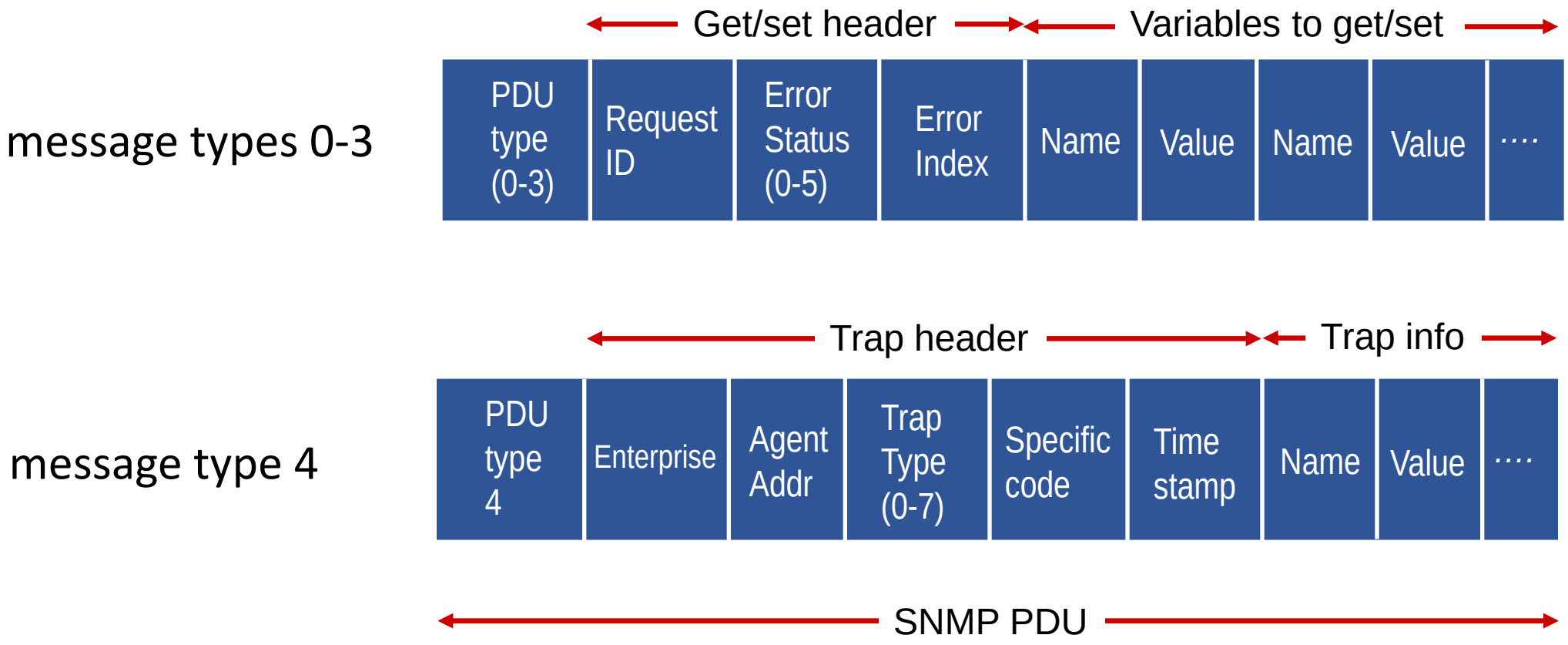


trap mode

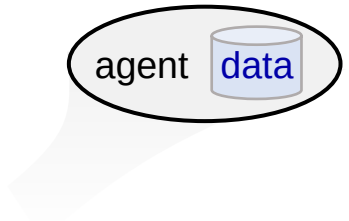
SNMP protocol: message types

Message type	Function
GetRequest GetNextRequest GetBulkRequest	manager-to-agent: "get me data" (data instance, next data in list, block of data).
SetRequest	manager-to-agent: set MIB value
Response	Agent-to-manager: value, response to Request
Trap	Agent-to-manager: inform manager of exceptional event

SNMP protocol: message formats



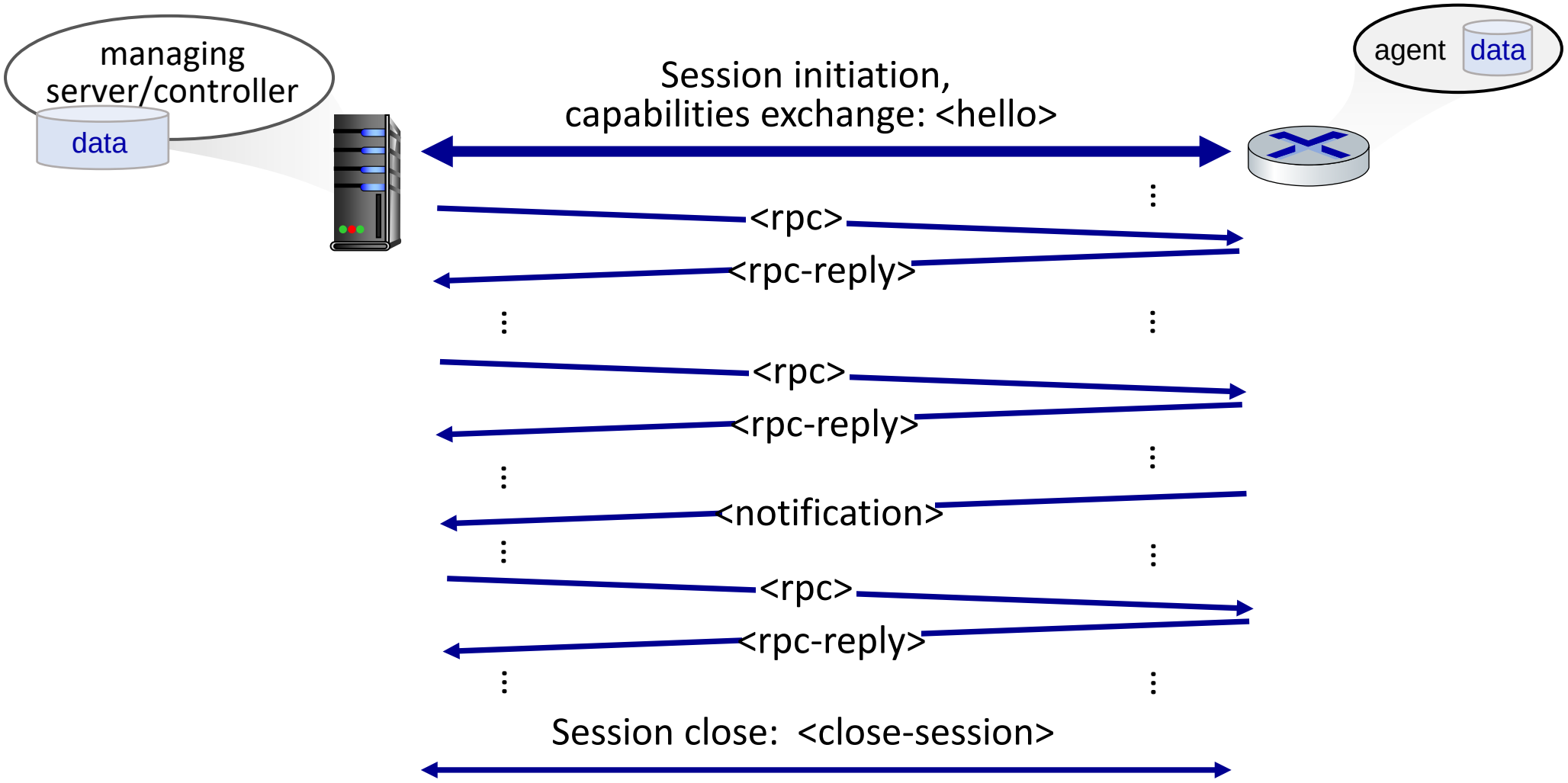
SNMP: Management Information Base (MIB)



- managed device's operational (and some configuration) data
- gathered into device MIB module
  - 400 MIB modules defined in RFC's; many more vendor-specific MIBs
- Structure of Management Information (SMI): data definition language
- example MIB variables for UDP protocol:

Object ID	Name	Type	Comments
1.3.6.1.2.1.7.1	UDPInDatagrams	32-bit counter	total # datagrams delivered
1.3.6.1.2.1.7.2	UDPNoPorts	32-bit counter	# undeliverable datagrams (no application at port)
1.3.6.1.2.1.7.3	UDInErrors	32-bit counter	# undeliverable datagrams (all other reasons)
1.3.6.1.2.1.7.4	UDPOutDatagrams	32-bit counter	total # datagrams sent
1.3.6.1.2.1.7.5	udpTable	SEQUENCE	one entry for each port currently in use

NETCONF initialization, exchange, close



Network layer: Summary

we've learned a lot!

- approaches to network control plane
  - per-router control (traditional)
  - logically centralized control (software defined networking)
- traditional routing algorithms
  - implementation in Internet: OSPF , BGP
- SDN controllers
  - implementation in practice: ODL, ONOS
- Internet Control Message Protocol
- network management

next stop: link layer!

Network layer, control plane: Done!

- introduction
- routing protocols
  - link state
  - distance vector
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol

- network management, configuration
  - SNMP
  - NETCONF/YANG

Additional Chapter 5 slides

Distance vector: another example

$D_x()$

from \ cost to	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

$D_y()$

from \ cost to	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

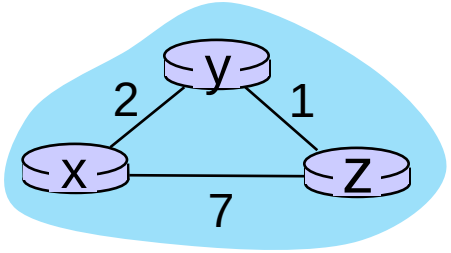
$D_z()$

from \ cost to	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0

from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	7	1	0

$D_x(z) = \min\{c_{x,y} + D_y(z),\ c_{x,z} + D_z(z)\}$
$= \min\{2+1\ ,\ 7+0\} = 3$

$D_x(y) = \min\{c_{x,y} + D_y(y),\ c_{x,z} + D_z(y)\}$
$= \min\{2+0\ ,\ 7+1\} = 2$



time

Distance vector: another example

$D_x()$

from \ cost to	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

$D_y()$

from \ cost to	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

$D_z()$

from \ cost to	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0

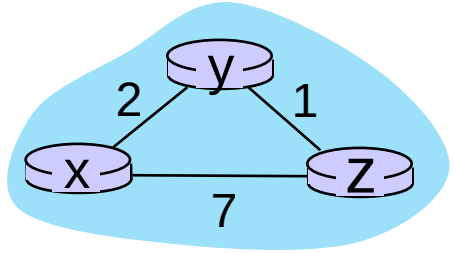
from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	7	1	0

from \ cost to	x	y	z
x	0	2	7
y	2	0	1
z	7	1	0

from \ cost to	x	y	z
x	0	2	7
y	2	0	1
z	3	1	0

from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

from \ cost to	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

time